Contents

CSSVD Flexible JavaScript API — Implementation and User Guide **3**

Flexible JavaScript API Analysis, Design, and Implementation Guide **4**

Executive summary . . . 4
1. Vocabulary and mental model . . . 5
2. Current system overview . . . 7
3. What exists today . . . 8
4. Why the current JS API is not enough . . . 10
5. Design principles . . . 11
6. Proposed API layers . . . 14
7. Replacing YAML concepts with JavaScript . . . 17
8. Proposed detailed API reference . . . 21
9. Architecture for implementation . . . 25
10. Goja adapter design with Proxy-backed handles . . . 32
11. Example usage patterns . . . 36
12. Implementation phases . . . 39
13. Testing strategy . . . 44
14. Error model . . . 46
15. Performance and concurrency guidance . . . 47
16. Migration strategy . . . 48
17. Risks and mitigations . . . 48
18. Acceptance criteria . . . 49
19. File-by-file implementation guide for an intern . . . 50
20. Review checklist . . . 52
21. References . . . 53
1. The Mental Model . . . 56
2. The Smallest Useful Script . . . 57
3. Turning a Function into a CLI Verb . . . 57
4. Reading What the Browser Actually Computed . . . 58
5. From Exploration to Repeatable Probes . . . 58
6. Snapshots: Capture the Facts You Care About . . . 59
7. Diffing Two States . . . 59
8. Writing Reports and Artifacts . . . 60
9. When to Use Inspect Artifacts Instead . . . 60
10. A Full Authoring Loop . . . 61
11. Debugging the Feedback Loop . . . 62
12. Key Points . . . 63
See Also . . . 63
Quick example . . . 64
Module exports . . . 65
Browser and page API . . . 65
Page methods . . . 66

Lower-level builders, extraction, snapshots, and diffs 69
Artifact formats 71
Catalog API 72
YAML interop 73
Error model 74
Concurrency guidance 74
Command namespace 76
Repository sources 76
Minimal verb file 77
Sentinels 77
Package metadata 77
Verb metadata 78
Fields and generated flags 78
Binding modes 79
Output modes 80
Built-in catalog commands 81
Duplicate command paths 81
Migration note 82
Replay and smoke scripts 82

Diary **83**
Goal 83
Step 1: Create the ticket and gather evidence 83
Step 2: Write the primary design guide 86
Step 3: Validate docs and skip reMarkable upload for now 88
Step 4: Retry reMarkable upload successfully with a short PDF filename 90
Step 5: Update design for jsapi subpackage, Go-backed Proxy objects, and no workflow builder 92
Step 6: Regenerate and upload the updated design guide to reMarkable 94
Step 7: Expand ticket tasks into granular implementation phases . . . 97
Step 8: Complete Phase 1 and Phase 2 implementation 99
Step 9: Complete Phase 3 page operation serialization 103
Step 10: Complete Phase 4 service DOM locator primitives 106
Step 11: Complete Phase 5 `page.locator()` JavaScript API 109
Step 12: Complete Phase 6 Go-backed target/probe/extractor builders 112
Step 13: Complete Phase 7 strict `cvd.extract(locator, extractors)` 116
Step 14: Complete Phase 8 strict `cvd.snapshot(page, probes, options)` 120
Step 15: Complete Phase 9 diff/report/write primitives 123
Step 16: Complete Phase 10 embedded help, textbook guide, examples, and smokes 126

Tasks **131**
Tracking rules 131
Phase 0 — Ticket, design, and baseline bookkeeping 131

Phase 1 — Move native API into `internal/cssvisualdiff/jsapi` with no behavior changes . . . 131
Phase 2 — Add Proxy infrastructure and typed unwrapping helpers . 133
Phase 3 — Add per-page operation serialization . . . 134
Phase 4 — Implement service DOM locator primitives . . . 134
Phase 5 — Expose `page.locator()` and locator methods as Go-backed Proxy handles . . . 135
Phase 6 — Add Go-backed target/probe/extractor builders . . . 135
Phase 7 — Add strict `cvd.extract(locator, extractors)` . . . 136
Phase 8 — Add strict `cvd.snapshot(page, probes, options)` . . . 136
Phase 9 — Add diff, report, and write primitives . . . 137
Phase 10 — Public docs, examples, smoke scripts, and delivery . . . 137

Changelog **138**
2026-04-24 . . . 138
2026-04-24 — Proxy-backed JS API architecture update . . . 138
2026-04-24 — Detailed implementation task breakdown . . . 139
2026-04-24 — Phase 1 and Phase 2 implementation . . . 139
2026-04-24 — Phase 3 page operation serialization . . . 139
2026-04-24 — Phase 4 service DOM locator primitives . . . 140
2026-04-24 — Phase 5 `page.locator()` JavaScript API . . . 140
2026-04-24 — Phase 6 Go-backed target/probe/extractor builders . . 141
2026-04-24 — Phase 7 strict `cvd.extract(locator, extractors)` . . 141
2026-04-24 — Phase 8 strict `cvd.snapshot(page, probes)` . . . 142
2026-04-24 — Phase 9 diff/report/write primitives . . . 142
2026-04-24 — Phase 10 embedded help, textbook guide, examples, and smokes . . . 143

CSSVD Flexible JavaScript API — Implementation and User Guide

Flexible JavaScript API Analysis, Design, and Implementation Guide

Executive summary

`css-visual-diff` now has a Promise-first JavaScript API exposed through `require("css-visual-diff")`, plus repository-scanned JavaScript verbs exposed under `css-visual-diff verbs ...`. That API is valuable, but it is still close to the high-level CLI and YAML workflows. A script can open a browser, load a page, preflight probes, inspect probes, and write catalog artifacts. In other words, the current API lets JavaScript **drive existing concepts**.

The next step is to let JavaScript **be the authoring model**. Instead of treating JavaScript as a thin wrapper around YAML targets, YAML sections, YAML styles, and CLI artifact modes, we should expose lower-level primitives that feel natural in JavaScript:

```
const cvd = require("css-visual-diff")

const browser = await cvd.browser()
const page = await browser.page("http://localhost:3000/booking", {
  viewport: cvd.viewport.desktop(),
})

const cta = page.locator(".booking-cta")

const snapshot = await cvd.extract(cta, [
  cvd.extractors.text(),
  cvd.extractors.bounds(),
  cvd.extractors.computedStyle(["color", "font-size", "background-color"]),
])

console.log(snapshot.text, snapshot.bounds, snapshot.computedStyle)
await browser.close()
```

The current API should stay. It is good for scripting the CLI workflow. The lower-level API proposed here should be additive. It should sit underneath the high-level API and eventually allow the CLI-style API to be implemented as a composition of lower-level primitives.

The desired end state is:

- YAML remains supported for stable, declarative, committed comparison plans.
- Current JS verbs remain supported for operator-facing commands and simple programmable catalogs.
- A new lower-level JS layer provides page locators, extraction pipelines, Go-backed fluent target/probe/extractor builders, snapshots, diffs, and

reporting. A workflow builder is intentionally out of scope for this design.
- Core browser, DOM, CSS, artifact, catalog, and diff logic remains in Go services.
- Goja adapters stay thin: decode JS objects, call Go services, return Promise-backed lowerCamel JS values, and construct typed JS-visible errors.

This document is written for a new intern. It explains the current system, why the current API is not enough, what the new API should look like, how it maps to Go services, and exactly how to implement it in phases.

---

1. Vocabulary and mental model

Before reading code, learn these terms. They are used throughout this design.

Target

A **target** is a browser page configuration: URL, viewport, wait time, optional root selector, and optional prepare step. In YAML this is `original` or `react`; in the lower-level JS API it should be authored through a Go-backed fluent builder such as `cvd.target("booking")`, not by passing arbitrary raw JS objects.

Current evidence:

- `internal/cssvisualdiff/config/config.go:25` defines `Target` with `Name`, `URL`, `WaitMS`, `Viewport`, `RootSelector`, and `Prepare`.
- `docs/js-api.md:84` documents `browser.page(url, options?)` as the current JS way to load a target-like page.

Prepare

A **prepare** step modifies the page after navigation and before inspection. Current prepare types are:

- `script`: run arbitrary JavaScript or a script file.
- `direct-react-global`: render a global React component into a controlled root.

Current evidence:

- `internal/cssvisualdiff/config/config.go:34` defines `PrepareSpec`.
- `internal/cssvisualdiff/service/prepare.go:14` implements `PrepareTarget`.
- `internal/cssvisualdiff/service/prepare.go:51` implements `RunScriptPrepare`.
- `internal/cssvisualdiff/service/prepare.go:64` implements `RunDirectReactGlobalPrepare`.

- `docs/js-api.md:143` documents `page.prepare(spec)` and explicitly states that `directReactGlobal` is a prepare/rendering mode.

Probe

A **probe** is an instruction to inspect one selector. Today it contains a name, selector, CSS properties, attributes, source, and required flag. In a lower-level API, a probe should be a reusable JavaScript value that can be built with functions.

Current evidence:

- `internal/cssvisualdiff/service/types.go:13` defines `ProbeSpec`.
- `docs/js-api.md:176` documents `page.preflight(probes)`.
- `docs/js-api.md:246` documents `page.inspectAll(probes, options)`.

Locator

A **locator** is the proposed lower-level abstraction for "an element query on a page." This is not implemented yet. It should be created with:

```
const hero = page.locator(".hero")
```

A locator should support low-level DOM/style/layout operations:

```
await hero.exists()
await hero.visible()
await hero.text()
await hero.bounds()
await hero.computedStyle(["display", "gap"])
await hero.screenshot({ path: "out/hero.png" })
```

This concept is deliberately similar to Playwright's `locator`, but it should remain much smaller and focused on visual inspection, not full browser testing.

Extractor

An **extractor** is a reusable description of data to collect from a locator. Examples:

```
cvd.extractors.text()
cvd.extractors.bounds()
cvd.extractors.computedStyle(["color", "font-size"])
cvd.extractors.html()
cvd.extractors.screenshot({ path: "out/cta.png" })
```

Extractors make inspection composable. Instead of hardcoding artifact modes into `inspectAll`, a script can ask for exactly the facts it needs.

Snapshot

A **snapshot** is the structured result of extracting facts from one or more locators/probes at a point in time. It should be JSON-serializable and suitable for diffing, reporting, or catalog storage.

Diff

A **diff** compares two snapshots. It should understand CSS values, text, attributes, bounds, artifacts, tolerances, and ignore rules.

Catalog

A **catalog** is a durable manifest/index of targets, preflight records, results, and failures. It already exists as a Go-backed service.

Current evidence:

- `internal/cssvisualdiff/service/catalog_service.go:28` defines `CatalogManifest`.
- `internal/cssvisualdiff/service/catalog_service.go:81` defines `NewCatalog`.
- `internal/cssvisualdiff/service/catalog_service.go:203` and `:217` write manifest and index files.
- `docs/js-api.md:284` documents the current JS catalog API.

---

2. Current system overview

At a high level, `css-visual-diff` is a Go CLI that drives Chromium, captures rendered DOM/CSS/PNG artifacts, and compares or reports on them.

```
flowchart TD
    User[Operator / CI] --> CLI[cmd/css-visual-diff/main.go]
    CLI --> Config[config.Load YAML]
    CLI --> Verbs[css-visual-diff verbs]
    Config --> Runner[runner / modes]
    Verbs --> Goja[go-go-goja runtime]
    Goja --> CVDModule[require("css-visual-diff")]
    CVDModule --> Services[internal/cssvisualdiff/service]
    Runner --> Services
    Services --> Driver[internal/cssvisualdiff/driver]
    Driver --> Chrome[chromedp / Chromium]
    Services --> Artifacts[PNG / HTML / CSS JSON / Markdown / inspect JSON]
    Services --> Catalog[manifest.json / index.md]
```

The current implementation has three important layers:

1. **CLI and YAML layer**: `cmd/css-visual-diff/main.go`, `internal/cssvisualdiff/config`, `internal/cssvisualdiff/runner`, and `internal/cssvisualdiff/modes`.
2. **JavaScript verb layer**: `internal/cssvisualdiff/verbcli` and `internal/cssvisualdiff/dsl` scan annotated JS files and expose them as CLI commands.
3. **Go service layer**: `internal/cssvisualdiff/service` owns reusable browser, prepare, preflight, inspect, style, and catalog behavior.

The lower-level JS API should plug into layers 2 and 3. It should not bypass them.

---

3. What exists today

3.1 Current high-level JS API

`docs/js-api.md:1` documents the current `require("css-visual-diff")` API. It exports:

```
const cvd = require("css-visual-diff")

await cvd.browser(options)
cvd.catalog(options)
await cvd.loadConfig(path)
cvd.CvdError
cvd.SelectorError
cvd.PrepareError
cvd.BrowserError
cvd.ArtifactError
```

The typical current flow is:

```
const browser = await cvd.browser()
const page = await browser.page(url, {
  viewport: { width: 1280, height: 720 },
})

const probes = [
  { name: "cta", selector: "#cta", props: ["color", "font-size"] },
]

const statuses = await page.preflight(probes)
const result = await page.inspectAll(probes, {
  outDir: "/tmp/cssvd/artifacts/cta",
  artifacts: "css-json",
})

await browser.close()
```

This is already useful. It is Promise-first, works from repository-scanned verbs, writes artifacts, and integrates with the Go-backed catalog.

3.2 Current JS implementation

`internal/cssvisualdiff/dsl/cvd_module.go:16` registers the native module. It installs typed errors, then exposes `catalog`, `loadConfig`, and `browser`.

Important implementation details:

- `promiseValue(...)` at `internal/cssvisualdiff/dsl/cvd_module.go:75` creates Goja Promises and posts resolution/rejection back to the go-go-goja runtime owner thread.
- `cvdErrorValue(...)` at `internal/cssvisualdiff/dsl/cvd_module.go:96` constructs typed JS-visible errors.
- `wrapBrowser(...)` at `internal/cssvisualdiff/dsl/cvd_module.go:128` exposes `browser.newPage`, `browser.page`, and `browser.close`.
- `wrapPage(...)` around `internal/cssvisualdiff/dsl/cvd_module.go:193` exposes `page.goto`, `page.prepare`, `page.preflight`, `page.inspect`, `page.inspectAll`, and `page.close`.
- `decodePrepareSpec(...)` around `internal/cssvisualdiff/dsl/cvd_module.go:335` maps lowerCamel JS prepare options to Go `config.PrepareSpec`.
- `decodeProbes(...)` around `internal/cssvisualdiff/dsl/cvd_module.go:385` maps JS probe objects to `service.ProbeSpec`.

This file is the correct place to add the first lower-level module methods, but it should not grow indefinitely. As the lower-level API expands, split it into focused adapter files.

3.3 Current JS verbs implementation

`docs/js-verbs.md:1` documents repository-scanned JavaScript verbs. A JS file can declare:

```
function hello(name) {
  return `hello ${name}`
}

__verb__("hello", {
  parents: ["custom"],
  fields: {
    name: { argument: true, required: true }
  }
})
```

Then the CLI can run it as:

```
css-visual-diff verbs --repository ./verbs custom hello Manuel
```

Implementation evidence:

- `internal/cssvisualdiff/verbcli/command.go:16` defines the lazy `verbs` command.
- `internal/cssvisualdiff/verbcli/bootstrap.go:26` defines `Bootstrap`.
- `internal/cssvisualdiff/verbcli/bootstrap.go:30` defines `Repository`.
- `internal/cssvisualdiff/verbcli/bootstrap.go:276` scans repositories.
- `internal/cssvisualdiff/verbcli/bootstrap.go:298` detects duplicate verb paths.
- `internal/cssvisualdiff/dsl/host.go:34` builds the runtime factory with `require()` loader support and native module registrars.

This matters because the lower-level JS API is not just an internal scripting surface. It should be usable from operator-facing verbs with generated flags.

3.4 Current YAML concepts

The current YAML schema is in `internal/cssvisualdiff/config/config.go`:

- `Target` at `:25`.
- `PrepareSpec` at `:34`.
- `Viewport` at `:50`.
- `SectionSpec` at `:55`.
- `StyleSpec` at `:86`.
- `OutputSpec` at `:96`.
- `Config` at `:106`.

The lower-level JS API should make these concepts expressible as JavaScript values and functions. It does not need to delete YAML. Instead, it should let users stop writing YAML when YAML becomes an awkward programming language.

---

4. Why the current JS API is not enough

The current JS API is close to the CLI workflow. It is centered on:

```
await page.preflight(probes)
await page.inspectAll(probes, { outDir, artifacts })
```

That shape is good when the user already thinks in terms of "probes" and "artifact bundles." It is less ideal for more flexible tasks:

- Query an element's text without writing any artifact files.
- Read bounds for several selectors and compute a custom layout assertion.
- Extract only text and a few CSS properties from many elements.
- Compare two states in memory before deciding whether to write a report.
- Build targets/probes from ordinary JS functions rather than YAML or object literals.

- Compose custom extraction pipelines.
- Use locators as reusable handles.
- Use ordinary JavaScript functions and branches for authoring mode vs CI mode, without generating intermediate YAML and without adding a special workflow builder abstraction.

Current examples are high-level and file-artifact oriented. For example, `docs/js-api.md:246` documents `inspectAll` as writing artifacts below `outDir`. That is correct, but a lower-level API should support "extract data now, maybe write later."

The core gap is this:

```
Current API:    JavaScript controls CLI-like operations.
Desired API:    JavaScript composes browser inspection primitives.
```

---

5. Design principles

Principle 1: Keep browser and artifact logic in Go services

Do not implement CSS extraction, screenshotting, catalog writing, or diffing as large JavaScript helper libraries. JavaScript should orchestrate. Go should own the stable, tested implementation.

Good boundary:

```
JS script                 describes intent and control flow
Go-backed Proxy object    validates fluent API usage and provides LLM-friendly errors
Goja adapter              unwraps typed Go handles/builders and returns Promises
Go service                performs browser/DOM/CSS/artifact/diff work
Driver                    talks to chromedp
```

Bad boundary:

```
JS helper library         reimplements DOM extraction, artifact layout, diff logic, and cata
Raw JS objects            flow through the API until a late map decode fails with generic er
Go adapter                only provides raw page.evaluate
```

Principle 2: Use Go-backed Proxy objects for live handles and DSL builders

The lower-level API should not be a collection of arbitrary plain JavaScript objects. It should use Go structs wrapped with Goja Proxy traps, following the UI DSL pattern from `/home/manuel/code/wesen/2026-04-20--js-discord-bot`.

Use Go-backed Proxy wrappers for objects that have behavior or validation:

- `BrowserHandle`
- `PageHandle`

- `LocatorHandle`
- `TargetBuilder`
- `ProbeBuilder`
- `ExtractorSpec` / extractor builders
- `CatalogHandle`

This lets Go control:

- which methods exist,
- wrong-parent errors,
- unknown-method errors,
- argument validation,
- typed unwrapping,
- situation-specific feedback for LLM-written scripts.

Example desired error:

```
cvd.locator: .styles() is not available here. .styles() belongs to cvd.probe().
For direct style reads on a locator, use .computedStyle(["color", ...]).
```

That is much better than:

```
TypeError: locator.styles is not a function
```

Principle 3: Builders are synchronous; browser/file work is Promise-based

A fluent Go-backed builder can be synchronous because it only mutates or validates a Go struct:

```
const target = cvd.target("booking").url(baseUrl + "/booking").viewport(1440, 900)
```

But anything that talks to Chromium, waits, reads files, writes files, or mutates catalog files should return a Promise:

```
const page = await browser.page(target.url(), { viewport: target.viewport() })
const snap = await cvd.extract(page.locator("#cta"), extractors)
await catalog.writeManifest()
```

This matches the current Promise-first rule documented in `docs/js-api.md:1` and implemented by `promiseValue(...)` in `internal/cssvisualdiff/dsl/cvd_module.go:75`.

Principle 4: Do not accept raw JS object specs in the new lower-level API

The existing high-level API can keep accepting raw probe objects for compatibility:

```
await page.inspectAll([{ name: "cta", selector: "#cta", props: ["color"] }], opts)
```

The new lower-level API should prefer strict Go-backed values:

```
await cvd.snapshot(page, [
  cvd.probe("cta").selector("#cta").text().bounds().styles(["color"]),
])
```

If a user or LLM passes a raw object to a strict API, return a precise error:

```
cvd.snapshot: expected cvd.probe() builders, got object at probes[0].
Use cvd.probe("cta").selector("#cta").text() instead.
```

YAML and raw-object interop should be explicit through conversion helpers such as `cvd.probes.fromConfig(...)`, not implicit everywhere.

Principle 5: Locators should be the ergonomic primitive

`page.locator(selector)` should be the natural entry point for low-level operations. It gives the user a stable page-bound Go-backed handle to pass to extractors, assertions, snapshots, and screenshots.

A locator is not a builder. It is a live page-bound handle. Invalid builder-style usage should receive contextual feedback:

```
cvd.locator: .selector() is not available here. A locator is already bound to a selector.
Use page.locator("#other") to create another locator.
```

Principle 6: Results should be plain JSON-serializable data

Use Go-backed Proxy objects for live handles and authoring builders, but return plain serializable data for final results:

- selector statuses,
- element snapshots,
- page snapshots,
- diffs,
- catalog manifests,
- report summaries.

Every extracted result should be easy to persist, diff, and inspect:

```
await cvd.write.json("out/snapshot.json", snapshot)
```

So the rule is:

```
Objects with behavior / validation: Go-backed Proxy wrappers.
Objects that are final data/results: plain JS values.
```

Principle 7: YAML interop remains important, but should be explicit

`cvd.loadConfig(path)` already exists and is documented at `docs/js-api.md:371`. The lower-level API should add helpers to convert YAML targets/styles/sections into Go-backed JS target/probe builders or built specs. That gives users a migration path without weakening the strict lower-level API.

Principle 8: Per-page operations should be serialized by default

`docs/js-api.md:428` already warns that per-page CDP operations are effectively serialized. A lower-level API makes it tempting to run many locator operations concurrently on one page. The implementation should guard against that by adding a per-page operation queue in `PageHandle` or `service.PageService`.

---

6. Proposed API layers

The lower-level API should be built as layers. Each layer can ship independently. There is intentionally **no workflow builder layer** in this design. Workflow builders can look elegant, but they hide where browser work happens and they make LLM-generated code harder to debug. Keep the API explicit: handles, builders, extraction, snapshots, diffs, catalog/reporting.

```
flowchart TD
    A[Layer 1: Go-backed browser/page/locator handles]
    B[Layer 2: Go-backed target/probe/extractor builders]
    C[Layer 3: Extraction and snapshots]
    D[Layer 4: Diff and comparison]
    E[Layer 5: Catalog/report/write integration]

    A --> C
    B --> C
    C --> D
    C --> E
    D --> E
```

Layer 1: Go-backed browser, page, and locator handles

The existing `browser` and `page` values should move toward Proxy-backed handles. New locator values should be Proxy-backed from the beginning.

```typescript
interface Page {
  locator(selector: string, options?: LocatorOptions): Locator
  evaluate<T = any>(script: string, ...args: any[]): Promise<T>
  html(options?: HtmlOptions): Promise<string>
  screenshot(options?: PageScreenshotOptions): Promise<ScreenshotResult>
}

interface Locator {
  exists(): Promise<boolean>
  visible(): Promise<boolean>
  status(): Promise<SelectorStatus>
  text(options?: TextOptions): Promise<string>
  html(options?: HtmlOptions): Promise<string>
```

```
  outerHTML(options?: HtmlOptions): Promise<string>
  bounds(): Promise<Bounds | null>
  computedStyle(props?: string[]): Promise<Record<string, string>>
  attributes(names?: string[]): Promise<Record<string, string | null>>
  screenshot(options: LocatorScreenshotOptions): Promise<ScreenshotResult>
}
```

Example:

```
const page = await browser.page(url, { viewport: { width: 390, height: 844 } })
const nav = page.locator("nav")

if (await nav.visible()) {
  console.log(await nav.bounds())
  console.log(await nav.computedStyle(["display", "height", "background-color"]))
}
```

A locator is a page-bound handle, not a reusable recipe. It answers: "which element on this loaded page?"

Layer 2: Go-backed target, probe, and extractor builders

Builders should be Go structs wrapped in Goja Proxies, not raw JavaScript objects. They should validate arguments at the point of fluent method calls and produce precise errors for wrong property access.

```
interface CVD {
  target(name: string): TargetBuilder
  probe(name: string): ProbeBuilder
  viewport: ViewportFactory
  extractors: ExtractorFactory
}
```

Example:

```
const target = cvd.target("booking-mobile")
  .url(`${baseUrl}/booking`)
  .viewport("iphone14")
  .root("#app")
  .waitMs(250)

const ctaProbe = cvd.probe("primary-cta")
  .selector(".booking-cta")
  .text()
  .bounds()
  .styles(["color", "font-size", "background-color", "border-radius"])

const extractors = [
  cvd.extractors.text(),
```

```
  cvd.extractors.bounds(),
  cvd.extractors.computedStyle(["color", "font-size"]),
]
```

Builder methods should not talk to the browser. They only mutate and validate Go-owned state. `.build()` can be exposed for debugging/serialization, but strict APIs should also be able to unwrap builder proxies directly.

Layer 3: Extraction and snapshots

New exports:

```
interface CVD {
  extract(locator: Locator, extractors: Extractor[], options?: ExtractOptions): Promise<Elem
  snapshot(page: Page, probes: ProbeBuilder[], options?: SnapshotOptions): Promise<PageSnaps
}
```

`cvd.extract(...)` should require a Go-backed locator and Go-backed extractors. `cvd.snapshot(...)` should require a Go-backed page handle and Go-backed probe builders or built probe specs.

Example:

```
const ctaSnapshot = await cvd.extract(page.locator("#cta"), [
  cvd.extractors.exists(),
  cvd.extractors.visible(),
  cvd.extractors.text(),
  cvd.extractors.bounds(),
  cvd.extractors.computedStyle(["color", "font-size", "background-color"]),
])
```

Expected result shape is plain data:

```
{
  "selector": "#cta",
  "exists": true,
  "visible": true,
  "text": "Book now",
  "bounds": { "x": 120, "y": 400, "width": 180, "height": 48 },
  "computedStyle": {
    "color": "rgb(255, 255, 255)",
    "font-size": "16px",
    "background-color": "rgb(0, 102, 255)"
  },
  "artifacts": {}
}
```

Layer 4: Diff and comparison

New exports:

```
interface CVD {
  diff(left: Snapshot, right: Snapshot, options?: DiffOptions): DiffResult
}
```

Example:

```
const before = await cvd.snapshot(originalPage, probes)
const after = await cvd.snapshot(reactPage, probes)

const diff = cvd.diff(before, after, {
  ignore: ["metadata.createdAt", "bounds.x", "bounds.y"],
  tolerances: {
    "bounds.width": 2,
    "bounds.height": 2,
  },
})

await cvd.report(diff).writeMarkdown("out/diff.md")
```

The first diff implementation can be JSON/value based. Later versions can become CSS-aware.

Layer 5: Catalog, report, and write integration

The existing catalog API should remain Go-backed. The lower-level API should make it easy to add snapshots and diffs, not only inspect results.

Potential additions:

```
catalog.addSnapshot(target, snapshot)
catalog.addDiff(name, diff)
catalog.writeIndex({ includeSnapshots: true, includeDiffs: true })
```

The report/write helpers should return Promises because they touch files:

```
await cvd.write.json("out/snapshot.json", snapshot)
await cvd.report(diff).writeMarkdown("out/diff.md")
```

---

7. Replacing YAML concepts with JavaScript

YAML currently defines targets, sections, styles, output, and modes. JavaScript can express the same concepts with more abstraction.

7.1 YAML target to JS target

Current YAML shape:

```
original:
  name: prototype
```

```yaml
  url: http://localhost:7070/Pyxis%20Public%20Site.html
  wait_ms: 1000
  viewport: { width: 1200, height: 2200 }
  root_selector: "#capture-root"
  prepare:
    type: direct-react-global
    component: PPXDesktop
    root_selector: "#capture-root"
    width: 920
```

Legacy/current APIs may still expose a lowerCamel raw object for compatibility and YAML interop:

```go
const legacyTargetObject = {
  name: "prototype",
  url: "http://localhost:7070/Pyxis%20Public%20Site.html",
  waitMs: 1000,
  viewport: { width: 1200, height: 2200 },
  rootSelector: "#capture-root",
  prepare: {
    type: "directReactGlobal",
    component: "PPXDesktop",
    rootSelector: "#capture-root",
    width: 920,
  },
}
```

The preferred lower-level API should use a Go-backed target builder:

```go
const target = cvd.target("prototype")
  .url("http://localhost:7070/Pyxis%20Public%20Site.html")
  .waitMs(1000)
  .viewport(1200, 2200)
  .root("#capture-root")
  .directReactGlobal("PPXDesktop", {
    rootSelector: "#capture-root",
    width: 920,
  })
```

7.2 YAML styles to JS probes

Current YAML style:

```yaml
styles:
  - name: button-primary
    selector_react: ".btn-primary"
    props: [display, color, background-color, font-size]
    attributes: [class]
```

Equivalent JS builder:

```
const buttonPrimary = cvd.probe("button-primary")
  .selector(".btn-primary")
  .styles(["display", "color", "background-color", "font-size"])
  .attributes(["class"])
```

Reusable JS factory:

```
function buttonProbe(name, selector) {
  return cvd.probe(name)
    .selector(selector)
    .text()
    .bounds()
    .styles([
      "display",
      "align-items",
      "justify-content",
      "font-size",
      "font-weight",
      "line-height",
      "color",
      "background-color",
      "border",
      "border-radius",
      "padding",
    ])
    .attributes(["class", "aria-label"])
}

const probes = [
  buttonProbe("primary", ".btn-primary"),
  buttonProbe("secondary", ".btn-secondary"),
]
```

This is the first major advantage over YAML: reusable functions.

7.3 YAML sections to JS probes or locators

A YAML `section` is usually a screenshot/layout region. In JS, that can become a probe:

```
const heroSection = cvd.probe("hero")
  .selector("main .hero")
  .bounds()
  .screenshot()
  .html()
```

Or, for immediate low-level use:

```
const hero = page.locator("main .hero")
await hero.screenshot({ path: "out/hero.png" })
const bounds = await hero.bounds()
```

7.4 YAML output modes to JS writer calls

YAML output flags:

```
output:
  write_json: true
  write_markdown: true
  write_pngs: true
  write_prepared_html: true
  write_inspect_json: true
```

JS can express this as artifact policy:

```
const artifactPolicy = cvd.artifacts.bundle()
```

Or explicitly:

```
const snapshot = await cvd.snapshot(page, probes, {
  extractors: [
    cvd.extractors.bounds(),
    cvd.extractors.computedStyle(commonProps),
    cvd.extractors.screenshot(),
    cvd.extractors.html(),
  ],
  outDir: catalog.artifactDir(target.slug),
})

await cvd.write.json("out/snapshot.json", snapshot)
await cvd.report(snapshot).writeMarkdown("out/snapshot.md")
```

7.5 YAML modes to explicit JS control flow

YAML modes are good for stable CI runs:

```
modes: [capture, cssdiff, matched-styles, pixeldiff, html-report]
```

JavaScript does not need a special `cvd.workflow(...)` builder to replace this. Ordinary functions and control flow are enough, and they remain easier to debug:

```
if (values.fast) {
  await runCssOnlyPass(targets)
} else if (values.authoring) {
  await runAuthoringCatalog(targets, { failOnMissing: false })
} else {
```

```
  await runStrictComparison(targets, { failOnMissing: true, includePixelDiff: true })
}
```

This is the second major advantage over YAML: explicit control flow without adding another fluent workflow abstraction.

---

8. Proposed detailed API reference

This section is intentionally API-reference-like. It should become `docs/js-low-level-api.md` when implemented.

8.1 Top-level module

```
interface CVD {
  browser(options?: BrowserOptions): Promise<Browser>
  target(name: string): TargetBuilder      // Go-backed Proxy builder
  probe(name: string): ProbeBuilder        // Go-backed Proxy builder
  extract(locator: Locator, extractors: Extractor[], options?: ExtractOptions): Promise<Elem
  snapshot(page: Page, probes: ProbeBuilder[], options?: SnapshotOptions): Promise<PageSnaps
  diff(left: SnapshotLike, right: SnapshotLike, options?: DiffOptions): DiffResult
  catalog(options: CatalogOptions): Catalog
  loadConfig(path: string): Promise<Config>
  targetFromConfig(config: Config, side: "original" | "react"): TargetBuilder
  probesFromConfig(config: Config, options?: ProbesFromConfigOptions): ProbeBuilder[]
  viewport: ViewportFactory
  extractors: ExtractorFactory
  write: WriteAPI
  report(value: any): ReportBuilder
}
```

8.2 Browser

Current methods should remain:

```
interface Browser {
  newPage(options?: PageOptions): Promise<Page>
  page(url: string, options?: PageOptions): Promise<Page>
  close(): Promise<void>
}
```

Potential additions:

```
interface Browser {
  pages(): Promise<Page[]>
  version(): Promise<string>
}
```

8.3 Page

Current methods should remain and be extended:

```
interface Page {
  goto(url: string, options?: GotoOptions): Promise<TargetSummary>
  prepare(spec: PrepareSpec): Promise<void>
  preflight(probes: ProbeLike[]): Promise<SelectorStatus[]>
  inspect(probe: ProbeLike, options: InspectOptions): Promise<InspectArtifactResult>
  inspectAll(probes: ProbeLike[], options: InspectAllOptions): Promise<InspectResult>
  close(): Promise<void>

  locator(selector: string, options?: LocatorOptions): Locator
  evaluate<T = any>(script: string, ...args: any[]): Promise<T>
  html(options?: HtmlOptions): Promise<string>
  screenshot(options?: PageScreenshotOptions): Promise<ScreenshotResult>
}
```

8.4 Locator

```
interface Locator {
  page: Page
  selector: string
  options: LocatorOptions

  exists(): Promise<boolean>
  visible(): Promise<boolean>
  status(): Promise<SelectorStatus>
  text(options?: TextOptions): Promise<string>
  html(options?: HtmlOptions): Promise<string>
  outerHTML(options?: HtmlOptions): Promise<string>
  bounds(): Promise<Bounds | null>
  computedStyle(props?: string[]): Promise<Record<string, string>>
  attributes(names?: string[]): Promise<Record<string, string | null>>
  screenshot(options?: LocatorScreenshotOptions): Promise<ScreenshotResult>
  extract(extractors: Extractor[], options?: ExtractOptions): Promise<ElementSnapshot>
}
```

A locator is a lightweight JS handle. It should not eagerly query the DOM.
Each method runs a service operation.

8.5 Extractors

```
type Extractor =
  | ExistsExtractor
  | VisibleExtractor
  | TextExtractor
  | HtmlExtractor
```

```
  | BoundsExtractor
  | ComputedStyleExtractor
  | AttributesExtractor
  | ScreenshotExtractor
  | InspectJsonExtractor

interface ExtractorFactory {
  exists(): Extractor
  visible(): Extractor
  text(options?: { normalizeWhitespace?: boolean; maxLength?: number }): Extractor
  html(options?: { outer?: boolean }): Extractor
  bounds(): Extractor
  computedStyle(props?: string[]): Extractor
  attributes(names?: string[]): Extractor
  screenshot(options?: { path?: string; omitBackground?: boolean }): Extractor
  inspectJson(options?: object): Extractor
}
```

8.6 Snapshot result shapes

```
interface ElementSnapshot {
  name?: string
  selector: string
  source?: string
  exists?: boolean
  visible?: boolean
  text?: string
  html?: string
  outerHTML?: string
  bounds?: Bounds | null
  computedStyle?: Record<string, string>
  attributes?: Record<string, string | null>
  artifacts?: Record<string, string>
  errors?: SnapshotError[]
  metadata: SnapshotMetadata
}

interface PageSnapshot {
  target?: TargetRecord
  url: string
  viewport: Viewport
  createdAt: string
  elements: ElementSnapshot[]
  errors: SnapshotError[]
  timing: TimingTree
}
```

8.7 Diff API

```
interface DiffOptions {
  ignore?: string[]
  include?: string[]
  tolerances?: Record<string, number>
  normalize?: {
    whitespace?: boolean
    colors?: boolean
    numericCssValues?: boolean
  }
}

interface DiffResult {
  equal: boolean
  summary: {
    changed: number
    added: number
    removed: number
    ignored: number
  }
  changes: DiffChange[]
}
```

Example:

```
const diff = cvd.diff(before, after, {
  ignore: ["metadata.createdAt", "bounds.x", "bounds.y"],
  tolerances: { "bounds.width": 2, "bounds.height": 2 },
})
```

8.8 Report/write API

```
interface WriteAPI {
  json(path: string, value: any, options?: JsonWriteOptions): Promise<string>
  markdown(path: string, markdown: string): Promise<string>
  text(path: string, text: string): Promise<string>
}

interface ReportBuilder {
  writeMarkdown(path: string): Promise<string>
  writeHTML(path: string): Promise<string>
  writeJSON(path: string): Promise<string>
}
```

Writing should be Promise-based because it touches files.

---

9. Architecture for implementation

9.1 Target package layout

Do not put the expanded native API into `internal/cssvisualdiff/dsl/cvd_module.go`. The current `dsl` package is the jsverbs host/scanning/runtime package. The native JavaScript API is becoming its own subsystem and should move into a dedicated subpackage.

Recommended layout:

```
internal/cssvisualdiff/service/
  browser.go              # exists
  prepare.go              # exists
  preflight.go            # exists
  style.go                # exists
  inspect.go              # exists, may later split into inspect_* files
  catalog_service.go      # exists, may later split if snapshots/diffs join catalog schema
  dom.go                  # new: locator/status/text/html/bounds/attributes operations
  extract.go              # new: extraction pipeline and snapshot structs
  diff.go                 # new: JSON/snapshot diff logic
  report.go               # new or later: markdown/html report helpers for snapshots/diffs

internal/cssvisualdiff/dsl/
  host.go                 # jsverbs embedded host and runtime factory wiring
  embed.go                # embedded script filesystem
  sections.go             # shared jsverbs sections
  registrar.go            # registers native modules, delegates css-visual-diff module to js
  scripts/                # embedded built-in JS verbs

internal/cssvisualdiff/jsapi/
  module.go               # Register(ctx, reg) and top-level require("css-visual-diff") expo
  promises.go             # Promise bridge using go-go-goja runtime owner
  errors.go               # CvdError classes and Go error -> JS error mapping
  proxy.go                # generic Goja Proxy helpers, method-owner registry, error helpers
  unwrap.go               # unwrap typed Go-backed proxy values
  browser.go              # BrowserHandle Proxy
  page.go                 # PageHandle Proxy and page operation serialization
  locator.go              # LocatorHandle Proxy
  target.go               # TargetBuilder Proxy
  probe.go                # ProbeBuilder Proxy
  extractor.go            # ExtractorSpec/factory Proxies
  catalog.go              # CatalogHandle Proxy, migrated from dsl/catalog_adapter.go
  config.go               # loadConfig/fromConfig adapters, migrated from dsl/config_adapter
  snapshot.go             # cvd.extract and cvd.snapshot adapters
  diff.go                 # cvd.diff/report/write adapters
```

This is a stronger split than a simple file split inside `dsl`, but it is justified

because the Proxy-backed lower-level API will have its own helpers, typed unwrapping, and validation/error vocabulary.

The `dsl` package should still own jsverbs script execution. It should import `jsapi` only to register the native module:

```
import "github.com/go-go-golems/css-visual-diff/internal/cssvisualdiff/jsapi"

func (runtimeRegistrar) RegisterRuntimeModules(ctx *engine.RuntimeModuleContext, reg *nodere
    // existing compatibility modules, if retained: diff/report
    jsapi.Register(ctx, reg)
    return nil
}
```

Do not create many service subpackages yet. Keep `internal/cssvisualdiff/service` as one package with multiple files until the new data model stabilizes. This avoids premature import cycles and duplicate common types such as `Bounds`, `SelectorStatus`, `ProbeSpec`, and future `ElementSnapshot`.

9.2 Service layer must not import Goja

`internal/cssvisualdiff/service` should remain free of Goja. This keeps it testable and usable by CLI modes.

```
flowchart LR
    JS[JavaScript script] --> Adapter[jsapi Proxy adapters]
    Adapter --> Service[service package]
    Service --> Driver[driver/chromedp]

    Service -. must not import .-> Goja[goja]
```

9.3 Proxy-backed JS API design

The lower-level API should follow the Goja Proxy pattern already proven in the Discord bot UI DSL. Each live handle or builder is a Go struct wrapped by `vm.NewProxy(...)`. The Proxy `Get` trap returns only the methods that are valid for that object and panics with a typed Goja error for wrong-parent or unknown methods.

The key pattern is:

```
type ProbeBuilder struct {
    name       string
    selector   string
    extractors []service.ExtractorSpec
    required   bool
    source     string
}
```

```go
func newProbeBuilder(vm *goja.Runtime, registry *ProxyRegistry, name string) goja.Value {
    b := &ProbeBuilder{name: strings.TrimSpace(name)}
    target := vm.NewObject()

    proxy := vm.NewProxy(target, &goja.ProxyTrapConfig{
        Get: func(target *goja.Object, property string, receiver goja.Value) goja.Value {
            switch property {
            case "selector":
                return vm.ToValue(func(call goja.FunctionCall) goja.Value {
                    b.selector = requiredString(vm, "cvd.probe.selector", call.Argument(0))
                    return receiver
                })
            case "text":
                return vm.ToValue(func(call goja.FunctionCall) goja.Value {
                    b.extractors = append(b.extractors, service.ExtractorSpec{Kind: service.
                    return receiver
                })
            case "styles":
                return vm.ToValue(func(call goja.FunctionCall) goja.Value {
                    props := requiredStringList(vm, "cvd.probe.styles", call.Argument(0))
                    b.extractors = append(b.extractors, service.ExtractorSpec{Kind: service.
                    return receiver
                })
            case "build":
                return vm.ToValue(func(call goja.FunctionCall) goja.Value {
                    spec, err := b.Build()
                    if err != nil { panic(cvdTypeError(vm, err)) }
                    return vm.ToValue(lowerProbeSpec(spec))
                })
            default:
                checkCVDMethod(vm, "cvd.probe", property, probeAvailableMethods)
                return goja.Undefined()
            }
        },
    })

    return registry.Bind(vm.ToValue(proxy), b, "cvd.probe()")
}
```

Every Proxy `Get` trap should have three branches:

1. **Own method**: return the chain or async method.
2. **Known method on another object**: throw a wrong-parent error.
3. **Unknown method**: throw an unknown-method error with available methods.

Example wrong-parent errors:

```
cvd.locator: .styles() is not available here. .styles() belongs to cvd.probe().
For direct style reads on a locator, use .computedStyle(["color", ...]).
```

```
cvd.browser: .locator() is not available here. You probably meant:
  const page = await browser.page(url)
  const el = page.locator("#selector")
```

Example raw-object rejection:

```
cvd.snapshot: expected cvd.probe() builders, got object at probes[0].
Use cvd.probe("cta").selector("#cta").text() instead.
```

The `jsapi/proxy.go` file should contain shared helpers:

```go
type ProxyRegistry struct {
    // Tracks Go-backed proxy identities for unwrapping.
}

type ProxySpec struct {
    BuilderName string
    Methods map[string]MethodSpec
    Available []string
}

func newProxy(vm *goja.Runtime, registry *ProxyRegistry, spec ProxySpec, backing any) goja.V
func checkCVDMethod(vm *goja.Runtime, owner, property string, available []string)
func wrongParentError(vm *goja.Runtime, owner, property string)
func unknownMethodError(vm *goja.Runtime, owner, property string, available []string)
func typeMismatchError(vm *goja.Runtime, operation, expected string, got goja.Value)
```

The `jsapi/unwrap.go` file should contain typed unwrappers:

```go
func unwrapPage(vm *goja.Runtime, v goja.Value) (*PageHandle, error)
func unwrapLocator(vm *goja.Runtime, v goja.Value) (*LocatorHandle, error)
func unwrapProbe(vm *goja.Runtime, v goja.Value) (*ProbeBuilder, error)
func unwrapExtractor(vm *goja.Runtime, v goja.Value) (service.ExtractorSpec, error)
func unwrapCatalog(vm *goja.Runtime, v goja.Value) (*CatalogHandle, error)
```

Implementation note: a Proxy can close over its Go struct for methods, but cross-API calls such as `cvd.snapshot(page, [probe])` need a reliable unwrapping registry. Use a Go-side registry keyed by an internal proxy id or object identity. `.build()` may exist for debugging and serialization, but strict APIs should not require users to call `.build()` before passing builders.

9.4 Locator service design

Add `internal/cssvisualdiff/service/dom.go`.

Suggested Go types:

```go
type LocatorSpec struct {
    Name     string `json:"name,omitempty"`
    Selector string `json:"selector"`
    Source   string `json:"source,omitempty"`
}

type ElementHTML struct {
    Selector string `json:"selector"`
    Exists   bool   `json:"exists"`
    HTML     string `json:"html,omitempty"`
    OuterHTML string `json:"outer_html,omitempty"`
}

type TextOptions struct {
    NormalizeWhitespace bool
    MaxLength int
}

func LocatorStatus(page *driver.Page, locator LocatorSpec) (SelectorStatus, error)
func LocatorText(page *driver.Page, locator LocatorSpec, opts TextOptions) (string, error)
func LocatorHTML(page *driver.Page, locator LocatorSpec, outer bool) (ElementHTML, error)
func LocatorBounds(page *driver.Page, locator LocatorSpec) (*Bounds, error)
func LocatorAttributes(page *driver.Page, locator LocatorSpec, attrs []string) (map[string]s
func LocatorComputedStyle(page *driver.Page, locator LocatorSpec, props []string) (map[strin
```

Implementation note: `PreflightProbes(...)` in `internal/cssvisualdiff/service/preflight.go:10` already contains JavaScript for selector status. Reuse or factor that script rather than duplicating selector logic.

9.5 Extraction service design

Add `internal/cssvisualdiff/service/extract.go`.

Suggested types:

```go
type ExtractorKind string

const (
    ExtractorExists ExtractorKind = "exists"
    ExtractorVisible ExtractorKind = "visible"
    ExtractorText ExtractorKind = "text"
    ExtractorHTML ExtractorKind = "html"
    ExtractorBounds ExtractorKind = "bounds"
    ExtractorComputedStyle ExtractorKind = "computedStyle"
    ExtractorAttributes ExtractorKind = "attributes"
    ExtractorScreenshot ExtractorKind = "screenshot"
    ExtractorInspectJSON ExtractorKind = "inspectJson"
```

```go
)

type ExtractorSpec struct {
    Kind ExtractorKind `json:"kind"`
    Props []string `json:"props,omitempty"`
    Attributes []string `json:"attributes,omitempty"`
    Path string `json:"path,omitempty"`
    Options map[string]any `json:"options,omitempty"`
}

type ElementSnapshot struct {
    Name string `json:"name,omitempty"`
    Selector string `json:"selector"`
    Source string `json:"source,omitempty"`
    Exists *bool `json:"exists,omitempty"`
    Visible *bool `json:"visible,omitempty"`
    Text string `json:"text,omitempty"`
    HTML string `json:"html,omitempty"`
    OuterHTML string `json:"outer_html,omitempty"`
    Bounds *Bounds `json:"bounds,omitempty"`
    ComputedStyle map[string]string `json:"computed_style,omitempty"`
    Attributes map[string]string `json:"attributes,omitempty"`
    Artifacts map[string]string `json:"artifacts,omitempty"`
    Errors []SnapshotError `json:"errors,omitempty"`
    Metadata SnapshotMetadata `json:"metadata"`
}

func ExtractElement(page *driver.Page, locator LocatorSpec, extractors []ExtractorSpec, opts
func SnapshotPage(page *driver.Page, target config.Target, probes []ProbeSpec, extractors []
```

The first implementation can call existing services:

- `PreflightProbes` for status.
- `EvaluateStyle` for computed styles (`internal/cssvisualdiff/service/style.go:18`).
- `page.Screenshot` for screenshots.
- `WritePreparedHTML` / `WriteInspectJSON` helpers from `inspect.go` for HTML/inspect JSON.

9.6 Diff service design

Add `internal/cssvisualdiff/service/diff.go`.

First implementation can be structural JSON diff:

```go
type DiffOptions struct {
    Ignore []string
    Include []string
    Tolerances map[string]float64
```

```go
}

type DiffResult struct {
    Equal bool `json:"equal"`
    Summary DiffSummary `json:"summary"`
    Changes []DiffChange `json:"changes"`
}

func DiffSnapshots(left, right any, opts DiffOptions) (DiffResult, error)
```

Rules:

- Ignore paths before comparison.
- Numeric values can use tolerances.
- String values compare exactly in v1.
- Later: CSS color normalization, length normalization, class list normalization.

9.7 Page operation serialization

The current JS API starts goroutines for Promise work. For one page, multiple JS Promises could call chromedp concurrently if a user does this:

```python
await Promise.all([
  page.locator("#a").computedStyle(["color"]),
  page.locator("#b").computedStyle(["color"]),
])
```

The docs already warn against assuming per-page CDP parallelism (`docs/js-api.md:428`). A lower-level API should enforce safer behavior.

Add a mutex or queue to `pageState`:

```go
type pageState struct {
    page *service.PageService
    target config.Target
    mu sync.Mutex
}

func (p *pageState) runExclusive(fn func() (any, error)) (any, error) {
    p.mu.Lock()
    defer p.mu.Unlock()
    return fn()
}
```

Then every page/locator operation uses `runExclusive` inside `promiseValue`.

This does not prevent target-level parallelism. Users can still open multiple pages and use a worker limit.

10. Goja adapter design with Proxy-backed handles

This section replaces the earlier plain-object adapter idea. The new adapter rule is:

```
No new lower-level live handles or DSL builders should be plain JS objects.
They should be Go structs wrapped in Goja Proxy objects.
```

10.1 Register the module from `jsapi.Register`

`internal/cssvisualdiff/jsapi/module.go` should expose one entry point:

```go
package jsapi

func Register(ctx *engine.RuntimeModuleContext, reg *noderequire.Registry) {
    reg.RegisterNativeModule("css-visual-diff", func(vm *goja.Runtime, module *goja.Object)
        exports := module.Get("exports").(*goja.Object)
        state := NewModuleState(ctx, vm)

        installCVDErrorClasses(vm, exports)
        installBrowserAPI(state, exports)
        installCatalogAPI(state, exports)
        installConfigAPI(state, exports)
        installBuilderAPI(state, exports)
        installExtractorAPI(state, exports)
        installSnapshotAPI(state, exports)
        installDiffAPI(state, exports)
        installWriteAPI(state, exports)
    })
}
```

`ModuleState` should hold shared runtime context and the proxy registry:

```go
type ModuleState struct {
    ctx      *engine.RuntimeModuleContext
    vm       *goja.Runtime
    registry *ProxyRegistry
}
```

10.2 Browser and page handles

`cvd.browser()` should resolve to a `BrowserHandle` Proxy:

```go
type BrowserHandle struct {
    service *service.BrowserService
}
```

Valid methods:

```
browser.newPage()
browser.page(url, options)
browser.close()
```

`browser.page(...)` should resolve to a `PageHandle` Proxy:

```go
type PageHandle struct {
    page   *service.PageService
    target config.Target
    mu     sync.Mutex
}
```

Valid methods:

```
page.goto(url, options)
page.prepare(spec)
page.preflight(probes)       // compatibility path can accept raw probe objects
page.inspect(probe, options) // compatibility path can accept raw probe objects
page.inspectAll(probes, options)
page.locator(selector)
page.close()
```

New strict lower-level APIs should unwrap `PageHandle` rather than accepting arbitrary objects.

10.3 Locator handle

`page.locator(selector)` should synchronously return a `LocatorHandle` Proxy:

```go
type LocatorHandle struct {
    page     *PageHandle
    selector string
    name     string
    source   string
}
```

Valid methods:

```
locator.status()
locator.exists()
locator.visible()
locator.text(options)
locator.bounds()
locator.computedStyle(props)
locator.attributes(names)
locator.html(options)
```

```
locator.screenshot(options)
locator.extract(extractors, options)
```

The methods that touch Chromium return Promises. The handle itself is synchronous because it only stores the selector and page reference.

Wrong usage should be contextual:

```
page.locator("#cta").styles(["color"])
```

Should produce something like:

```
cvd.locator: .styles() is not available here. .styles() belongs to cvd.probe().
For direct style reads on a locator, use .computedStyle(["color"]).
```

10.4 Target and probe builders

`cvd.target(name)` and `cvd.probe(name)` should return Go-backed builders.

```go
type TargetBuilder struct {
    Name         string
    URL          string
    WaitMS       int
    Viewport     config.Viewport
    RootSelector string
    Prepare      *config.PrepareSpec
}

type ProbeBuilder struct {
    Name       string
    Selector   string
    Source     string
    Required   bool
    Extractors []service.ExtractorSpec
}
```

Example JS:

```go
const target = cvd.target("booking")
  .url("http://localhost:3000/booking")
  .viewport(1440, 900)
  .waitMs(250)
  .root("#app")

const probe = cvd.probe("cta")
  .selector("#cta")
  .text()
  .bounds()
  .styles(["color", "font-size"])
```

These builders should validate immediately. Examples:

```
cvd.target.viewport: expected (width: number, height: number) or known viewport name, got st
Known names: desktop, tablet, mobile, iphone14.
```

```
cvd.probe.styles: expected array of CSS property names, got string.
Use .styles(["color"]) instead.
```

10.5 Extractor factory

`cvd.extractors.*` should return Go-backed extractor specs, not raw maps.

```
const extractors = [
  cvd.extractors.text(),
  cvd.extractors.bounds(),
  cvd.extractors.computedStyle(["color", "font-size"]),
]
```

Backed by:

```
type ExtractorHandle struct {
    spec service.ExtractorSpec
}
```

The extractor factory is a good place for argument-specific errors:

```
cvd.extractors.computedStyle: expected array of CSS property names, got string.
Use computedStyle(["color"]) instead.
```

10.6 Strict `cvd.extract` and `cvd.snapshot`

`cvd.extract(locator, extractors)` should require:

- a `LocatorHandle` Proxy,
- an array of `ExtractorHandle` values.

Pseudocode:

```
func installSnapshotAPI(state *ModuleState, exports *goja.Object) {
    _ = exports.Set("extract", func(call goja.FunctionCall) goja.Value {
        locator, err := unwrapLocator(state.vm, call.Argument(0))
        if err != nil { panic(typeMismatchError(...)) }

        extractors, err := unwrapExtractors(state.vm, call.Argument(1))
        if err != nil { panic(typeMismatchError(...)) }

        return promiseValue(state.ctx, state.vm, "css-visual-diff.extract", func() (any, err
            return locator.page.runExclusive(func() (any, error) {
                return service.ExtractElement(locator.page.page.Page(), locator.Spec(), extr
            })
        }, lowerElementSnapshotValue)
```

```
    })
}
```

`cvd.snapshot(page, probes, options)` should require:

- a `PageHandle` Proxy,
- an array of `ProbeBuilder` proxies or built probe specs from the same Go-backed API.

Raw objects should be rejected with guidance unless the method is explicitly a legacy compatibility method.

10.7 Existing high-level methods remain compatibility methods

Do not break existing scripts:

```
await page.preflight([{ name: "cta", selector: "#cta" }])
await page.inspectAll([{ name: "cta", selector: "#cta", props: ["color"] }], opts)
```

Those current high-level methods can keep their raw-object decode path. The new lower-level APIs should be stricter:

```
await cvd.extract(page.locator("#cta"), [cvd.extractors.text()])
await cvd.snapshot(page, [cvd.probe("cta").selector("#cta").text()])
```

This gives compatibility without weakening the new LLM-friendly surface.

---

11. Example usage patterns

11.1 Simple low-level inspection

```
async function inspectOne(url, selector) {
  const cvd = require("css-visual-diff")
  const browser = await cvd.browser()
  try {
    const page = await browser.page(url, { viewport: { width: 1280, height: 720 } })
    const el = page.locator(selector)

    return await cvd.extract(el, [
      cvd.extractors.exists(),
      cvd.extractors.visible(),
      cvd.extractors.text({ maxLength: 200 }),
      cvd.extractors.bounds(),
      cvd.extractors.computedStyle(["display", "color", "font-size"]),
    ])
  } finally {
    await browser.close()
  }
}
```

11.2 Replace YAML with JS factories

```
function responsiveTargets(path) {
  return [
    cvd.target("desktop").url(baseUrl + path).viewport(1440, 900),
    cvd.target("tablet").url(baseUrl + path).viewport(768, 1024),
    cvd.target("mobile").url(baseUrl + path).viewport(390, 844),
  ]
}

function typographyProbe(name, selector) {
  return cvd.probe(name)
    .selector(selector)
    .text()
    .bounds()
    .styles(["font-family", "font-size", "font-weight", "line-height", "color"])
}

const targets = responsiveTargets("/booking")
const probes = [
  typographyProbe("headline", "h1"),
  typographyProbe("body", ".booking-copy"),
  cvd.probe("cta").selector(".booking-cta").text().bounds().styles(buttonProps),
]
```

11.3 Snapshot and diff two rendered states

```
const original = await browser.page(originalUrl, { viewport })
const react = await browser.page(reactUrl, { viewport })

const probes = [
  cvd.probe("cta").selector(".cta").styles(["color", "font-size"]).bounds(),
  cvd.probe("calendar").selector(".calendar").styles(["display", "gap"]).bounds(),
]

const before = await cvd.snapshot(original, probes)
const after = await cvd.snapshot(react, probes)

const diff = cvd.diff(before, after, {
  ignore: ["metadata.createdAt", "elements.*.bounds.x", "elements.*.bounds.y"],
  tolerances: {
    "elements.*.bounds.width": 2,
    "elements.*.bounds.height": 2,
  },
})
```

```
if (!diff.equal) {
  await cvd.report(diff).writeMarkdown("out/diff.md")
}
```

11.4 Catalog authoring mode

```
const catalog = cvd.catalog({ title: "Booking catalog", outDir: "out/catalog" })

for (const target of targets) {
  const page = await browser.page(target.url(), { viewport: target.viewport() })
  try {
    await target.preparePage(page)

    const snapshot = await cvd.snapshot(page, target.probes(), {
      outDir: catalog.artifactDir(target.slug()),
      failOnMissing: false,
    })

    catalog.addTarget(target.toJSON())
    catalog.addSnapshot(target.toJSON(), snapshot)
  } catch (err) {
    catalog.addFailure(target.toJSON(), err)
  } finally {
    await page.close()
  }
}

await catalog.writeManifest()
await catalog.writeIndex()
```

11.5 Generated CLI verb that uses the lower-level API

```
async function inspectSelector(url, selector, values) {
  const cvd = require("css-visual-diff")
  const browser = await cvd.browser()
  try {
    const page = await browser.page(url, {
      viewport: { width: values.width, height: values.height },
      waitMs: values.waitMs,
    })

    const snapshot = await cvd.extract(page.locator(selector), [
      cvd.extractors.text(),
      cvd.extractors.bounds(),
      cvd.extractors.computedStyle(values.props),
    ])
```

```
    if (values.outDir) {
      await cvd.write.json(`${values.outDir}/snapshot.json`, snapshot)
    }

    return snapshot
  } finally {
    await browser.close()
  }
}

__verb__("inspectSelector", {
  parents: ["custom"],
  output: "structured",
  fields: {
    url: { argument: true, required: true },
    selector: { argument: true, required: true },
    values: { bind: "all" },
    width: { type: "int", default: 1280 },
    height: { type: "int", default: 720 },
    waitMs: { type: "int", default: 0 },
    props: { type: "stringList", default: ["display", "color", "font-size"] },
    outDir: { type: "string", default: "" },
  }
})
```

Run:

```
css-visual-diff verbs --repository ./verbs custom inspect-selector \
  http://localhost:3000 '#cta' \
  --width 390 \
  --height 844 \
  --props color,font-size,background-color \
  --outDir out/cta
```

---

12. Implementation phases

Phase 0: document and protect current behavior

Goal: make sure the current high-level API stays stable while adding lower-level pieces.

Tasks:

- Keep this guide in the ticket.
- Add a short future-facing `docs/js-low-level-api.md` once implementation starts.

- Add or preserve tests proving current methods still work:
  - `cvd.browser()`
  - `browser.page()`
  - `page.preflight()`
  - `page.inspectAll()`
  - `cvd.catalog()`

Relevant existing tests:

- `internal/cssvisualdiff/verbcli/command_test.go:115` tests the Promise-first module and `inspectAll` path.
- `internal/cssvisualdiff/verbcli/command_test.go:169` tests catalog manifest/index writing from a JS verb.
- `internal/cssvisualdiff/verbcli/command_test.go:231` tests typed errors.

Phase 1: move the native API out of `dsl/cvd_module.go` into `internal/cssvisualdiff/jsapi`

Goal: perform a no-behavior refactor before adding more API surface.

Tasks:

- Create `internal/cssvisualdiff/jsapi`.
- Move `registerCVDModule` logic into `jsapi.Register(ctx, reg)`.
- Move `promiseValue` into `jsapi/promises.go`.
- Move typed error helpers into `jsapi/errors.go`.
- Move current browser/page wrappers into `jsapi/browser.go` and `jsapi/page.go`.
- Move `catalog_adapter.go` to `jsapi/catalog.go` or adapt it behind `installCatalogAPI(...)`.
- Move `config_adapter.go` to `jsapi/config.go` or adapt it behind `installConfigAPI(...)`.
- Update `internal/cssvisualdiff/dsl/registrar.go` to import `jsapi` and call `jsapi.Register(ctx, reg)`.
- Keep package `dsl` focused on jsverbs host/scanning/runtime wiring.

No behavior changes should happen in this phase.

Validation:

```
go test ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./internal/cssvisualdi
go test ./...
```

Phase 2: add Proxy infrastructure and typed unwrapping

Goal: establish the Go-backed object model before adding locator/extractor APIs.

Files:

- Add `internal/cssvisualdiff/jsapi/proxy.go`.
- Add `internal/cssvisualdiff/jsapi/unwrap.go`.

Tasks:

- Add a `ProxyRegistry` or equivalent identity mechanism for mapping Goja Proxy values back to Go structs.
- Add method-owner registry for wrong-parent errors.
- Add shared unknown-method, wrong-parent, and type-mismatch errors.
- Add tests proving Proxy traps produce useful errors.
- Optionally convert existing `Browser` / `Page` / `Catalog` wrappers to Proxy-backed handles in this phase, or do so incrementally in Phase 3.

The goal is LLM-friendly feedback such as:

```
cvd.probe: unknown method .style(). Available: selector, text, bounds, styles, attributes, s
Did you mean .styles()?
```

Phase 3: add page operation serialization

Goal: make lower-level locator APIs safe even if users call them concurrently.

Tasks:

- Add `sync.Mutex` or a promise queue to `PageHandle`.
- Wrap existing `goto`, `prepare`, `preflight`, `inspect`, `inspectAll`, and `close` calls.
- Add a test that runs several page operations via `Promise.all` and confirms no race/panic/hang.

This phase changes internals but not the public API.

Phase 4: implement service DOM primitives

Goal: add reusable Go operations for locator status/text/html/bounds/attributes/style.

Files:

- Add `internal/cssvisualdiff/service/dom.go`.
- Add `internal/cssvisualdiff/service/dom_test.go`.

Service functions:

```go
func LocatorStatus(page *driver.Page, locator LocatorSpec) (SelectorStatus, error)
func LocatorText(page *driver.Page, locator LocatorSpec, opts TextOptions) (string, error)
func LocatorHTML(page *driver.Page, locator LocatorSpec, outer bool) (ElementHTML, error)
func LocatorBounds(page *driver.Page, locator LocatorSpec) (*Bounds, error)
func LocatorAttributes(page *driver.Page, locator LocatorSpec, attrs []string) (map[string]s
func LocatorComputedStyle(page *driver.Page, locator LocatorSpec, props []string) (map[strin
```

Implementation detail:

- Reuse the selector-status logic from `PreflightProbes` where possible.

- Reuse `EvaluateStyle` for computed style where possible.
- Return structured "exists false" results for missing selectors when the operation is naturally optional.
- Throw/return errors for invalid selectors.

Phase 5: expose `page.locator()` and locator methods as Proxy-backed handles

Goal: first visible lower-level JS API.

Files:

- Add `internal/cssvisualdiff/jsapi/locator.go`.
- Update `internal/cssvisualdiff/jsapi/page.go` to install `page.locator`.

Methods to ship first:

```
page.locator(selector)
await locator.status()
await locator.exists()
await locator.visible()
await locator.text()
await locator.bounds()
await locator.computedStyle(props)
await locator.attributes(names)
```

Defer screenshots/html until the basic data path is stable if needed.

Tests:

- JS verb opens a test server page.
- Calls locator methods.
- Returns structured rows.
- Assert values in Go test.
- Assert invalid property access gives a situation-specific Proxy error.

Phase 6: add Go-backed target/probe/extractor builders

Goal: make YAML replacement ergonomic and strongly validated.

Files:

- Add `internal/cssvisualdiff/jsapi/target.go`.
- Add `internal/cssvisualdiff/jsapi/probe.go`.
- Add `internal/cssvisualdiff/jsapi/extractor.go`.

Initial API:

```
cvd.viewport(width, height)
cvd.viewport.desktop()
cvd.viewport.mobile()
cvd.target(name).url(url).viewport(width, height).root(selector).build()
```

```
cvd.probe(name).selector(selector).text().bounds().styles(props).attributes(attrs).build()
cvd.extractors.text()
cvd.extractors.bounds()
cvd.extractors.computedStyle(props)
```

Important details:

- Builders are Go structs wrapped by Proxies.
- Strict lower-level APIs should unwrap builders directly; users should not be forced to call `.build()`.
- `.build()` exists for debugging/serialization.
- Raw JS objects should be rejected by strict APIs with helpful errors.

Phase 7: add strict `cvd.extract(locator, extractors)`

Goal: composable extraction with typed Go-backed inputs.

Files:

- Add `internal/cssvisualdiff/service/extract.go`.
- Add `internal/cssvisualdiff/jsapi/snapshot.go` or `extract.go`.

Minimum extractors:

```
cvd.extractors.exists()
cvd.extractors.visible()
cvd.extractors.text()
cvd.extractors.bounds()
cvd.extractors.computedStyle(props)
cvd.extractors.attributes(names)
```

Tests:

- Extract multiple facts from one locator.
- Missing selector returns `exists: false` plus an error entry or status depending on options.
- Invalid selector rejects with `SelectorError`.
- Passing raw objects instead of locator/extractor proxies produces a clear type-mismatch error.

Phase 8: add strict `cvd.snapshot(page, probes, options)`

Goal: inspect many Go-backed probe builders into an in-memory structured result without necessarily writing standard inspect artifacts.

API:

```
const snapshot = await cvd.snapshot(page, [
  cvd.probe("cta").selector("#cta").text().bounds().styles(["color"]),
], {
  outDir: "",             // optional
```

```
  failOnMissing: false,
})
```

Implementation:

- Unwrap `PageHandle` and `ProbeBuilder` proxies.
- For each probe, call `ExtractElement` with the probe's extractor list.
- Return plain `PageSnapshot` data.
- Optionally write artifact files only when screenshot/html extractors include paths or when `outDir` is provided.

Phase 9: add diff, report, and write primitives

Goal: compare snapshots without YAML.

API:

```
const diff = cvd.diff(before, after, options)
await cvd.report(diff).writeMarkdown("out/diff.md")
await cvd.write.json("out/diff.json", diff)
```

Implementation:

- Start with deterministic JSON diff.
- Add markdown rendering for changed paths.
- Add CSS-specific normalization later.

There is no workflow builder phase. Use ordinary JavaScript functions and loops for orchestration.

---

13. Testing strategy

13.1 Service tests

Add pure Go tests for each new service file:

```
go test ./internal/cssvisualdiff/service
```

Test cases:

- Locator status for existing, missing, hidden, and invalid selectors.
- Locator text normalization.
- Bounds extraction.
- Attribute extraction.
- Computed style extraction.
- Element extraction with multiple extractors.
- Snapshot of multiple probes.
- Diff equal/changed/ignored/tolerance cases.

13.2 Goja adapter tests

Use repository-scanned JS verbs as integration tests, following `internal/cssvisualdiff/verbcli/command_te`

Test pattern:

1. Start `httptest.Server` with tiny HTML.
2. Write a temporary JS verb file.
3. Scan repository.
4. Build commands.
5. Invoke command through Glazed processor.
6. Assert returned rows and artifact files.

Example JS test body:

```
async function lowLevel(url) {
  const cvd = require("css-visual-diff")
  const browser = await cvd.browser()
  try {
    const page = await browser.page(url)
    const cta = page.locator("#cta")
    const snapshot = await cvd.extract(cta, [
      cvd.extractors.text(),
      cvd.extractors.bounds(),
      cvd.extractors.computedStyle(["color"]),
    ])
    return {
      text: snapshot.text,
      color: snapshot.computedStyle.color,
      width: snapshot.bounds.width,
    }
  } finally {
    await browser.close()
  }
}
```

13.3 CLI smoke tests

Add ticket scripts under this ticket's `scripts/` directory when implementation begins:

```
scripts/001-low-level-locator-smoke.sh
scripts/002-extractors-smoke.sh
scripts/003-snapshot-diff-smoke.sh
scripts/004-yaml-migration-smoke.sh
```

Each script should build the binary and run a real JS verb through `css-visual-diff verbs --repository ....`

13.4 Documentation tests

Every code block in future docs should either be:

- included in a smoke script, or
- marked as pseudocode.

This matters because JS APIs drift quickly.

---

14. Error model

The existing typed errors are good but currently classified heuristically by operation name and error text. Evidence: `classifyCVDError(...)` in `internal/cssvisualdiff/dsl/cvd_module.go:113`.

For a lower-level API, errors should become more structured.

Recommended Go service errors:

```go
type ErrorCode string

const (
    CodeSelectorMissing ErrorCode = "SELECTOR_MISSING"
    CodeSelectorInvalid ErrorCode = "SELECTOR_INVALID"
    CodePrepareFailed ErrorCode = "PREPARE_FAILED"
    CodeBrowserFailed ErrorCode = "BROWSER_FAILED"
    CodeArtifactWriteFailed ErrorCode = "ARTIFACT_WRITE_FAILED"
    CodeDiffFailed ErrorCode = "DIFF_FAILED"
)

type CvdError struct {
    Code ErrorCode
    Operation string
    Message string
    Details map[string]any
    Cause error
}
```

Then the Goja adapter can map codes to JS classes without guessing.

JS-visible errors should keep the current shape:

```go
try {
  await page.locator("#missing").screenshot({ path: "out.png" })
} catch (err) {
  if (err instanceof cvd.SelectorError) {
    console.error(err.code, err.operation, err.details.selector)
  }
}
```

For authoring workflows, prefer structured results over thrown errors when the user asks for non-strict behavior:

```
const snapshot = await cvd.snapshot(page, probes, { failOnMissing: false })
console.log(snapshot.errors)
```

For strict workflows, throw:

```
await cvd.snapshot(page, probes, { failOnMissing: true })
```

---

15. Performance and concurrency guidance

15.1 What should be parallel

Parallelize at target/page level:

```
await cvd.parallel(targets, { limit: 3 }, async target => {
  const page = await browser.page(target.url, { viewport: target.viewport })
  try {
    return await cvd.snapshot(page, target.probes)
  } finally {
    await page.close()
  }
})
```

15.2 What should not be assumed parallel

Do not assume multiple operations on the same page will run in true parallel. Chromedp/CDP operations serialize at practical runtime boundaries, and concurrent access can be confusing.

Bad user expectation:

```
await Promise.all([
  page.locator("#a").computedStyle(["color"]),
  page.locator("#b").computedStyle(["color"]),
])
```

Allowed, but internally queued:

```
operation 1: computedStyle(#a)
operation 2: computedStyle(#b)
```

The API can still accept `Promise.all`; it should just preserve safety.

15.3 Batch where possible

The service layer should offer batch operations for efficiency:

```
func SnapshotPage(page *driver.Page, target config.Target, probes []ProbeSpec, extractors []
```

This can evaluate multiple selectors in one page script when possible, reducing CDP round trips.

---

16. Migration strategy

Stage 1: YAML interop helpers

Add:

```
const cfg = await cvd.loadConfig("page.css-visual-diff.yml")
const target = cvd.targetFromConfig(cfg, "react")
const probes = cvd.probesFromConfig(cfg, { source: "styles" })
```

This lets users reuse existing configs.

Stage 2: JS produces equivalent target/probe objects

Users can replace YAML gradually:

```
const target = cvd.target("react").url(storybookUrl).viewport(1280, 720)
const probes = cfg.styles.map(style => cvd.probe(style.name).selector(style.selectorReact).s
```

Stage 3: JS owns orchestration with ordinary functions

No YAML required, and no special workflow builder required:

```
for (const target of generatedTargets()) {
  await inspectTarget(target, standardProbes(target.kind))
}
```

Stage 4: optional export back to YAML

Later, if useful, explicit conversion helpers can export builder-backed definitions to YAML-compatible config values:

```
const config = cvd.config.fromTargets({ original, react, probes })
await cvd.writeConfig("generated.css-visual-diff.yml", config)
```

This is optional and should not block lower-level API implementation.

---

17. Risks and mitigations

Risk: The API becomes too large

Mitigation: ship in layers and explicitly keep workflow builders out of scope. Use ordinary JavaScript functions and loops for orchestration.

Risk: JavaScript reimplements Go logic

Mitigation: every durable operation belongs in `internal/cssvisualdiff/service`. JS adapters should remain thin.

Risk: Result shapes drift between YAML, inspect, snapshot, and catalog

Mitigation: define service structs for snapshots and diffs. Use adapters for lowerCamel conversion consistently, as `config_adapter.go` and `catalog_adapter.go` already do.

Risk: Concurrent per-page calls create flaky behavior

Mitigation: add page operation serialization before exposing many locator methods.

Risk: Builders hide when work happens

Mitigation: Go-backed builders must be synchronous and side-effect-free. Any browser/file work must use `await` and should happen on handles or top-level async functions, not inside target/probe builder methods.

Risk: Catalog schema grows accidentally

Mitigation: keep catalog schema versioned and Go-backed. If adding snapshots/diffs to catalog manifests, update `CatalogSchemaVersion` or add compatibility handling.

---

18. Acceptance criteria

A successful first lower-level API milestone should satisfy:

1. `page.locator(selector)` exists and is synchronous.
2. Locator methods return Promises.
3. `locator.status()`, `exists()`, `visible()`, `text()`, `bounds()`, `computedStyle()`, and `attributes()` work against a tiny HTTP test page.
4. `cvd.extract(locator, extractors)` requires Go-backed locator/extractor proxies and returns a plain JSON-serializable snapshot.
5. Go-backed extractor builders exist for text, bounds, computed style, and attributes.
6. Missing selectors produce structured status or typed `SelectorError` depending on strictness.
7. Existing high-level JS API tests still pass.
8. Existing CLI/YAML behavior still passes.

9. Documentation includes runnable examples.
10. At least one external repository-scanned verb demonstrates the lower-level API.

A successful second milestone should satisfy:

1. Go-backed `cvd.target`, `cvd.probe`, and `cvd.viewport` builders exist.
2. Strict `cvd.snapshot(page, probes, options)` exists and rejects raw objects with helpful errors.
3. `cvd.diff(before, after, options)` exists.
4. Markdown/JSON reporting exists for snapshot diffs.
5. YAML interop helpers convert configs into target/probe values.

---

19. File-by-file implementation guide for an intern

`internal/cssvisualdiff/jsapi/module.go` — new

Create the dedicated native API subpackage and move `require("css-visual-diff")` registration here. This file should expose `Register(ctx, reg)` and install top-level exports by delegating to small installer functions.

`internal/cssvisualdiff/jsapi/promises.go` — new

Move `promiseValue` here. Later add helpers such as `promiseVoid`, `promisePlain`, or operation-specific helpers if they reduce boilerplate.

`internal/cssvisualdiff/jsapi/errors.go` — new

Move JS error class installation and error mapping here. Later replace heuristic classification with service-layer typed errors.

`internal/cssvisualdiff/jsapi/proxy.go` — new

Implement the reusable Goja Proxy pattern: method-owner lookup, wrong-parent errors, unknown-method errors, type-mismatch errors, and helper constructors for Go-backed proxies.

`internal/cssvisualdiff/jsapi/unwrap.go` — new

Implement typed unwrapping for Go-backed Proxy values: page handles, locator handles, target builders, probe builders, extractor specs, and catalog handles.

`internal/cssvisualdiff/jsapi/browser.go` — new/refactor

Move `wrapBrowser` into a `BrowserHandle` Proxy. Valid methods should include `newPage`, `page`, and `close`.

`internal/cssvisualdiff/jsapi/page.go` — new/refactor

Move `pageState` / `wrapPage` into a `PageHandle` Proxy. Add per-page operation serialization here before exposing many locator methods.

`internal/cssvisualdiff/jsapi/locator.go` — new

Wrap `service.LocatorSpec` plus the owning `PageHandle`. Expose `status`, `exists`, `visible`, `text`, `bounds`, `computedStyle`, and `attributes` as Promise-returning methods. Use Proxy traps for invalid property access.

`internal/cssvisualdiff/jsapi/target.go` — new

Implement `cvd.target(name)` as a Go-backed `TargetBuilder` Proxy. It should validate URL, viewport, wait time, root selector, and prepare options.

`internal/cssvisualdiff/jsapi/probe.go` — new

Implement `cvd.probe(name)` as a Go-backed `ProbeBuilder` Proxy. It should validate selector, source, required flag, and requested extractors. It should produce useful errors for `.style(...)` vs `.styles(...)` and wrong-parent methods.

`internal/cssvisualdiff/jsapi/extractor.go` — new

Install `cvd.extractors` and create Go-backed extractor spec proxies. Validate extractor arguments at construction time.

`internal/cssvisualdiff/jsapi/catalog.go` — new/refactor

Move the current catalog adapter here and consider making catalog handles Proxy-backed too. Existing behavior should remain compatible.

`internal/cssvisualdiff/jsapi/config.go` — new/refactor

Move the current config adapter here. Add explicit conversion helpers from YAML configs to Go-backed target/probe builders.

`internal/cssvisualdiff/jsapi/snapshot.go` — new

Install strict `cvd.extract` and `cvd.snapshot`. These should unwrap only known Go-backed proxies and reject raw objects with helpful errors.

`internal/cssvisualdiff/jsapi/diff.go` — new

Install `cvd.diff`, `cvd.report`, and `cvd.write` once service diff/report functions exist.

`internal/cssvisualdiff/service/dom.go` — new

Implement low-level selector operations. Keep it independent from Goja.

`internal/cssvisualdiff/service/extract.go` — new

Implement extractor specs and element/page snapshots. Reuse `dom.go`, `style.go`, and selected helpers from `inspect.go`.

`internal/cssvisualdiff/service/diff.go` — new

Implement structural diff. Keep CSS-aware normalization for a later PR.

`internal/cssvisualdiff/dsl/registrar.go` — update

Import `internal/cssvisualdiff/jsapi` and call `jsapi.Register(ctx, reg)` instead of owning the full module registration locally.

`docs/js-low-level-api.md` — new

Turn this design's API reference into user-facing docs. Document that the new lower-level API uses Go-backed builders/handles and rejects raw objects in strict APIs.

`examples/verbs/low-level-inspect.js` — new

Add a small external example using `page.locator` and `cvd.extract`.

---

20. Review checklist

When reviewing an implementation PR, check:

- Does any service package import Goja? It should not.
- Did the native API move into `internal/cssvisualdiff/jsapi` while `dsl` remains focused on jsverbs host/runtime wiring?
- Are live handles and lower-level builders Go-backed Proxy objects rather than plain JS objects?
- Do Proxy traps produce wrong-parent, unknown-method, and type-mismatch errors with actionable guidance?
- Do strict lower-level APIs reject raw JS object specs with clear migration suggestions?
- Does every browser/file operation return a Promise in JS?
- Are lowerCamel JS fields converted consistently for compatibility/result data?
- Are missing selectors structured and clear?
- Are page operations serialized?
- Do existing high-level API examples still work?

- Does the new API avoid duplicating `inspect.go` artifact logic?
- Do tests cover service functions, Proxy error behavior, and repository-scanned JS verbs?
- Do docs include runnable examples?

---

21. References

Documentation read for this design

- `README.md:37` — current JavaScript verbs and programmable catalog overview.
- `README.md:82` — current short JS API example.
- `README.md:98` — YAML config directory workflow.
- `README.md:120` — inspect-one-side workflow.
- `README.md:177` — prepared target documentation.
- `docs/js-api.md:1` — current Promise-first `require("css-visual-diff")` API.
- `docs/js-api.md:72` — browser/page API.
- `docs/js-api.md:143` — prepare API.
- `docs/js-api.md:176` — preflight API.
- `docs/js-api.md:206` — inspect API.
- `docs/js-api.md:246` — inspectAll API.
- `docs/js-api.md:284` — catalog API.
- `docs/js-api.md:371` — YAML interop.
- `docs/js-api.md:395` — error model.
- `docs/js-api.md:428` — concurrency guidance.
- `docs/js-verbs.md:1` — repository-scanned JS verbs overview.
- `docs/js-verbs.md:24` — repository sources.
- `docs/js-verbs.md:134` — generated flags.
- `docs/js-verbs.md:182` — binding modes.
- `docs/js-verbs.md:278` — built-in catalog commands.
- `/home/manuel/code/wesen/2026-04-20--js-discord-bot/ttmp/2026/04/21/DISCORD-BOT-019--kno` — Goja Proxy pattern for Go-backed fluent builders.
- `/home/manuel/code/wesen/2026-04-20--js-discord-bot/ttmp/2026/04/21/DISCORD-BOT-019--kno` — method-owner wrong-parent error strategy.

Source files read for this design

- `internal/cssvisualdiff/config/config.go:25` — `Target` YAML schema.
- `internal/cssvisualdiff/config/config.go:34` — `PrepareSpec` YAML schema.
- `internal/cssvisualdiff/config/config.go:55` — `SectionSpec` YAML schema.

- `internal/cssvisualdiff/config/config.go:86` — `StyleSpec` YAML schema.
- `internal/cssvisualdiff/config/config.go:96` — `OutputSpec` YAML schema.
- `internal/cssvisualdiff/config/config.go:106` — top-level `Config` schema.
- `internal/cssvisualdiff/dsl/cvd_module.go:16` — module registration.
- `internal/cssvisualdiff/dsl/cvd_module.go:75` — Promise bridge.
- `internal/cssvisualdiff/dsl/cvd_module.go:96` — typed error value construction.
- `internal/cssvisualdiff/dsl/cvd_module.go:128` — browser wrapper.
- `internal/cssvisualdiff/dsl/cvd_module.go:193` — page wrapper.
- `internal/cssvisualdiff/dsl/cvd_module.go:335` — prepare spec decoder.
- `internal/cssvisualdiff/dsl/cvd_module.go:385` — probe decoder.
- `internal/cssvisualdiff/dsl/catalog_adapter.go:23` — catalog JS wrapper.
- `internal/cssvisualdiff/dsl/catalog_adapter.go:244` — lowerCamel catalog manifest conversion.
- `internal/cssvisualdiff/dsl/config_adapter.go:8` — lowerCamel YAML config conversion.
- `internal/cssvisualdiff/dsl/host.go:34` — runtime factory and `require()` loader wiring.
- `internal/cssvisualdiff/dsl/registrar.go:26` — native module registrar pattern.
- `internal/cssvisualdiff/verbcli/command.go:16` — lazy verbs command.
- `internal/cssvisualdiff/verbcli/bootstrap.go:26` — `Bootstrap` model.
- `internal/cssvisualdiff/verbcli/bootstrap.go:204` — CLI repository argument parsing.
- `internal/cssvisualdiff/verbcli/bootstrap.go:276` — repository scanning.
- `internal/cssvisualdiff/verbcli/bootstrap.go:298` — duplicate command path detection.
- `internal/cssvisualdiff/service/browser.go:11` — `BrowserService` / `PageService`.
- `internal/cssvisualdiff/service/browser.go:65` — `LoadAndPreparePage`.
- `internal/cssvisualdiff/service/preflight.go:10` — selector preflight service.
- `internal/cssvisualdiff/service/style.go:18` — computed style extraction service.
- `internal/cssvisualdiff/service/prepare.go:14` — prepare orchestration.

- `internal/cssvisualdiff/service/inspect.go:24` — inspect request model.
- `internal/cssvisualdiff/service/inspect.go:65` — inspect prepared page service.
- `internal/cssvisualdiff/service/catalog_service.go:28` — catalog manifest model.
- `internal/cssvisualdiff/service/catalog_service.go:81` — catalog constructor.
- `internal/cssvisualdiff/service/catalog_service.go:203` — manifest writer.
- `internal/cssvisualdiff/service/catalog_service.go:217` — index writer.
- `/home/manuel/code/wesen/2026-04-20--js-discord-bot/internal/jsdiscord/ui_errors.go` — concrete method-owner, wrong-parent, unknown-method, and type-mismatch helpers for Goja Proxy DSLs.
- `/home/manuel/code/wesen/2026-04-20--js-discord-bot/internal/jsdiscord/ui_embed.go` — concrete Go-backed builder wrapped with `vm.NewProxy`.

Tests read for this design

- `internal/cssvisualdiff/verbcli/command_test.go:115` — Promise-first JS module integration test.
- `internal/cssvisualdiff/verbcli/command_test.go:169` — catalog manifest/index JS verb integration test.
- `internal/cssvisualdiff/verbcli/command_test.go:231` — typed error integration test.
- `internal/cssvisualdiff/service/inspect_test.go:17` — output-file multi-request guard.
- `internal/cssvisualdiff/service/inspect_test.go:26` — prepared page reused across multiple inspect probes.
- `internal/cssvisualdiff/service/catalog_service_test.go:11` — catalog manifest/index service test.

Pixel accuracy is not a single screenshot. It is a feedback loop. You render the page, look at the part that matters, measure the DOM and CSS that produced it, change the implementation, and run the same measurement again. `css-visual-diff` gives that loop a programmable shape: browser pages, locators, probes, extractors, snapshots, diffs, reports, and catalogs.

This guide teaches that shape from the ground up. The goal is not to memorize every method. The goal is to learn how to build small scripts that answer precise visual questions while you are building a website: Does the CTA exist? Is it visible? What text does it render? What color, spacing, font size, and bounds does the browser compute? Did those facts change after my CSS edit? When the answer is yes, can I write a report that explains the change?

1. The Mental Model

A visual comparison has three different layers. Keeping them separate makes scripts easier to write and easier to debug.

Layer	Question	API shape
Page	What browser state am I inspecting?	`const page = await browser.page(url, options)`
Locator	Which element on this loaded page matters now?	`const cta = page.locator("#cta")`
Probe	What reusable recipe should I apply to a page?	`cvd.probe("cta").selector("#cta").text().style`
Extractor	What facts should I read from an element?	`cvd.extractors.text()`, `cvd.extractors.computedStyle([...])`
Snapshot	What did a set of probes observe on this page?	`await cvd.snapshot(page, probes)`
Diff	What changed between two snapshots?	`cvd.diff(before, after)`
Report	How do I show the difference to a human or CI log?	`cvd.report(diff).markdown()`

The distinction between a locator and a probe is the most important one. A locator is bound to one live page. It answers: "on this loaded page, find `#cta`." A probe is a reusable recipe. It answers: "whenever I inspect a page, call this thing `cta`, use selector `#cta`, and extract text plus these CSS properties."

When you are exploring, use locators. When you are building repeatable checks, use probes and snapshots.

2. The Smallest Useful Script

A script starts by opening a browser and page. Browser and page operations are asynchronous because they touch Chromium. Always close the browser in a `finally` block; otherwise an error in the middle of the script may leave a browser process running.

```
async function inspectHome(url) {
  const cvd = require("css-visual-diff")
  const browser = await cvd.browser()

  try {
    const page = await browser.page(url, {
      viewport: { width: 1280, height: 720 },
      waitMs: 250,
      name: "home",
    })

    const cta = page.locator("#cta")
    return {
      exists: await cta.exists(),
      visible: await cta.visible(),
      text: await cta.text({ normalizeWhitespace: true, trim: true }),
    }
  } finally {
    await browser.close()
  }
}
```

This script does not compare images yet. That is intentional. Pixel accuracy starts with confidence that you are looking at the right element. A screenshot of the wrong element can be perfectly stable and completely useless.

3. Turning a Function into a CLI Verb

A JavaScript file becomes a CLI command when it declares a verb. The metadata is static so `css-visual-diff` can scan the file without executing browser work.

```
async function inspectHome(url) {
  // same function as above
}

__verb__("inspectHome", {
  parents: ["site"],
  short: "Inspect the homepage CTA",
  fields: {
    url: { argument: true, required: true, help: "Page URL" },
```

```
  },
})
```

Run it from a repository directory:

```
css-visual-diff verbs --repository ./verbs site inspect-home http://localhost:3000 --output
```

The command name is normalized from `inspectHome` to `inspect-home`. The important design choice is that scripts live under `css-visual-diff verbs ...`, not at the root command. This keeps script discovery, duplicate command names, and repository-specific flags local to the programmable layer.

4. Reading What the Browser Actually Computed

CSS source files show intent. The browser shows the result. For pixel-accurate work, the result is what matters: computed color, actual font size, final bounds, rendered text, and real attributes after React, CSS, layout, and browser defaults have done their work.

```
async function inspectButton(url) {
  const cvd = require("css-visual-diff")
  const browser = await cvd.browser()
  try {
    const page = await browser.page(url, { viewport: { width: 390, height: 844 } })
    const button = page.locator("[data-testid='primary-cta']")

    const [status, text, bounds, styles, attrs] = await Promise.all([
      button.status(),
      button.text({ normalizeWhitespace: true, trim: true }),
      button.bounds(),
      button.computedStyle(["display", "height", "padding-left", "padding-right", "font-size
      button.attributes(["class", "aria-label", "disabled"]),
    ])

    return { status, text, bounds, styles, attrs }
  } finally {
    await browser.close()
  }
}
```

`Promise.all` is safe here. Internally, operations on one page are serialized so Chromium is not asked to do conflicting page work at the same time. The JavaScript stays ergonomic while the Go side preserves page safety.

5. From Exploration to Repeatable Probes

Locators are good while exploring. Probes are better once you know what matters. A probe names the check, stores the selector, and declares which

extractors should run.

```
const ctaProbe = cvd.probe("primary-cta")
  .selector("[data-testid='primary-cta']")
  .required()
  .text()
  .bounds()
  .styles(["height", "font-size", "font-weight", "color", "background-color", "border-radius
  .attributes(["class", "aria-label"])
```

The probe is a Go-backed builder. If you call a method that belongs somewhere else, the API tells you. For example, `.computedStyle(...)` belongs to locators, while `.styles(...)` belongs to probes. This distinction helps both humans and LLM-written scripts recover from mistakes.

6. Snapshots: Capture the Facts You Care About

A snapshot applies a set of probes to one loaded page and returns plain serializable data. It does not write artifacts unless you explicitly choose another API that writes files. This makes snapshots ideal for quick authoring loops and CI assertions.

```
async function snapshotHome(url) {
  const cvd = require("css-visual-diff")
  const browser = await cvd.browser()
  try {
    const page = await browser.page(url, { viewport: cvd.viewport.desktop() })
    return await cvd.snapshot(page, [
      cvd.probe("hero-title").selector("h1").text().styles(["font-size", "line-height", "col
      cvd.probe("primary-cta").selector("[data-testid='primary-cta']").text().bounds().style
    ])
  } finally {
    await browser.close()
  }
}
```

The result has one entry per probe. Each entry contains the probe name, selector, and extracted snapshot data. Because the result is plain data, you can write it to JSON, compare it, store it in a catalog, or feed it into another script.

7. Diffing Two States

The simplest useful comparison is structural: take two snapshots and ask what changed. This works well for text, CSS properties, attributes, and bounds.

```
const before = await snapshotHome("http://localhost:3000/before")
const after = await snapshotHome("http://localhost:3000/after")
```

```
const diff = cvd.diff(before, after, {
  ignorePaths: [
    "results[1].snapshot.bounds.x",
    "results[1].snapshot.bounds.y",
  ],
})

if (!diff.equal) {
  console.log(cvd.report(diff).markdown())
}
```

Ignore paths are useful when a value is expected to move but the important visual contract is somewhere else. For example, a responsive layout may shift `x` and `y` while preserving height, typography, and color. Ignore paths let you state that policy directly.

8. Writing Reports and Artifacts

A script becomes useful to a team when it leaves evidence behind. `cvd.write.json(...)` writes machine-readable data. `cvd.report(diff).writeMarkdown(...)` writes a human-readable explanation.

```
async function compareAndWrite(beforeUrl, afterUrl, outDir) {
  const before = await snapshotHome(beforeUrl)
  const after = await snapshotHome(afterUrl)
  const diff = cvd.diff(before, after)

  await cvd.write.json(`${outDir}/before.json`, before)
  await cvd.write.json(`${outDir}/after.json`, after)
  await cvd.write.json(`${outDir}/diff.json`, diff)
  await cvd.report(diff).writeMarkdown(`${outDir}/diff.md`)

  return {
    equal: diff.equal,
    changeCount: diff.changeCount,
    report: `${outDir}/diff.md`,
  }
}
```

For CI, return structured rows and let Glazed format them as JSON, YAML, table, or CSV. For humans, write Markdown reports and screenshots/catalogs when needed.

9. When to Use Inspect Artifacts Instead

Snapshots are compact and script-friendly. Inspect artifacts are evidence-rich. Use `page.inspect(...)` or `page.inspectAll(...)` when you need screen-

shots, prepared HTML, CSS JSON/Markdown, or DOM inspection JSON on disk.

Use snapshots when…	Use inspect artifacts when…
You need fast data comparisons.	A human must review a screenshot or HTML file.
You want in-memory diffs.	You need a durable artifact bundle.
You are iterating on CSS selectors.	You are preparing CI evidence or a catalog.
You are writing custom assertions.	You want the standard bundle shape.

A good workflow often uses both. During authoring, use locators and snapshots to quickly answer questions. Before review or CI, write inspect artifacts and a catalog so the evidence is durable.

10. A Full Authoring Loop

Here is a complete script that supports the kind of pixel-accuracy loop you use while building a page. It captures a baseline, captures the current implementation, diffs them, writes evidence, and returns a structured summary.

```
async function compareCheckout(beforeUrl, afterUrl, outDir) {
  const cvd = require("css-visual-diff")

  async function capture(url) {
    const browser = await cvd.browser()
    try {
      const page = await browser.page(url, { viewport: cvd.viewport.desktop(), waitMs: 250 }
      return await cvd.snapshot(page, [
        cvd.probe("checkout-title").selector("h1").text().styles(["font-size", "font-weight'
        cvd.probe("checkout-card").selector("[data-testid='checkout-card']").bounds().styles
        cvd.probe("pay-button").selector("[data-testid='pay-button']").text().bounds().style
      ])
    } finally {
      await browser.close()
    }
  }

  const before = await capture(beforeUrl)
  const after = await capture(afterUrl)
  const diff = cvd.diff(before, after)

  await cvd.write.json(`${outDir}/before.json`, before)
  await cvd.write.json(`${outDir}/after.json`, after)
  await cvd.write.json(`${outDir}/diff.json`, diff)
```

```
  await cvd.report(diff).writeMarkdown(`${outDir}/diff.md`)

  return {
    ok: diff.equal,
    changeCount: diff.changeCount,
    report: `${outDir}/diff.md`,
  }
}

__verb__("compareCheckout", {
  parents: ["site"],
  short: "Compare checkout page visual facts",
  fields: {
    beforeUrl: { argument: true, required: true },
    afterUrl: { argument: true, required: true },
    outDir: { argument: true, required: true },
  },
})
```

Run it:

```
css-visual-diff verbs --repository ./verbs site compare-checkout \
  http://localhost:3000/baseline/checkout \
  http://localhost:3000/checkout \
  /tmp/cssvd-checkout \
  --output json
```

11. Debugging the Feedback Loop

When a script fails, debug from the outside in. First prove the page loaded. Then prove the selector exists. Then prove the element is visible. Only then inspect CSS values or compare snapshots.

Problem	Likely cause	What to do
`exists` is false	The selector does not match the prepared DOM.	Use `page.locator(selector).status()` and inspect prepared HTML with `css-visual-diff html`.
`visible` is false	The element exists but has zero bounds, `display:none`, or hidden visibility.	Extract bounds and computed `display` / `visibility`.

Problem	Likely cause	What to do
Text differs only in spacing	DOM text contains newlines or multiple spaces.	Use `text({ normalizeWhitespace: true, trim: true })`.
Diff changes bounds on every run	Layout is responsive or unstable.	Ignore expected paths or compare more stable CSS/text facts.
Script says a method belongs elsewhere	Locator/probe/extractor concepts were mixed.	Use locators for live page reads, probes for reusable recipes, extractors for what to read.

12. Key Points

- Pixel accuracy is a loop, not a one-time screenshot. The loop is render, locate, extract, compare, report, and adjust.
- Locators are page-bound handles. They are best for exploration and direct questions about the current page.
- Probes are reusable recipes. They are best when you want repeatable checks across targets, runs, or CI jobs.
- Extractors make the question explicit. A script that asks for `text`, `bounds`, and `computedStyle(["color"])` is easier to review than a script that dumps everything.
- Snapshots are plain data. They are cheap to diff, write, store, and reason about.
- Inspect artifacts are durable evidence. Use them when a human needs screenshots, HTML, or full CSS/DOM files.

See Also

- `css-visual-diff help javascript-api`
- `css-visual-diff help javascript-verbs`
- `css-visual-diff help inspect-workflow`
- `css-visual-diff help config-selectors`

`css-visual-diff` exposes a Promise-first JavaScript API for repository-scanned verbs. Scripts use it to drive Chromium pages, prepare targets, preflight selectors, inspect artifacts, and write visual catalog manifests.

This API is available inside scripts executed by:

```
css-visual-diff verbs ...
```

It is intentionally asynchronous from day one. Any operation that touches Chromium, timers, files, or catalog writes returns a Promise.

Quick example

```
async function inspect(url, selector, outDir) {
  const cvd = require("css-visual-diff")
  const browser = await cvd.browser()

  try {
    const page = await browser.page(url, {
      viewport: { width: 1280, height: 720 },
      waitMs: 250,
      name: "homepage",
    })

    const probes = [{
      name: "cta",
      selector,
      props: ["display", "color", "background-color", "font-size"],
      attributes: ["class"],
    }]

    const preflight = await page.preflight(probes)
    if (!preflight[0].exists) {
      throw new cvd.SelectorError(`selector did not match: ${selector}`)
    }

    const result = await page.inspectAll(probes, {
      outDir,
      artifacts: "css-json",
    })

    return {
      ok: true,
      outputDir: result.outputDir,
      resultCount: result.results.length,
    }
  } finally {
```

```
    await browser.close()
  }
}
```

Module exports

```
const cvd = require("css-visual-diff")
```

Exports:

- `cvd.browser(options?)`
- `cvd.catalog(options)`
- `cvd.loadConfig(path)`
- `cvd.viewport(width, height)` and named viewport helpers
- `cvd.target(name)`
- `cvd.probe(name)`
- `cvd.extractors.*`
- `cvd.extract(locator, extractors)`
- `cvd.snapshot(page, probes, options?)`
- `cvd.diff(before, after, options?)`
- `cvd.report(diff)`
- `cvd.write.json(path, value)`
- `cvd.write.markdown(path, markdown)`
- `cvd.CvdError`
- `cvd.SelectorError`
- `cvd.PrepareError`
- `cvd.BrowserError`
- `cvd.ArtifactError`

Browser and page API

`await cvd.browser(options?)`

Creates a Chromium-backed browser service.

```
const browser = await cvd.browser()
```

Current options are reserved for future use.

`await browser.page(url, options?)`

Creates a new page, sets the viewport, navigates to `url`, waits if requested, and applies no prepare step unless the target has one.

```
const page = await browser.page("http://localhost:3000", {
  viewport: { width: 1280, height: 720 },
  waitMs: 500,
  name: "prototype",
})
```

Options:

- `viewport.width` — viewport width, default `1280`
- `viewport.height` — viewport height, default `720`
- `waitMs` — wait after navigation in milliseconds
- `name` — target/page name used in metadata

`await browser.newPage()`

Creates a blank page. Use `page.goto(...)` afterwards.

```
const page = await browser.newPage()
await page.goto(url, { viewport: { width: 800, height: 600 } })
```

`await browser.close()`

Closes the browser service.

Always close browsers in `finally` blocks.

Page methods

`await page.goto(url, options?)`

Navigates an existing page.

```
const target = await page.goto(url, {
  viewport: { width: 1024, height: 768 },
  waitMs: 200,
  name: "target-name",
})
```

Returns:

```
{
  name: "target-name",
  url: "...",
  waitMs: 200,
  viewport: { width: 1024, height: 768 }
}
```

`await page.prepare(spec)`

Runs a prepare step on the already-loaded page.

Script prepare:

```
await page.prepare({
  type: "script",
  waitFor: "window.React && window.ReactDOM",
  waitForTimeoutMs: 5000,
```

```
  script: `document.body.dataset.ready = "true"`,
  afterWaitMs: 250,
})
```

Direct React global prepare:

```
await page.prepare({
  type: "directReactGlobal",
  waitFor: "window.React && window.ReactDOM && window.PPXDesktop",
  component: "PPXDesktop",
  props: { page: "shows" },
  rootSelector: "#capture-root",
  width: 920,
  minHeight: 1200,
  background: "#fff",
})
```

`directReactGlobal` is a prepare/rendering mode, not a selector mode. It creates or targets a controlled root and renders a global React component into it before inspection.

`await page.preflight(probes)`

Checks selectors before expensive extraction.

```
const statuses = await page.preflight([
  { name: "cta", selector: "#cta", source: "styles", required: true },
])
```

Result entries are lowerCamel objects:

```
{
  name: "cta",
  selector: "#cta",
  source: "styles",
  exists: true,
  visible: true,
  bounds: { x: 10, y: 20, width: 120, height: 40 },
  textStart: "Book now",
  error: ""
}
```

Use preflight to decide authoring vs CI policy:

- authoring mode: record misses and continue,
- CI mode: throw or rethrow on missing selectors.

`await page.inspect(probe, options)`

Inspects one probe and writes artifacts.

```
const artifact = await page.inspect(
  { name: "cta", selector: "#cta", props: ["color"] },
  { outDir: "/tmp/cssvd/cta", artifacts: "css-json" }
)
```

Result:

```
{
  metadata: {
    side: "script",
    targetName: "prototype",
    url: "http://...",
    viewport: { width: 1280, height: 720 },
    name: "cta",
    selector: "#cta",
    selectorSource: "styles",
    rootSelector: "",
    prepareType: "",
    format: "css-json",
    createdAt: "2026-04-24T...Z"
  },
  style: {
    exists: true,
    computed: { color: "rgb(255, 0, 0)" },
    bounds: { x: 0, y: 0, width: 100, height: 32 },
    attributes: { class: "button" }
  },
  screenshot: "",
  html: "",
  inspectJson: ""
}
```

`await page.inspectAll(probes, options)`

Inspects multiple probes against one already-loaded/prepared page.

```
const result = await page.inspectAll(probes, {
  outDir: "/tmp/cssvd/catalog/artifacts/homepage",
  artifacts: "bundle",
})
```

Result:

```
{
  outputDir: "/tmp/cssvd/catalog/artifacts/homepage",
  results: [ /* inspect artifacts */ ]
}
```

For multiple probes, artifacts are written below per-probe subdirectories. For one probe, artifacts are written directly into `outDir`.

`page.locator(selector)`

Creates a synchronous page-bound locator handle. Creating a locator does not query the browser. The async locator methods do.

```
const cta = page.locator("#cta")
const exists = await cta.exists()
```

Locator methods:

- `await locator.status()` — returns selector status with existence, visibility, bounds, text start, and selector error.
- `await locator.exists()` — returns a boolean.
- `await locator.visible()` — returns a boolean.
- `await locator.text(options?)` — returns text content. Use `{ normalizeWhitespace: true, trim: true }` for stable comparisons.
- `await locator.bounds()` — returns `{ x, y, width, height }` or `null` for a missing selector.
- `await locator.computedStyle(props)` — returns a map of CSS property values.
- `await locator.attributes(names)` — returns a map of attribute values.

Example:

```
const cta = page.locator("#cta")
const [text, bounds, styles] = await Promise.all([
  cta.text({ normalizeWhitespace: true, trim: true }),
  cta.bounds(),
  cta.computedStyle(["height", "color", "background-color"]),
])
```

Operations on one page are serialized internally, so `Promise.all` is safe for page-bound reads.

`await page.close()`

Closes the page.

Lower-level builders, extraction, snapshots, and diffs

The lower-level API is for script-native visual checks that do not always need inspect artifacts. Builder and handle objects are Go-backed values with controlled methods. If you call a method on the wrong object, the API reports which object owns that method.

`cvd.viewport(...)`

```
cvd.viewport(1280, 720)
cvd.viewport({ width: 1280, height: 720 })
cvd.viewport.desktop()
cvd.viewport.tablet()
cvd.viewport.mobile()
```

`cvd.target(name)`

Builds a page target definition.

```
const target = cvd.target("homepage")
  .url("http://localhost:3000")
  .viewport(cvd.viewport.desktop())
  .waitMs(250)
  .root("#app")
  .build()
```

`cvd.probe(name)`

Builds a reusable inspection recipe.

```
const probe = cvd.probe("cta")
  .selector("#cta")
  .required()
  .text()
  .bounds()
  .styles(["color", "font-size", "background-color"])
  .attributes(["class"])
```

Use probes with `cvd.snapshot(...)`. Use locators when you want to inspect one already-loaded page directly.

`cvd.extractors.*`

Extractor handles describe which facts to read from a locator.

```
const extractors = [
  cvd.extractors.exists(),
  cvd.extractors.visible(),
  cvd.extractors.text(),
  cvd.extractors.bounds(),
  cvd.extractors.computedStyle(["color"]),
  cvd.extractors.attributes(["id", "class"]),
]
```

`await cvd.extract(locator, extractors)`

Strictly extracts facts from a page-bound locator. The first argument must be a locator returned by `page.locator(...)`. The second argument must be an array of `cvd.extractors.*` handles.

```
const snapshot = await cvd.extract(page.locator("#cta"), [
  cvd.extractors.exists(),
  cvd.extractors.text(),
  cvd.extractors.computedStyle(["color"]),
])
```

Raw object locators are rejected. Use `page.locator("#selector")` instead.

`await cvd.snapshot(page, probes, options?)`

Strictly evaluates probe builders against a page.

```
const snapshot = await cvd.snapshot(page, [
  cvd.probe("title").selector("h1").text().styles(["font-size", "color"]),
  cvd.probe("cta").selector("#cta").text().bounds().styles(["background-color"]),
])
```

Raw object probes are rejected. Use `cvd.probe("name").selector("...")` builders.

`cvd.diff(...)`, `cvd.report(...)`, and `cvd.write.*`

Compare two plain snapshot-like values and write evidence.

```
const diff = cvd.diff(before, after, {
  ignorePaths: ["results[0].snapshot.bounds.x"],
})

const markdown = cvd.report(diff).markdown()
await cvd.write.json("out/diff.json", diff)
await cvd.report(diff).writeMarkdown("out/diff.md")
```

The current diff is a deterministic structural JSON diff. CSS-aware normalization and numeric tolerances can be layered on top later.

Artifact formats

Supported `artifacts` / `format` values:

- `bundle` — metadata, screenshot, prepared HTML, CSS JSON/Markdown, DOM inspect JSON
- `png` — selector screenshot
- `html` — prepared HTML for selector/root
- `css-json` — computed CSS JSON

- `css-md` — computed CSS Markdown
- `inspect-json` — DOM inspection JSON
- `metadata-json` — metadata JSON only

Catalog API

The catalog implementation is Go-backed. JavaScript orchestrates workflows, while Go owns schema versioning, path normalization, summaries, manifest writing, and index writing.

```
const catalog = cvd.catalog({
  title: "Prototype Catalog",
  outDir: "/tmp/catalog",
  artifactRoot: "artifacts",
  indexName: "index.md",
})
```

`catalog.artifactDir(slug)`

Returns a normalized artifact directory below `outDir/artifactRoot`.

```
catalog.artifactDir("../Prototype Public Shows!")
// /tmp/catalog/artifacts/prototype-public-shows
```

`catalog.addTarget(target)`

Adds a target record.

```
catalog.addTarget({
  slug: "homepage",
  name: "Homepage",
  url: "http://localhost:3000",
  selector: "#root",
  viewport: { width: 1280, height: 720 },
  metadata: { source: "storybook" },
})
```

`catalog.recordPreflight(target, statuses)`

Records selector preflight results.

`catalog.addResult(target, inspectResult)`

Records a `page.inspectAll(...)`-shaped result.

`catalog.addFailure(target, error)`

Records a failure. If passed a typed `cvd.*Error`, the catalog captures name/code/operation/message.

catalog.summary()

Returns lowerCamel summary counts:

```
{
  targetCount: 1,
  preflightCount: 1,
  resultCount: 1,
  failureCount: 0,
  artifactCount: 1
}
```

catalog.manifest()

Returns the current manifest object in lowerCamel form.

await catalog.writeManifest()

Writes:

`<outDir>/manifest.json`

Returns the manifest path.

await catalog.writeIndex()

Writes:

`<outDir>/<indexName>`

Returns the index path.

YAML interop

`cvd.loadConfig(path)` loads an existing `*.css-visual-diff.yml` file through the Go config loader and returns a lowerCamel JS object.

```
const cfg = await cvd.loadConfig("examples/pyxis-public-shows.yaml")
const target = cfg.original
const probes = cfg.styles.map((style) => ({
  name: style.name,
  selector: style.selectorOriginal || style.selector,
  props: style.props,
  attributes: style.attributes,
}))
```

The built-in command below uses this helper:

```
css-visual-diff verbs catalog inspect-config \
  ./example.css-visual-diff.yml original /tmp/catalog \
```

```
  --artifacts css-json \
  --output json
```

Error model

Promise rejections use JS-visible typed errors:

```
try {
  await page.inspect({ name: "missing", selector: "#missing" }, { outDir, artifacts: "html"
} catch (err) {
  if (err instanceof cvd.SelectorError) {
    // selector/preflight failure
  }
  if (err instanceof cvd.CvdError) {
    console.error(err.code, err.operation, err.message)
  }
}
```

Classes:

- `CvdError` — base class
- `SelectorError` — selector/preflight failures
- `PrepareError` — prepare/wait failures
- `BrowserError` — browser/page/navigation failures
- `ArtifactError` — inspect/artifact writing failures

Each error has:

- `name`
- `code`
- `operation`
- `details`
- `message`

Concurrency guidance

Prefer coarse target/page-level parallelism. For example, process independent catalog targets with a worker limit and give each worker its own page.

Avoid assuming per-page CDP operations are meaningfully parallel. Operations on one Chromium page are effectively serialized by chromedp/CDP and should be awaited in order:

```
for (const target of targets) {
  const page = await browser.page(target.url, { viewport: target.viewport })
  try {
    await page.prepare(target.prepare)
    const preflight = await page.preflight(target.probes)
    const result = await page.inspectAll(target.probes, { outDir: catalog.artifactDir(target
```

```
    catalog.addResult(target, result)
  } finally {
    await page.close()
  }
}
```

Add explicit worker limits before scaling this pattern to many pages.

`css-visual-diff verbs` turns annotated JavaScript files into CLI commands. It is intended for programmable visual catalog workflows where YAML alone is too static.

Command namespace

Repository-scanned commands live under:

```
css-visual-diff verbs ...
```

They are not injected at the root of the CLI. This keeps scan errors, duplicate command paths, and script-specific flags local to the `verbs` subtree.

Built-ins include:

```
css-visual-diff verbs script compare region ...
css-visual-diff verbs script compare brief ...
css-visual-diff verbs catalog inspect-page ...
css-visual-diff verbs catalog inspect-config ...
```

Repository sources

The lazy `verbs` command discovers scripts from four sources:

1. embedded built-ins,
2. app config,
3. `CSS_VISUAL_DIFF_VERB_REPOSITORIES`,
4. CLI flags.

CLI repository

```
css-visual-diff verbs --repository ./examples/verbs examples catalog inspect-page \
  http://127.0.0.1:8767/ '#cta' /tmp/cssvd-example \
  --output json
```

`--verb-repository` is an alias for `--repository`.

Environment repository

```
export CSS_VISUAL_DIFF_VERB_REPOSITORIES="/path/to/project/verbs:/path/to/other/verbs"
css-visual-diff verbs custom command ...
```

Use the platform path-list separator (`:` on Linux/macOS).

App config repositories

App config can declare repositories:

```
verbs:
  repositories:
    - name: local
```

```
      path: ./verbs
    - name: disabled
      path: ./disabled
      enabled: false
```

Relative paths are resolved relative to the config file.

Minimal verb file

```
function hello(name) {
  return `hello ${name}`
}

__verb__("hello", {
  parents: ["custom"],
  output: "text",
  fields: {
    name: { argument: true, required: true }
  }
})
```

Run:

```
css-visual-diff verbs --repository ./verbs custom hello Manuel
```

Sentinels

Supported metadata sentinels:

- `__package__({...})`
- `__section__("slug", {...})`
- `__verb__("functionName", {...})`
- `doc`...`

These are statically scanned to generate commands. At runtime they are installed as no-op helpers so the script can be required safely.

Metadata must be static literal JavaScript: object literals, arrays, strings, numbers, booleans, and null. Do not use computed values, spreads, function calls, or template substitutions inside metadata.

Package metadata

`__package__` controls default command grouping.

```
__package__({
  name: "catalog",
  parents: ["examples"],
  short: "Example catalog workflows"
})
```

Verb metadata

```
__verb__("inspectPage", {
  parents: ["examples", "catalog"],
  short: "Inspect one page into a catalog",
  output: "structured",
  fields: {
    url: { argument: true, required: true, help: "URL to inspect" },
    selector: { argument: true, required: true, help: "CSS selector" },
    outDir: { argument: true, required: true, help: "Output directory" },
    values: { bind: "all" },
    failOnMissing: { type: "bool", default: false },
  }
})
```

Command names are normalized for CLI usage. A function named `inspectPage` becomes command `inspect-page`.

Fields and generated flags

Common field keys:

- `type`
- `help`
- `short`
- `default`
- `choices`
- `required`
- `argument` / `arg`
- `bind`
- `section`

Supported field types:

- `string`
- `bool` / `boolean`
- `int` / `integer`
- `float` / `number`
- `stringList` / `list` / `[]string`
- `choice`
- `choiceList`

Argument fields become positional arguments. Other fields become flags.

Example:

```
fields: {
  url: { argument: true, required: true },
  artifacts: {
    type: "choice",
```

```
    choices: ["bundle", "css-json", "html"],
    default: "css-json",
  },
  failOnMissing: { type: "bool", default: false },
}
```

Generates usage like:

```
verbs examples catalog inspect-page <url> [flags]
  --artifacts css-json
  --failOnMissing
```

Binding modes

Positional/default binding

A parameter with the same name as a field receives that value:

```
function inspect(url, selector, outDir) {}
```

`bind: "all"`

Passes one object containing all resolved field values:

```
function inspectPage(url, selector, outDir, values) {
  if (values.failOnMissing) { /* ... */ }
}

__verb__("inspectPage", {
  fields: {
    url: { argument: true, required: true },
    selector: { argument: true, required: true },
    outDir: { argument: true, required: true },
    values: { bind: "all" },
    failOnMissing: { type: "bool", default: false },
  }
})
```

`bind: "context"`

Passes invocation context:

```
function debug(ctx) {
  return ctx
}

__verb__("debug", {
  fields: { ctx: { bind: "context" } }
})
```

Context contains verb name, function name, module path, source file, root directory, raw values, and section values.

Section binding

Sections group related flags.

```
__section__("target", {
  fields: {
    url: { type: "string", required: true },
    waitMs: { type: "int", default: 0 },
  }
})

function run(target) {
  return target.url
}

__verb__("run", {
  fields: { target: { bind: "target" } }
})
```

Output modes

Default output is structured/Glazed output. Return objects or arrays to get rows that can be formatted as table/json/yaml/csv.

```
css-visual-diff verbs ... --output json
```

For plain text commands:

```
__verb__("brief", {
  output: "text",
  fields: { /* ... */ }
})
```

Common output aliases include:

- `structured`
- `glaze`
- `table`
- `text`
- `raw`
- `writer`
- `plain`

Prefer structured output for catalog workflows and write large artifacts to files.

Built-in catalog commands

Inspect one page

```
css-visual-diff verbs catalog inspect-page \
  http://127.0.0.1:8767/ '#cta' /tmp/cssvd-page \
  --slug cta \
  --name CTA \
  --artifacts css-json \
  --output json
```

Authoring mode: selector misses are recorded and returned as structured rows without failing:

```
css-visual-diff verbs catalog inspect-page \
  http://127.0.0.1:8767/ '#missing' /tmp/cssvd-authoring \
  --slug missing \
  --output json
```

CI mode: selector misses write manifest/index and exit non-zero:

```
css-visual-diff verbs catalog inspect-page \
  http://127.0.0.1:8767/ '#missing' /tmp/cssvd-ci \
  --slug missing \
  --failOnMissing \
  --output json
```

Inspect a YAML config

```
css-visual-diff verbs catalog inspect-config \
  ./page.css-visual-diff.yml original /tmp/cssvd-config \
  --artifacts css-json \
  --output json
```

`inspect-config` loads the Go YAML config, picks `original` or `react`, derives probes from `styles` first and `sections` second, then writes a catalog manifest/index.

Duplicate command paths

If two repositories define the same generated command path, command construction fails with a clear error such as:

```
duplicate jsverb path "script compare region" from builtin:scripts/compare.js and /repo/verb
```

Resolve by changing `parents`, function name, or `__verb__({ name/command })` metadata.

Migration note

Earlier prototypes injected generated JavaScript commands at the root command. The supported shape is now:

```
css-visual-diff verbs ...
```

This avoids surprising root command conflicts and makes script repository problems local to the `verbs` subtree.

Replay and smoke scripts

The ticket for the Goja/jsverbs work includes reproducible scripts under:

```
ttmp/2026/04/24/CSSVD-GOJA-JS-API--design-goja-javascript-api-for-programmable-visual-catalo
```

Useful binary smokes:

```
scripts/006-binary-help-smoke.sh
scripts/007-binary-js-api-success-smoke.sh
scripts/008-binary-js-api-typed-error-smoke.sh
scripts/009-binary-catalog-smoke.sh
scripts/010-binary-built-in-catalog-inspect-page-smoke.sh
scripts/011-binary-built-in-catalog-inspect-config-smoke.sh
```

Diary

Goal

This diary records the research and documentation work for designing a more flexible, lower-level JavaScript API for `css-visual-diff`. The goal was to create a new ticket and write an intern-facing guide that explains the current system, the gaps in the current high-level JS API, and a phased implementation plan for locators, extractors, snapshots, diffs, fluent builders, workflow composition, and YAML migration.

Step 1: Create the ticket and gather evidence

I created a new ticket workspace for the lower-level JavaScript API design and then read the existing documentation and implementation files that define the current behavior. This step established the baseline: the current JS API is Promise-first and useful, but it is intentionally close to the high-level CLI/YAML concepts.

The important conclusion from this evidence pass was that the new API should be additive. It should not replace `page.preflight()` / `page.inspectAll()` immediately. Instead, it should add lower-level primitives beneath those methods and reuse the same Go service layer.

Prompt Context

User prompt (verbatim): "Ok, read the documents, create a new ticket for a more flexible JS API, and reate a detailed analysis / design / implementation guide that is very detailed for a new intern, explaining all the parts of the system needed to understand what it is, with prose paragraphs and bullet point sand pseudocode and diagrams and api references and file references. It should be very clear and detailed. Store in the ticket and the nupload to remarkable."

Assistant interpretation: Create a new ticket, study the existing CSS visual diff documentation/source, write a very detailed design and implementation guide for a flexible lower-level JS API, store it in the ticket, and upload it to reMarkable.

Inferred user intent: The user wants the next API design to be concrete enough that an intern can implement it without needing to rediscover the existing architecture or design rationale.

What I did

- Created ticket `CSSVD-FLEX-JS-API` under the repository-local `ttmp` root.
- Created a primary design document and an investigation diary using `docmgr`.
- Read current user-facing docs:

  - `README.md`
  - `docs/js-api.md`
  - `docs/js-verbs.md`
- Read the prior Goja/jsverbs design guide:
  - `ttmp/2026/04/24/CSSVD-GOJA-JS-API--design-goja-javascript-api-for-programmable-visu`
- Read current source files for:
  - YAML schema and config loading,
  - native Goja module registration,
  - catalog/config adapters,
  - lazy JS verbs command discovery,
  - browser/preflight/style/prepare/inspect/catalog services,
  - integration tests proving the current API.

Why

A design for a lower-level API has to be grounded in what already exists. The repository already has a working Goja/jsverbs layer and a reusable service layer, so the correct design is not greenfield. The guide needed to explain how to extend the current system safely.

What worked

- `docmgr ticket create-ticket --root ./ttmp ...` created the ticket in the repository-local `ttmp` directory.
- The current docs clearly distinguish the high-level JS API from repository-scanned verbs.
- The current service layer already provides the right foundation for lower-level APIs: browser/page, prepare, preflight, style extraction, inspect artifacts, and catalog writing.

What didn't work

- No command failure occurred during this step.
- The only known environmental caveat from the previous ticket remains relevant: `docmgr status` still resolves a parent workspace config by default, so I explicitly used `--root ./ttmp` when creating the ticket and docs.

What I learned

- `docs/js-api.md` already documents Promise-first behavior, typed errors, preflight, inspect, catalog, YAML interop, and concurrency guidance.
- `docs/js-verbs.md` already documents the operator-facing repository-scanned CLI layer.
- `internal/cssvisualdiff/dsl/cvd_module.go` is the current adapter hot spot and should be split before adding many more methods.

- `internal/cssvisualdiff/service` is the correct place to add locator, extraction, snapshot, and diff logic.

What was tricky to build

The tricky part was drawing a clear boundary between "current implemented API" and "future proposed API." The design must not imply that locators, extractors, snapshots, or diffs already exist. It therefore names those as proposed layers and maps each one to concrete future files and phases.

What warrants a second pair of eyes

- Whether `page.locator()` should mimic Playwright-style behavior or remain intentionally smaller.
- Whether `cvd.diff()` should start as a pure structural JSON diff or immediately include CSS-aware normalization.
- Where exactly to implement the per-page operation queue: `pageState` in the adapter or `service.PageService` in the service layer.

What should be done in the future

- Run `docmgr doctor` after all documents are written.
- Upload the resulting document bundle to reMarkable.
- When implementation begins, start with adapter refactoring and page-operation serialization before adding locator methods.

Code review instructions

- Start with the design guide:
  - `ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/des`
- Check the references section for all files read.
- Validate that proposed phases preserve existing high-level JS API compatibility.

Technical details

Commands used:

```
docmgr status --summary-only

docmgr ticket create-ticket \
  --root ./ttmp \
  --ticket CSSVD-FLEX-JS-API \
  --title "Design a flexible lower-level JavaScript API" \
  --topics css-visual-diff,javascript,goja,api-design,visual-diff

docmgr doc add \
  --root ./ttmp \
```

```
  --ticket CSSVD-FLEX-JS-API \
  --doc-type design-doc \
  --title "Flexible JavaScript API analysis design and implementation guide"

docmgr doc add \
  --root ./ttmp \
  --ticket CSSVD-FLEX-JS-API \
  --doc-type reference \
  --title "Investigation diary"
```

Step 2: Write the primary design guide

I wrote the primary analysis/design/implementation guide. The document explains the current system, defines vocabulary, maps current YAML concepts to proposed JavaScript concepts, gives detailed API sketches, proposes implementation files, includes pseudocode and Mermaid diagrams, and lays out phased implementation and validation plans.

The design intentionally proposes a layered API rather than one large new abstraction. The recommended implementation order is: refactor adapters, serialize per-page operations, implement DOM locator services, expose `page.locator()`, add extractor builders and `cvd.extract()`, add builders, add snapshots, add diffs/reports, then add workflow builders last.

Prompt Context

User prompt (verbatim): same as Step 1.

Assistant interpretation: Produce the requested detailed intern-facing design guide and store it in the ticket.

Inferred user intent: Make the next implementation phase easy to delegate and review.

What I did

- Wrote `design-doc/01-flexible-javascript-api-analysis-design-and-implementation-guide.md`
- Updated `index.md` with links and ticket overview.
- Updated `tasks.md` with completed documentation work, delivery tasks, and future implementation phases.
- Updated `changelog.md` with the documentation milestone.
- Updated this diary.

Why

A lower-level JS API will touch many parts of the system: Goja runtime adapters, services, selectors, DOM extraction, artifact writing, catalog data, and CLI verbs. The guide gives an intern a map before implementation begins.

What worked

- The existing codebase already has strong seams for the design:
  - `service/browser.go` for browser/page ownership,
  - `service/preflight.go` for selector status,
  - `service/style.go` for computed CSS,
  - `service/inspect.go` for artifact writing,
  - `service/catalog_service.go` for Go-backed manifests and indexes,
  - `dsl/cvd_module.go` for Promise-first Goja exposure.
- The docs already contain enough current API references to ground the proposal.

What didn't work

- No command failure occurred while writing the documents.
- Upload is not yet completed at this diary step.

What I learned

The lower-level API should be implemented as an additive second layer, not as a replacement. The safest first public feature is `page.locator()` plus a handful of async locator methods. Workflow builders should come much later.

What was tricky to build

The design needed to be detailed without overcommitting to too much implementation in one PR. I addressed this by separating:

- API reference sketches,
- service architecture,
- Goja adapter pseudocode,
- phased implementation plan,
- acceptance criteria.

What warrants a second pair of eyes

- The proposed `cvd.extract()` result shape.
- The proposed `cvd.snapshot()` vs current `page.inspectAll()` relationship.
- The proposed file split under `internal/cssvisualdiff/dsl`.

What should be done in the future

- Convert the API reference sections into `docs/js-low-level-api.md` once implementation starts.
- Add smoke scripts under this ticket during implementation.
- Consider generating TypeScript declaration files from the documented API once stable.

Code review instructions

Read the design in this order:

1. Executive summary.
2. Section 3, current system.
3. Section 6, proposed API layers.
4. Section 9, implementation architecture.
5. Section 12, implementation phases.
6. Section 19, file-by-file implementation guide.

Technical details

Main written file:

`ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/design-doc/0`

Step 3: Validate docs and skip reMarkable upload for now

I validated the ticket documentation and prepared the PDF bundle for delivery. The reMarkable upload path hit the same cloud mutation/upload failure pattern seen earlier, so the upload was intentionally skipped after the user asked to skip it for now.

The useful result is that the ticket and local PDF are ready. The remaining blocker is only external reMarkable upload behavior, not the design documentation itself.

Prompt Context

User prompt (verbatim): "ok skip the upload for now"

Assistant interpretation: Stop retrying reMarkable upload and leave the generated local ticket/PDF artifacts in place.

Inferred user intent: Avoid spending more time on the known reMarkable cloud upload failure and finish the documentation handoff.

What I did

- Ran `docmgr doctor --root ./ttmp --ticket CSSVD-FLEX-JS-API --stale-after 30`.
- Fixed unknown-topic warnings by changing ticket topics to known vocabulary values.
- Re-ran doctor successfully.
- Ran `remarquee status` and `remarquee cloud account --non-interactive` successfully.
- Ran a dry-run bundle upload successfully.
- Tried actual bundle upload to `/ai/2026/04/24/CSSVD-FLEX-JS-API`; it failed creating the remote ticket directory with HTTP 400.

- Tried actual bundle upload directly to `/ai/2026/04/24`; it failed uploading the PDF with HTTP 400.
- Generated a local PDF manually with pandoc at `pdf/CSSVD-FLEX-JS-API-flexible-javascript-api-d`
- Tried `remarquee cloud put` for that PDF; it also failed with HTTP 400.
- Updated tasks and changelog to record the skipped upload.

Why

The design deliverable should not be blocked by a known cloud upload mutation issue when local docs and the generated PDF are already available.

What worked

- `docmgr doctor` passes.
- `remarquee status` and account lookup work.
- `remarquee upload bundle --dry-run` works.
- Local PDF generation works.

What didn't work

Actual cloud upload failed:

```
Error: failed to create remote directory "CSSVD-FLEX-JS-API" under "24": request failed with
```

Then direct upload to the existing date folder failed:

```
Error: failed to upload file [/tmp/remarquee-upload-bundle-.../CSSVD-FLEX-JS-API - Flexible
```

Finally, direct `cloud put` of the locally generated PDF failed:

```
Error: failed to upload file [.../CSSVD-FLEX-JS-API-flexible-javascript-api-design-guide.pdf
```

What I learned

The reMarkable issue is not specific to bundle conversion or PDF generation. The account can read/list, but cloud mutation/upload operations are failing with HTTP 400.

What was tricky to build

The generated markdown bundle initially failed direct pandoc conversion because concatenated documents included multiple YAML frontmatter blocks. I generated the local PDF by stripping frontmatter before invoking pandoc.

What warrants a second pair of eyes

The reMarkable upload tooling or account/cloud API state needs separate investigation if upload becomes important again.

What should be done in the future

Retry upload later or transfer the local PDF manually:

`ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/pdf/CSSVD-FI`

Code review instructions

No code changed. Review the ticket docs and generated PDF.

Technical details

Validation command:

```
docmgr doctor --root ./ttmp --ticket CSSVD-FLEX-JS-API --stale-after 30
```

Successful result:

```
## Doctor Report (1 findings)

### CSSVD-FLEX-JS-API

-  All checks passed
```

Step 4: Retry reMarkable upload successfully with a short PDF filename

The user asked to try the upload again. I retried using the already-generated local PDF but copied it to a short temporary filename first. This succeeded, which suggests the earlier HTTP 400 may have been related to remote folder creation, long names, spaces, or metadata generated by the bundle upload path rather than the PDF content itself.

The document is now available in the reMarkable cloud date folder as `cssvd-flex-api`.

Prompt Context

User prompt (verbatim): "try upload again"

Assistant interpretation: Retry the reMarkable upload now instead of leaving it skipped.

Inferred user intent: See whether the previous cloud upload failure was transient or avoidable with a different upload path.

What I did

- Checked `remarquee status`.
- Checked the cloud account with `remarquee cloud account --non-interactive`.
- Listed `/ai/2026/04/24`.

- Copied the local generated PDF to `/tmp/cssvd-flex-api.pdf`.
- Uploaded the short-named PDF with `remarquee cloud put /tmp/cssvd-flex-api.pdf /ai/2026/04/24 --non-interactive`.
- Verified the remote listing.
- Updated `tasks.md` and `changelog.md`.

Why

The previous upload attempts failed with HTTP 400 using folder creation and longer bundle/document names. Retrying with a short filename was a low-risk way to test whether upload itself could succeed.

What worked

Upload succeeded:

```
OK: uploaded /tmp/cssvd-flex-api.pdf -> /ai/2026/04/24
```

Remote listing verified:

```
[f] 02-postmortem-and-next-developer-handoff
[f] cssvd-flex-api
[d] CSSVD-GOJA-JS-API
```

What didn't work

No new failure occurred in this step. Earlier failures remain recorded in Step 3.

What I learned

A short filename and direct `cloud put` to an existing date folder can avoid the previous upload failure path. The failed operation may have involved remote directory creation or document naming rather than basic account/auth/upload capability.

What was tricky to build

The upload target is not the originally requested ticket subfolder because creating `/ai/2026/04/24/CSSVD-FLEX-JS-API` previously failed. The successful upload is in the existing date folder:

```
/ai/2026/04/24/cssvd-flex-api
```

What warrants a second pair of eyes

If future organization matters, investigate why creating a new reMarkable folder under `/ai/2026/04/24` returned HTTP 400 while direct file upload to the existing date folder succeeded.

What should be done in the future

Optionally rename/move the document on-device or retry folder creation later. For now, the PDF is uploaded and verified.

Code review instructions

No code changed. Review the ticket docs and uploaded PDF.

Technical details

Commands:

```
remarquee status
remarquee cloud account --non-interactive
remarquee cloud ls /ai/2026/04/24 --long --non-interactive
cp ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/pdf/CSSVI
remarquee cloud put /tmp/cssvd-flex-api.pdf /ai/2026/04/24 --non-interactive
remarquee cloud ls /ai/2026/04/24 --long --non-interactive
```

Step 5: Update design for jsapi subpackage, Go-backed Proxy objects, and no workflow builder

After reviewing the design direction, the user clarified two important architectural preferences. First, the native JS API should use Go-backed structs wrapped in Goja Proxy objects rather than plain JavaScript objects, following the Discord bot UI DSL pattern. Second, there is no need for a workflow builder; ordinary JavaScript functions and loops should remain the orchestration mechanism.

I updated the design guide to reflect those decisions. The most important changes are that `internal/cssvisualdiff/dsl/cvd_module.go` should not simply grow new files in the same package; instead, the native `require("css-visual-diff")` API should move to a dedicated `internal/cssvisualdiff/jsapi` package. The `dsl` package should stay focused on jsverbs host/runtime/scanning responsibilities.

Prompt Context

User prompt (verbatim): "Update the design implementation guide with all these things (refactoring cvd_module.go, etc…)"

Follow-up prompt (verbatim): "no need for workflow builder."

Assistant interpretation: Update the existing ticket design guide with the subpackage/refactor plan, Goja Proxy-backed structs/builders, method-specific validation/error behavior, and remove the workflow-builder phase.

Inferred user intent: The user wants the design guide to reflect the refined implementation direction before anyone starts coding.

What I did

- Updated the primary design guide frontmatter and summary.
- Replaced the "plain objects plus optional fluent builders" principle with a strict Go-backed Proxy builder/handle principle.
- Added a dedicated `internal/cssvisualdiff/jsapi` package layout.
- Added Proxy infrastructure guidance: `proxy.go`, `unwrap.go`, method-owner maps, wrong-parent errors, unknown-method errors, and type-mismatch errors.
- Rewrote the Goja adapter section around Proxy-backed `BrowserHandle`, `PageHandle`, `LocatorHandle`, `TargetBuilder`, `ProbeBuilder`, `ExtractorHandle`, and `CatalogHandle`.
- Rewrote the implementation phases so Phase 1 moves the native API out of `dsl/cvd_module.go`, Phase 2 adds Proxy infrastructure, and there is no workflow-builder phase.
- Updated the review checklist, acceptance criteria, file-by-file implementation guide, references, tasks, and changelog.

Why

The Proxy-backed approach gives better control over validation and much better feedback for LLM-written JavaScript. It can distinguish a typo, an unknown method, a method that belongs to a different object type, and a raw-object type mismatch. This is exactly what the Discord bot UI DSL demonstrated.

What worked

- The design now has a clearer package boundary:
  - `dsl` = jsverbs host/runtime/scanning.
  - `jsapi` = native `require("css-visual-diff")` module, Goja Proxy wrappers, Promise bridge, typed errors, and adapters.
  - `service` = Go implementation with no Goja dependency.
- The workflow-builder references were removed from the implementation plan and replaced with explicit JavaScript control flow.

What didn't work

No command failed during this documentation update.

What I learned

The lower-level API should not optimize for arbitrary JavaScript flexibility. It should optimize for controlled fluent authoring with precise host-side errors. Raw object compatibility can remain in the existing high-level API, but new strict APIs should prefer typed Proxy-backed values.

What was tricky to build

The tricky part was preserving compatibility for current scripts while making the new lower-level API stricter. The design now handles this by separating compatibility methods (`page.preflight`, `page.inspectAll`) from strict new methods (`cvd.extract`, `cvd.snapshot`).

What warrants a second pair of eyes

- The Proxy registry/unwrapping design should be reviewed carefully before implementation.
- The exact package migration path from `dsl/catalog_adapter.go` and `dsl/config_adapter.go` into `jsapi` should be done as a no-behavior refactor first.
- The method-owner map should allow methods that intentionally exist on multiple object types, such as `close` or `attributes`.

What should be done in the future

- Regenerate and upload the updated PDF to reMarkable.
- When coding begins, implement Phase 1 as a mechanical refactor with no behavior changes.
- Add Proxy behavior tests before implementing many user-facing methods.

Code review instructions

Review the updated sections in the design guide:

1. Section 5: Design principles.
2. Section 6: Proposed API layers.
3. Section 9: Architecture for implementation.
4. Section 10: Goja adapter design with Proxy-backed handles.
5. Section 12: Implementation phases.
6. Section 19: File-by-file implementation guide.

Technical details

Key design files updated:

```
ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/design-doc/0
ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/tasks.md
ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/changelog.mo
```

Step 6: Regenerate and upload the updated design guide to reMarkable

After updating the design guide for the `jsapi` subpackage, Go-backed Proxy object model, and removal of the workflow builder phase, I regenerated the

PDF bundle and uploaded the updated version to reMarkable.

I used a short remote filename again because the earlier successful upload showed that short names avoid the HTTP 400 failure path. To avoid overwriting or deleting the previous upload, I uploaded the new PDF as `cssvd-flex-api-updated`.

Prompt Context

User prompt (verbatim): "upload document to remarkable when done"

Assistant interpretation: After finishing the design guide updates, regenerate the document bundle and upload the updated version to reMarkable.

Inferred user intent: Make the latest design, not the older pre-Proxy version, available on the reMarkable device/cloud.

What I did

- Regenerated a bundled PDF from the updated design guide, diary, tasks, and changelog.
- Stripped YAML frontmatter while building the bundle so pandoc would not treat multiple frontmatter blocks as invalid metadata.
- Saved the updated local PDF at:
  - `pdf/CSSVD-FLEX-JS-API-flexible-javascript-api-design-guide-updated.pdf`
- Copied it to a short temporary upload filename:
  - `/tmp/cssvd-flex-api-updated.pdf`
- Uploaded it with `remarquee cloud put`.
- Verified the remote listing.
- Updated tasks and changelog.

Why

The reMarkable copy needed to reflect the latest design decisions: `internal/cssvisualdiff/jsapi`, Proxy-backed handles/builders, strict lower-level APIs, typed unwrapping, and no workflow builder.

What worked

Upload succeeded:

```
OK: uploaded /tmp/cssvd-flex-api-updated.pdf -> /ai/2026/04/24
```

Remote listing verified:

```
[f] 02-postmortem-and-next-developer-handoff
[f] cssvd-flex-api
[f] cssvd-flex-api-updated
[d] CSSVD-GOJA-JS-API
```

```
[d] GCB-009
[d] PYXIS-COMPONENT-VISUAL-PARITY
```

What didn't work

Pandoc emitted a non-fatal font warning:

```
[WARNING] Missing character: There is no  (U+2705) (U+2705) in font [lmmono10-regular]:!
```

The PDF was still generated and uploaded successfully.

What I learned

The reliable upload path is still direct `remarquee cloud put` with a short PDF filename into an existing folder.

What was tricky to build

The generated ticket docs include YAML frontmatter and Unicode symbols. The frontmatter must be stripped for a concatenated bundle. The Unicode warning is cosmetic but may be worth avoiding in future PDF-bound docs.

What warrants a second pair of eyes

If annotations on uploaded docs become important, avoid overwriting existing documents with `--force`. This upload intentionally used a new filename instead.

What should be done in the future

Use `/ai/2026/04/24/cssvd-flex-api-updated` as the current reMarkable copy of the design.

Code review instructions

No code changed. Review the updated design guide and PDF bundle.

Technical details

Commands included:

```
pandoc "$TMP_MD" \
  -o "$TICKET_DIR/pdf/CSSVD-FLEX-JS-API-flexible-javascript-api-design-guide-updated.pdf" \
  --toc --toc-depth=2 --pdf-engine=xelatex
cp "$TICKET_DIR/pdf/CSSVD-FLEX-JS-API-flexible-javascript-api-design-guide-updated.pdf" /tmp
remarquee cloud put /tmp/cssvd-flex-api-updated.pdf /ai/2026/04/24 --non-interactive
remarquee cloud ls /ai/2026/04/24 --long --non-interactive
```

Step 7: Expand ticket tasks into granular implementation phases

Before continuing code work, I paused to make the ticket task list much more granular. The previous task list only had broad implementation phases, which was not detailed enough for task-by-task execution or for deciding when to commit.

The ticket now has a phase-by-phase checklist from the no-behavior `jsapi` refactor through Proxy infrastructure, page serialization, DOM services, locators, builders, extraction, snapshots, diffs, docs, smoke scripts, and final delivery. This gives us a concrete progress tracker for the rest of CSSVD-FLEX-JS-API.

Prompt Context

User prompt (verbatim): "Let's work on CSSVD-FLEX-JS-API, task by task, keep a detailed diary (read the current diary so far), commit at appropriate intervals."

Follow-up prompt (verbatim): "add detailed tasks split in phases to the ticket, so that we have a more granular wya of keeping track where we are."

Assistant interpretation: Read the existing diary, then make the ticket task list detailed enough to track implementation phase by phase before continuing code.

Inferred user intent: The user wants a disciplined implementation process with explicit checkpoints, diary updates, validation, and commits.

What I did

- Read the diary skill, docmgr skill, git commit instructions, and go-go-goja module authoring skill.
- Read the current CSSVD-FLEX-JS-API diary and tasks file.
- Checked branch and working tree state.
- Committed the previously untracked ticket/design documents before implementation work:
  - `17240de docs: add flexible js api implementation ticket`
- Rewrote `tasks.md` into detailed phased checklists:
  - Phase 0: ticket/design/baseline bookkeeping.
  - Phase 1: no-behavior `internal/cssvisualdiff/jsapi` package refactor.
  - Phase 2: Proxy infrastructure and typed unwrapping.
  - Phase 3: per-page operation serialization.
  - Phase 4: service DOM locator primitives.
  - Phase 5: `page.locator()` Proxy handle and locator methods.
  - Phase 6: target/probe/extractor builders.
  - Phase 7: strict `cvd.extract(locator, extractors)`.

  - Phase 8: strict `cvd.snapshot(page, probes, options)`.
  - Phase 9: diff/report/write primitives.
  - Phase 10: docs/examples/smoke scripts/delivery.

Why

The implementation will touch multiple subsystems. A granular task list prevents losing track of intermediate safety steps such as baseline tests, no-behavior refactor validation, Proxy error tests, per-page serialization tests, and documentation updates.

What worked

- The existing diary had enough context to resume safely.
- The ticket docs were committed cleanly before implementation changes.
- The task list now separates mechanical refactoring from behavior changes.

What didn't work

No command failed during this planning step.

What I learned

Phase 1 needs to be treated as a no-behavior refactor and validated before adding Proxy behavior. The initial code move also makes it clear that `decodeInto(...)` currently lives in `dsl/codec.go`, so Phase 1 must either move or duplicate the codec helper into `jsapi` while keeping `toPlainValue(...)` available to the existing `diff` and `report` modules.

What was tricky to build

The tricky part is that I had already started the mechanical `git mv` for Phase 1 before the follow-up request arrived. I left those implementation file moves uncommitted and used this step to improve the ticket tracking first. This keeps the task-detail commit separate from the still-in-progress code refactor.

What warrants a second pair of eyes

- The Phase 1 checklist should be reviewed to ensure it really is behavior-preserving.
- The Phase 2 Proxy checklist should be reviewed before implementation because it will define the long-term ergonomics and error model.

What should be done in the future

Resume Phase 1 from the current working tree, finish the `jsapi` package move, run targeted tests, update the diary/changelog, and commit the no-behavior refactor separately.

Code review instructions

Review:

`ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/tasks.md`

Validation:

```
docmgr doctor --root ./ttmp --ticket CSSVD-FLEX-JS-API --stale-after 30
```

Technical details

Commands included:

```
git status --short --branch
git add ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api
git commit -m "docs: add flexible js api implementation ticket"
git rev-parse HEAD
```

Step 8: Complete Phase 1 and Phase 2 implementation

I completed the first two implementation phases in one focused pass. Phase 1 was the no-behavior refactor that moves the native `require("css-visual-diff")` API out of the broad `dsl` package and into a dedicated `internal/cssvisualdiff/jsapi` package. Phase 2 added the first reusable Goja Proxy infrastructure and typed unwrapping helpers for future lower-level handles/builders.

The current public API behavior should be unchanged. Existing `css-visual-diff` JS scripts still load the same module name, and the repository-scanned verb tests still exercise the same browser/page/catalog behavior. The new Proxy helpers are currently infrastructure-only; they are covered by unit tests but are not yet wired into the existing browser/page/catalog wrappers.

Prompt Context

User prompt (verbatim): "continue, do phase 1 and phase 2 in their entirety."

Assistant interpretation: Finish both the no-behavior `jsapi` package refactor and the initial Proxy/unwrap infrastructure, validate them, update ticket bookkeeping, and prepare for a commit.

Inferred user intent: The user wants the implementation to progress through the granular ticket phases, not just update docs.

What I did

- Moved native JS API files into `internal/cssvisualdiff/jsapi`:
  - `dsl/cvd_module.go` -> `jsapi/module.go`
  - `dsl/catalog_adapter.go` -> `jsapi/catalog.go`
  - `dsl/config_adapter.go` -> `jsapi/config.go`
- Changed the moved files to `package jsapi`.

- Renamed `registerCVDModule(...)` to exported `Register(...)`.
- Added `internal/cssvisualdiff/jsapi/codec.go` with a local `decodeInto(...)` helper.
- Kept `dsl/codec.go` in place so existing `diff` and `report` native modules can still use `decodeInto(...)` and `toPlainValue(...)`.
- Updated `internal/cssvisualdiff/dsl/registrar.go` to import `jsapi` and call `jsapi.Register(ctx, reg)`.
- Added `internal/cssvisualdiff/jsapi/proxy.go` with:
  - `ProxyRegistry`,
  - `ProxySpec`,
  - `MethodSpec`,
  - `ProxyMethod`,
  - `newProxyValue(...)`,
  - unknown-method errors,
  - wrong-parent errors,
  - type-mismatch errors,
  - simple did-you-mean suggestions.
- Added `internal/cssvisualdiff/jsapi/unwrap.go` with:
  - `unwrapProxyBacking[T](...)`,
  - `unwrapProxyBinding(...)`,
  - `mustUnwrapProxyBacking[T](...)`.
- Added `internal/cssvisualdiff/jsapi/proxy_test.go` covering:
  - unknown method errors,
  - wrong-parent method errors,
  - successful backing unwrap,
  - raw object rejection,
  - wrong-owner rejection.
- Marked Phase 1 and Phase 2 tasks complete and set the next active phase to Phase 3.

Why

The native API had grown into a large `dsl/cvd_module.go` file even though `dsl` should own jsverbs host/scanning/runtime wiring. Moving the native `require("css-visual-diff")` adapter code into `jsapi` creates a cleaner boundary before adding lower-level handles, locators, builders, snapshots, and diffs.

The Proxy infrastructure is needed before adding strict lower-level APIs because those APIs should be able to distinguish Go-backed handles/builders from raw JS objects and provide LLM-friendly feedback when users call the wrong method on the wrong object.

What worked

The no-behavior refactor compiled cleanly after adding the local `jsapi` codec helper and rewiring the registrar. Existing targeted tests and the full test suite

passed.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/jsapi -count=1

go test ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff

go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdif

go test ./... -count=1
```

What didn't work

The first targeted test run failed as expected after moving files but before adding the codec/helper rewiring:

```
# github.com/go-go-golems/css-visual-diff/internal/cssvisualdiff/jsapi
internal/cssvisualdiff/jsapi/catalog.go:21:16: undefined: decodeInto
internal/cssvisualdiff/jsapi/catalog.go:108:16: undefined: decodeInto
internal/cssvisualdiff/jsapi/catalog.go:135:17: undefined: decodeInto
internal/cssvisualdiff/jsapi/catalog.go:190:16: undefined: decodeInto
internal/cssvisualdiff/jsapi/module.go:152:17: undefined: decodeInto
internal/cssvisualdiff/jsapi/module.go:196:17: undefined: decodeInto
internal/cssvisualdiff/jsapi/module.go:328:15: undefined: decodeInto
internal/cssvisualdiff/jsapi/module.go:358:16: undefined: decodeInto
internal/cssvisualdiff/jsapi/module.go:402:17: undefined: decodeInto
internal/cssvisualdiff/jsapi/module.go:414:17: undefined: decodeInto
internal/cssvisualdiff/jsapi/module.go:414:17: too many errors
FAIL    github.com/go-go-golems/css-visual-diff/internal/cssvisualdiff/jsapi [build failed]
# github.com/go-go-golems/css-visual-diff/internal/cssvisualdiff/dsl [github.com/go-go-golem
internal/cssvisualdiff/dsl/registrar.go:51:2: undefined: registerCVDModule
FAIL    github.com/go-go-golems/css-visual-diff/internal/cssvisualdiff/dsl [build failed]
```

The fix was to add `internal/cssvisualdiff/jsapi/codec.go` and replace the old registrar call with `jsapi.Register(ctx, reg)`.

What I learned

The file move is clean if `dsl/codec.go` remains in `dsl` for the existing `diff` and `report` native modules while `jsapi/codec.go` gets its own `decodeInto(...)`. This intentionally duplicates a tiny JSON codec helper for now rather than creating another shared internal package before it is necessary.

Goja's `Runtime.NewProxy(...)` API is sufficient for the future builder/handle design. A hidden internal proxy id exposed through the `Get` trap gives strict APIs a reliable way to unwrap only values created by the `jsapi` package.

What was tricky to build

The trickiest part was designing Proxy infrastructure that is useful now without prematurely converting browser/page/catalog wrappers. I kept Phase 2 infrastructure-only and added tests using a small fake `cvd.probe` Proxy. That proves the mechanics and error style without changing existing public API behavior.

Another subtle point is that `mustUnwrapProxyBacking(...)` currently panics with a Goja `TypeError` via `typeMismatchError(...)`. Future integration may want to route these through the existing CSSVD typed error class hierarchy, but for Phase 2 the important behavior is that wrong raw objects fail immediately with an actionable operation-specific message.

What warrants a second pair of eyes

- Whether the proxy id registry should eventually delete bindings when handles close, or whether runtime-lifetime bindings are acceptable.
- Whether the hidden `__cssVisualDiffProxyID` property should become a symbol-like internal key later.
- Whether `typeMismatchError(...)` should construct `CvdError` subclasses instead of `TypeError` once strict APIs are wired into the exported module.
- Whether the simple Levenshtein did-you-mean threshold is too aggressive or too conservative.

What should be done in the future

Phase 3 should add page operation serialization. After that, Phase 4 can add pure Go DOM locator services and Phase 5 can expose `page.locator()` using the new Proxy infrastructure.

Code review instructions

Start review with the package boundary:

```
internal/cssvisualdiff/dsl/registrar.go
internal/cssvisualdiff/jsapi/module.go
internal/cssvisualdiff/jsapi/catalog.go
internal/cssvisualdiff/jsapi/config.go
```

Then review the new Proxy infrastructure:

```
internal/cssvisualdiff/jsapi/proxy.go
internal/cssvisualdiff/jsapi/unwrap.go
internal/cssvisualdiff/jsapi/proxy_test.go
```

Validation commands:

```
go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff
go test ./... -count=1
```

Technical details

Important implementation details:

- `jsapi.Register(ctx, reg)` preserves the public module name `require("css-visual-diff")`.
- `dsl` still registers the legacy `diff` and `report` native modules.
- `ProxyRegistry` stores Go backing values keyed by an internal int64 id.
- `newProxyValue(...)` returns a Goja Proxy whose `Get` trap:
  - returns own methods,
  - returns a stable `toString`,
  - returns the hidden proxy id,
  - throws wrong-parent errors for known methods owned by another object,
  - throws unknown-method errors with available methods and did-you-mean suggestions otherwise.

Step 9: Complete Phase 3 page operation serialization

I completed Phase 3 by adding a per-page mutex to the JavaScript API page state and routing every existing page operation through that guard. This keeps concurrent JavaScript calls such as `Promise.all([page.preflight(...), page.preflight(...)])` safe on a single page while still allowing independent pages to proceed through their own page-level locks.

This phase does not add new public API methods. It hardens the current Promise-first browser/page API so future locator and extraction methods can safely share the same page handle without racing CDP operations or mutating page target state concurrently.

Prompt Context

User prompt (verbatim): "phase 3"

Assistant interpretation: Implement Phase 3 from the ticket: add per-page operation serialization, validate it, update docs/diary/tasks, and commit.

Inferred user intent: Prepare the current page API for lower-level locator/extractor methods by making same-page concurrent calls safe.

What I did

- Added a `sync.Mutex` to `pageState` in `internal/cssvisualdiff/jsapi/module.go`.
- Added `pageState.runExclusive(...)`.
- Wrapped all current page operations in `runExclusive(...)`:
  - `page.goto(...)`

  - `page.prepare(...)`
  - `page.preflight(...)`
  - `page.inspect(...)`
  - `page.inspectAll(...)`
  - `page.close()`
- Added repository-scanned JS verb tests in `internal/cssvisualdiff/verbcli/command_test.go`:
  - `TestCVDModuleSerializesSamePagePromiseAll`
  - `TestCVDModuleAllowsConcurrentOperationsOnSeparatePages`
- Marked Phase 3 tasks complete and set the active phase to Phase 4.

Why

Chromedp/CDP operations on a single page are not a good fit for unguarded concurrent use. The existing API returns Promises, so JavaScript users can naturally write `Promise.all(...)`. Rather than forbid that pattern, the page handle should serialize operations internally and preserve safety.

This is especially important before adding `page.locator()` and `locator.*` methods because users are likely to query several locators at once.

What worked

The per-page mutex was a small focused change. Same-page concurrent calls now complete safely in a repository-scanned JS verb test. Separate page handles also complete concurrently from the JavaScript point of view because each page has its own `pageState` and lock.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModule(SerializesSamePagePromiseAll|Al

go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff

go test ./... -count=1
```

What didn't work

My first version of the same-page concurrency test used:

```
page.inspect({ name: "a", selector: "#a" }, { artifacts: "json" })
```

That failed because `"json"` is not a supported inspect artifact format:

```
promise rejected: ArtifactError: unsupported inspect format "json"
```

I changed the test to use three concurrent `page.preflight(...)` calls instead. That still exercises concurrent same-page CDP script evaluation without introducing artifact-format concerns.

What I learned

The current `inspect` API expects the existing inspect format names such as `bundle`, `css-json`, or `html`; raw `json` is not a valid shortcut. For Phase 3, preflight is the cleaner concurrency smoke because it avoids filesystem artifact output and focuses on page-operation safety.

What was tricky to build

The key subtlety is the target state on `pageState`. `goto` mutates `state.target`, and `prepare` mutates `state.target.Prepare`, while `inspect` and `inspectAll` read `state.target`. The lock needs to cover both CDP calls and target state access; otherwise a future concurrent `goto` plus `inspect` could read a half-updated target.

The lock is intentionally per page rather than per browser. That keeps operations on one page serialized while preserving concurrency between independent pages.

What warrants a second pair of eyes

- Whether `page.close()` should set a closed flag and reject later operations with a clearer error.
- Whether long-running operations should support cancellation or timeouts at this layer.
- Whether Phase 5 locator methods should reuse this same `runExclusive(...)` helper or introduce batch operations to reduce queuing overhead.

What should be done in the future

Phase 4 should add pure Go DOM locator primitives in `internal/cssvisualdiff/service/dom.go`. Phase 5 can then expose those primitives through `page.locator()` and the Proxy infrastructure.

Code review instructions

Start with:

`internal/cssvisualdiff/jsapi/module.go`

Review `pageState`, `runExclusive(...)`, and the wrappers in `wrapPage(...)`.

Then review tests:

`internal/cssvisualdiff/verbcli/command_test.go`

Validation commands:

```
go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModule(SerializesSamePagePromiseAll|Al
go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff
go test ./... -count=1
```

Technical details

`pageState` now owns the synchronization primitive:

```go
type pageState struct {
    mu     sync.Mutex
    page   *service.PageService
    target config.Target
}

func (s *pageState) runExclusive(work func() (any, error)) (any, error) {
    s.mu.Lock()
    defer s.mu.Unlock()
    return work()
}
```

Each Promise work closure calls `state.runExclusive(...)`, so the work still runs asynchronously from JavaScript's perspective but only one operation touches the underlying page at a time.

Step 10: Complete Phase 4 service DOM locator primitives

I completed Phase 4 by adding pure Go service-layer DOM locator primitives. These functions are intentionally independent of Goja and provide the reusable browser-page operations that the future `page.locator()` JavaScript API will wrap.

The new service file covers selector status, text, HTML, bounds, attributes, and computed style. Missing selectors return empty/structured results where that is natural, while invalid selectors return errors for direct locator operations. `LocatorStatus` reuses the existing preflight implementation so status behavior stays aligned with the current high-level API.

Prompt Context

User prompt (verbatim): "phase 4"

Assistant interpretation: Implement Phase 4 from the ticket: add pure Go DOM locator primitives, test them, update docs/diary/tasks, and commit.

Inferred user intent: Build the Go service foundation required before exposing `page.locator()` in the JS API.

What I did

- Added `internal/cssvisualdiff/service/dom.go`.
- Defined:
  - `LocatorSpec`
  - `TextOptions`

  - `ElementHTML`
- Implemented:
  - `LocatorStatus(...)`
  - `LocatorText(...)`
  - `LocatorHTML(...)`
  - `LocatorBounds(...)`
  - `LocatorAttributes(...)`
  - `LocatorComputedStyle(...)`
- Added `internal/cssvisualdiff/service/dom_test.go`.
- Added tests for:
  - existing visible selector,
  - missing selector,
  - hidden selector,
  - invalid selector,
  - text normalization,
  - inner/outer HTML,
  - bounds,
  - attributes including missing attributes,
  - computed styles.
- Marked Phase 4 tasks complete and set the active phase to Phase 5.

Why

The lower-level JavaScript API should wrap reusable Go services rather than putting DOM logic directly in Goja adapters. This keeps service behavior testable without JavaScript runtime concerns and gives future CLI modes a shared implementation path.

What worked

`LocatorStatus(...)` could reuse `PreflightProbes(...)`, preserving existing selector status behavior. `LocatorComputedStyle(...)` could reuse `EvaluateStyle(...)`, preserving current computed-style extraction semantics.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/service -run 'TestLocatorDOMPrimitives' -count=1

go test ./internal/cssvisualdiff/service -count=1

go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff

go test ./... -count=1
```

What didn't work

The first `TestLocatorDOMPrimitives` run failed because the test HTML contained a literal backslash-n sequence instead of an actual newline:

```
Error: "Book\\n now" does not contain "Book now"
```

I fixed the fixture to use a real newline inside the button text so the existing whitespace normalization behavior matched the assertion.

What I learned

The current preflight text normalization collapses real whitespace, but not a literal `\\n` sequence. That is correct browser behavior, and the test fixture was wrong.

For missing selectors, empty direct results are more useful than errors:

- text -> `""`
- HTML -> `{ exists: false, html: "" }`
- bounds -> `nil`
- attributes -> empty map
- computed style -> empty map

For invalid selectors, direct operations should return errors because the selector itself is malformed.

What was tricky to build

The tricky part was deciding how much behavior to duplicate versus reuse. `LocatorStatus(...)` reuses `PreflightProbes(...)` and `LocatorComputedStyle(...)` reuses `EvaluateStyle(...)`. The direct text/html/bounds/attributes functions use small focused page scripts because the existing service layer did not expose those exact data shapes yet.

Another subtle point is attribute handling. DOM `getAttribute(...)` can return `null`, but the Go API returns `map[string]string`, so the script normalizes missing attributes to an empty string to avoid JSON unmarshalling surprises.

What warrants a second pair of eyes

- Whether `LocatorText` defaults should eventually normalize/trim by default in the JS API, even though the service options are explicit.
- Whether `LocatorComputedStyle` should avoid the small `configStyleSpec(...)` adapter helper and instead have `EvaluateStyle` accept a service-native spec later.
- Whether missing selectors should be configurable as strict errors for some future extraction modes.

What should be done in the future

Phase 5 should expose these service functions through a Go-backed `LocatorHandle` Proxy returned by `page.locator(selector)`.

Code review instructions

Start with:

`internal/cssvisualdiff/service/dom.go`

Then review tests:

`internal/cssvisualdiff/service/dom_test.go`

Validation commands:

```
go test ./internal/cssvisualdiff/service -run 'TestLocatorDOMPrimitives' -count=1
go test ./internal/cssvisualdiff/service -count=1
go test ./... -count=1
```

Technical details

The service functions are intentionally Goja-free and operate on `*driver.Page`. The future JS adapter should only decode inputs, call these functions through the per-page serialization guard, and lower results into JavaScript values.

Step 11: Complete Phase 5 `page.locator()` JavaScript API

I completed Phase 5 by exposing the new service DOM primitives through a Go-backed Proxy locator handle. `page.locator(selector)` is synchronous and returns a page-bound handle; methods that touch Chromium return Promises and run through the existing per-page serialization guard from Phase 3.

This is the first visible lower-level JS API milestone. It lets scripts query element status, existence, visibility, text, bounds, computed styles, and attributes without writing inspect artifacts or using YAML-style probe objects.

Prompt Context

User prompt (verbatim): "phase 5"

Assistant interpretation: Implement Phase 5 from the ticket: expose `page.locator(selector)` and async locator methods as Go-backed Proxy handles, validate behavior, update docs/diary/tasks, and commit.

Inferred user intent: Make the first lower-level JS API available on top of the Phase 4 DOM service primitives.

What I did

- Added `internal/cssvisualdiff/jsapi/locator.go`.
- Added `locatorHandle` with references to the owning `pageState` and selector.
- Added `wrapLocator(...)` using the Phase 2 `newProxyValue(...)` helper.

- Added synchronous `page.locator(selector)` in `wrapPage(...)`.
- Implemented Promise-returning locator methods:
  - `locator.status()`
  - `locator.exists()`
  - `locator.visible()`
  - `locator.text(options)`
  - `locator.bounds()`
  - `locator.computedStyle(props)`
  - `locator.attributes(names)`
- Routed all locator browser work through `pageState.runExclusive(...)`.
- Added wrong-parent Proxy guidance for common probe methods accidentally called on locators:
  - `.selector(...)`
  - `.styles(...)`
  - `.required(...)`
  - `.build()`
- Added repository-scanned JS verb tests:
  - `TestCVDModuleExposesLocatorMethods`
  - `TestCVDModuleLocatorWrongParentError`
- Marked Phase 5 tasks complete and set the active phase to Phase 6.

Why

The lower-level API should let JavaScript inspect the current page directly without forcing users through high-level artifact-writing `inspect` calls. A Go-backed Proxy handle gives us controlled method access, useful wrong-parent errors, and future typed unwrapping for strict APIs such as `cvd.extract(locator, extractors)`.

What worked

The Phase 4 service functions mapped cleanly into locator methods. The Phase 3 page operation lock also made it straightforward to safely run locator methods under `Promise.all(...)`.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModule(ExposesLocatorMethods|LocatorWr

go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff

go test ./... -count=1
```

What didn't work

No test failure occurred during Phase 5 after implementation. The main thing to watch is that locator Proxy errors are currently `TypeError`s, not `CvdError` subclasses.

What I learned

The Proxy infrastructure is now doing useful public API work. The wrong-parent error for `page.locator("#cta").styles(["color"])` produces an actionable message that points users to `.computedStyle(...)` on locators and explains that `.styles(...)` belongs to future probe builders.

What was tricky to build

The tricky part was argument handling for methods like `computedStyle(props)` and `attributes(names)`. These methods should reject non-array arguments immediately and clearly because future LLM-generated code may pass strings or object literals. I added a small `stringListArg(...)` helper that validates array inputs before starting async browser work.

Another subtle point is that `page.locator(selector)` itself is synchronous and does not validate selector syntax immediately. Syntax errors surface when a method queries the DOM. This matches the intended handle semantics: constructing a locator is cheap and page-bound, while evaluating it is async.

What warrants a second pair of eyes

- Whether `locator.text()` should default to normalized/trimmed text in the JS API. It currently follows explicit `TextOptions`, so callers use `{ normalizeWhitespace: true, trim: true }`.
- Whether locator handles should include a `.toString()` or `.selector` read-only property in the public API later.
- Whether Proxy registries should become module-scoped so future strict APIs can unwrap locator handles created by `page.locator(...)`. The current locator uses `newProxyValue` with a local registry because no strict cross-call unwrapping exists yet.

What should be done in the future

Phase 6 should add Go-backed target, probe, viewport, and extractor builders. Once extractor handles exist, Phase 7 can implement strict `cvd.extract(locator, extractors)` and should revisit the locator Proxy registry/unwrapping story.

Code review instructions

Start with:

```
internal/cssvisualdiff/jsapi/locator.go
internal/cssvisualdiff/jsapi/module.go
```

Then review tests:

```
internal/cssvisualdiff/verbcli/command_test.go
```

Validation commands:

```
go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModule(ExposesLocatorMethods|LocatorWr
go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff
go test ./... -count=1
```

Technical details

Example supported JS:

```
const cta = page.locator("#cta")
const [status, text, styles, attrs] = await Promise.all([
  cta.status(),
  cta.text({ normalizeWhitespace: true, trim: true }),
  cta.computedStyle(["color", "display"]),
  cta.attributes(["id", "class"]),
])
```

All methods that touch Chromium run through:

```
l.page.runExclusive(func() (any, error) { ... })
```

Step 12: Complete Phase 6 Go-backed target/probe/extractor builders

I completed Phase 6 by adding the first Go-backed fluent builders for targets, probes, viewports, and extractors. These builders are synchronous authoring objects backed by Go structs and wrapped in Goja Proxies, matching the design decision that lower-level DSL objects should provide controlled methods and actionable errors rather than behaving like arbitrary raw JavaScript objects.

This phase still does not add strict extraction or snapshot execution. It creates the values that Phase 7 and Phase 8 will consume. `.build()` returns plain serializable values for debugging, tests, and YAML/config interop, while the Proxy-backed handles remain the authoring-time API.

Prompt Context

User prompt (verbatim): "phase 6"

Assistant interpretation: Implement Phase 6 from the ticket: add Go-backed target/probe/extractor builders, tests, validation, diary/tasks/changelog updates, and commit.

Inferred user intent: Add the JS-native builder layer that will eventually replace many YAML object literals and feed strict extract/snapshot APIs.

What I did

- Added `internal/cssvisualdiff/jsapi/builder_helpers.go` with shared validation helpers.
- Added `internal/cssvisualdiff/jsapi/target.go`:
  - `cvd.target(name)`
  - `.url(...)`
  - `.waitMs(...)`
  - `.viewport(width, height)` / `.viewport({ width, height })`
  - `.root(...)`
  - `.prepare(...)`
  - `.build()`
  - `cvd.viewport(width, height)` plus `desktop`, `tablet`, and `mobile` helpers.
- Added `internal/cssvisualdiff/jsapi/probe.go`:
  - `cvd.probe(name)`
  - `.selector(...)`
  - `.required(...)`
  - `.source(...)`
  - `.text()`
  - `.bounds()`
  - `.styles(...)`
  - `.attributes(...)`
  - `.build()`
- Added `internal/cssvisualdiff/jsapi/extractor.go`:
  - `cvd.extractors.exists()`
  - `cvd.extractors.visible()`
  - `cvd.extractors.text()`
  - `cvd.extractors.bounds()`
  - `cvd.extractors.computedStyle(props)`
  - `cvd.extractors.attributes(names)`
- Updated module registration to install target, probe, and extractor APIs.
- Added `internal/cssvisualdiff/jsapi/builders_test.go` for builder chaining, validation errors, and wrong-parent errors.
- Added `TestCVDModuleExposesTargetProbeAndExtractorBuilders` to the repository-scanned JS verb tests.
- Marked Phase 6 tasks complete and set the active phase to Phase 7.

Why

Targets, probes, and extractors are the JS-native authoring layer for the future strict APIs. They let scripts express page targets, reusable inspection recipes, and extraction plans without YAML and without unvalidated raw object literals.

What worked

The Phase 2 Proxy infrastructure made it straightforward to add fluent chainable objects. The builders can validate immediately, return the receiver for chaining, and emit wrong-parent errors when users call methods that belong to another object type.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/jsapi -run 'Test(TargetProbeAndExtractorBuilders|BuilderVal

go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleExposesTargetProbeAndExtractorBu

go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff

go test ./... -count=1
```

What didn't work

The first builder unit test failed because Goja exported arrays from Go-backed maps as `[]string`, while the test expected `[]any`:

```
Error: Not equal:
expected: []interface {}([]interface {}{"color", "font-size"})
actual  : []string([]string{"color", "font-size"})
```

I fixed the test to assert `[]string` for Go-backed builder output.

The repository-scanned builder smoke initially failed when passing `cvd.viewport.mobile()` into `.viewport(...)`:

```
promise rejected: TypeError: cvd.target.viewport: expected positive width and height, got wi
```

The cause was that `vm.ExportTo` did not decode the lowerCamel map returned by `lowerViewport(...)` into `config.Viewport` as expected. I changed `viewportFromCall(...)` to route object decoding through the JSON-based `decodeInto[config.Viewport](args[0].Export())`, matching the rest of the JS adapter behavior.

What I learned

Go-backed builder `.build()` results may export Go slices as typed slices, not generic JavaScript `[]any`, in Go unit tests. Repository-scanned JS integration tests remain important because they exercise the real JavaScript-facing shape.

The existing JSON codec helper is a safer path for lowerCamel object decoding than direct `ExportTo(...)` when target structs do not have JSON tags.

What was tricky to build

The main design tension is that Phase 6 needs extractor builders before the service-layer `ExtractorSpec` exists. I kept extractor handles lightweight and JSAPI-local for now, with `.build()` producing plain serializable values. Phase 7 should formalize the extraction service structs and update extractor unwrapping as needed.

Another tricky point is registry lifetime. The builders use `newProxyValue(...)` with an internal registry just like locators. That is enough for fluent method trapping and `.build()` now. Phase 7 strict unwrapping should introduce module-level state/registry if `cvd.extract(...)` needs to unwrap builder/locator handles across API calls.

What warrants a second pair of eyes

- Whether `cvd.viewport.desktop/tablet/mobile` should be considered stable public names or examples only.
- Whether `.required()` should default to `true` when called with no argument. It currently does.
- Whether extractor kinds should use `computedStyle` or a shorter `style/styles` name in the eventual service schema.
- Whether Phase 7 should convert local extractor handles to service-native `ExtractorSpec` values instead of retaining JSAPI-local maps.

What should be done in the future

Phase 7 should define service extraction specs/results, introduce strict unwrapping of locator and extractor Proxy handles, and implement `cvd.extract(locator, extractors)`.

Code review instructions

Start with:

```
internal/cssvisualdiff/jsapi/target.go
internal/cssvisualdiff/jsapi/probe.go
internal/cssvisualdiff/jsapi/extractor.go
internal/cssvisualdiff/jsapi/builder_helpers.go
```

Then review registration and tests:

```
internal/cssvisualdiff/jsapi/module.go
internal/cssvisualdiff/jsapi/builders_test.go
internal/cssvisualdiff/verbcli/command_test.go
```

Validation commands:

```
go test ./internal/cssvisualdiff/jsapi -run 'Test(TargetProbeAndExtractorBuilders|BuilderVal
go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleExposesTargetProbeAndExtractorBu
```

```
go test ./... -count=1
```

Technical details

Example supported JS:

```
const target = cvd.target("booking")
  .url("http://example.test/booking")
  .viewport(cvd.viewport.mobile())
  .waitMs(25)
  .root("#app")

const probe = cvd.probe("cta")
  .selector("#cta")
  .required()
  .text()
  .bounds()
  .styles(["color"])
  .attributes(["id"])

const extractor = cvd.extractors.attributes(["id", "class"])
```

Step 13: Complete Phase 7 strict `cvd.extract(locator, extractors)`

I completed Phase 7 by adding service-level extraction specs/results and exposing strict `cvd.extract(locator, extractors)` in the JavaScript API. This is the first strict API that requires Go-backed Proxy values across API calls: the locator must come from `page.locator(...)`, and extractors must come from `cvd.extractors.*(...)`.

This phase turns the lower-level API into a useful composition surface. Users can now query a page-bound element and choose exactly which facts to extract without creating inspect artifacts, while raw object inputs are rejected with explicit type guidance.

Prompt Context

User prompt (verbatim): "phase 7"

Assistant interpretation: Implement Phase 7 from the ticket: define service extraction specs/results, implement extraction from locator plus extractors, expose strict `cvd.extract(...)`, validate raw-object rejection, update docs/diary/tasks, and commit.

Inferred user intent: Make the new locator and extractor builders work together as a strict lower-level extraction API.

What I did

- Added `internal/cssvisualdiff/service/extract.go`.
- Defined:
  - `ExtractorKind`
  - `ExtractorSpec`
  - `ElementSnapshot`
- Implemented `service.ExtractElement(...)` over a `LocatorSpec` and list of `ExtractorSpec` values.
- Added `internal/cssvisualdiff/service/extract_test.go` covering:
  - multiple extractors on one element,
  - missing selector behavior,
  - invalid selector error behavior.
- Updated `internal/cssvisualdiff/jsapi/proxy.go` so `newProxyValue(..., nil, ...)` uses a package-level default registry instead of creating isolated one-off registries.
- Updated `extractorHandle` to convert to `service.ExtractorSpec`.
- Added `internal/cssvisualdiff/jsapi/extract.go` with:
  - `installExtractAPI(...)`
  - `unwrapExtractorList(...)`
  - `lowerElementSnapshot(...)`
- Registered `cvd.extract(...)` from the native module.
- Added repository-scanned JS verb tests:
  - `TestCVDModuleExtractsFromLocatorWithExtractorHandles`
  - `TestCVDModuleExtractRejectsRawObjects`
- Marked Phase 7 tasks complete and set the active phase to Phase 8.

Why

The design requires lower-level APIs to be strict and Go-backed, not raw-object permissive. `cvd.extract(...)` is the first place where that matters because it receives values created elsewhere: locators from pages and extractor handles from `cvd.extractors`. The Proxy registry now enables that strict unwrapping.

What worked

The Phase 4 DOM service functions made `ExtractElement(...)` straightforward. The Phase 2 Proxy registry could be reused with one change: proxies created with a nil registry now bind to a shared default registry, so strict APIs can unwrap them later.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/service -run 'TestExtractElement' -count=1

go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleExtract' -count=1

go test ./internal/cssvisualdiff/service ./internal/cssvisualdiff/jsapi ./internal/cssvisual
```

```
go test ./... -count=1
```

What didn't work

No failing test occurred after the first implementation pass for Phase 7. The main design issue discovered during implementation was that previous locator/extractor proxies were created with isolated registries when callers passed `nil`. That would make cross-call strict unwrapping impossible. I fixed this by introducing a package-level default registry.

What I learned

Strict APIs need a shared identity boundary. A local per-proxy registry is fine for method trapping, but it is not enough when a top-level function such as `cvd.extract(...)` must prove that a value came from `page.locator(...)` or `cvd.extractors.*(...)`.

The current `ElementSnapshot` intentionally omits absent fields from the JS result unless an extractor was requested and produced data. Boolean fields use pointers internally so `exists: false` and omitted `exists` can be distinguished.

What was tricky to build

The tricky part was preserving the Go-backed Proxy requirement without over-engineering module state. I used a package-level default registry as the smallest step that makes strict unwrapping work. A future cleanup may replace it with per-runtime module state, but this phase keeps the implementation simple and covered by integration tests.

Another subtle point is invalid selectors. `LocatorStatus(...)` returns a status with an error string rather than a Go error. `ExtractElement(...)` promotes that status error into a Go error for strict `exists`/`visible` extraction so malformed selectors reject the Promise.

What warrants a second pair of eyes

- Whether the package-level default Proxy registry is acceptable or should become per-runtime state before this API is considered stable.
- Whether `ElementSnapshot` should always include requested fields even when empty, especially for `text: ""` and missing selectors.
- Whether extractor default text options should normalize/trim text; currently extractor text defaults to normalized/trimmed through `extractorHandle.toSpec()`.

What should be done in the future

Phase 8 should implement strict `cvd.snapshot(page, probes, options)` and likely reuse `service.ExtractElement(...)` for each probe. If per-runtime state is introduced, Phase 8 may also be a good time to clean up the registry shape.

Code review instructions

Start with service extraction:

```
internal/cssvisualdiff/service/extract.go
internal/cssvisualdiff/service/extract_test.go
```

Then review JS strict API plumbing:

```
internal/cssvisualdiff/jsapi/proxy.go
internal/cssvisualdiff/jsapi/extractor.go
internal/cssvisualdiff/jsapi/extract.go
internal/cssvisualdiff/jsapi/module.go
```

Then review integration tests:

```
internal/cssvisualdiff/verbcli/command_test.go
```

Validation commands:

```
go test ./internal/cssvisualdiff/service -run 'TestExtractElement' -count=1
go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleExtract' -count=1
go test ./... -count=1
```

Technical details

Example supported JS:

```
const snapshot = await cvd.extract(page.locator("#cta"), [
  cvd.extractors.exists(),
  cvd.extractors.visible(),
  cvd.extractors.text(),
  cvd.extractors.bounds(),
  cvd.extractors.computedStyle(["color"]),
  cvd.extractors.attributes(["id", "class"]),
])
```

Raw object locator inputs are rejected:

```
await cvd.extract({ selector: "#cta" }, [cvd.extractors.text()])
```

with an error containing:

```
css-visual-diff.extract: expected cvd.locator
```

Step 14: Complete Phase 8 strict `cvd.snapshot(page, probes, options)`

I completed Phase 8 by adding service-level page snapshots and exposing strict `cvd.snapshot(page, probes, options)` in the JavaScript API. The new API requires a Go-backed page handle and Go-backed `cvd.probe(...)` builders, rejecting raw probe objects with explicit guidance.

This phase builds directly on Phase 7. A snapshot is now a batch of probe recipes evaluated against one already-loaded page, with each probe producing a plain serializable element snapshot. It is intentionally in-memory and does not write inspect artifacts.

Prompt Context

User prompt (verbatim): "phase 8"

Assistant interpretation: Implement Phase 8 from the ticket: add strict page snapshot support using Go-backed page/probe values, validate raw-object rejection and result shape, update docs/diary/tasks, and commit.

Inferred user intent: Add the next lower-level composition primitive after `cvd.extract(...)`: snapshot multiple probe builders against one page.

What I did

- Added `internal/cssvisualdiff/service/snapshot.go`.
- Defined:
  - `SnapshotProbeSpec`
  - `ProbeSnapshot`
  - `PageSnapshot`
- Implemented `service.SnapshotPage(...)` using `service.ExtractElement(...)` for each probe.
- Added `internal/cssvisualdiff/service/snapshot_test.go` covering:
  - successful multi-probe snapshots,
  - optional probe error recording.
- Marked JS page objects with the shared Proxy registry id so strict APIs can unwrap them as `cvd.page` values.
- Updated `probeBuilder` to retain service-native extractor specs in addition to `.build()` debug output.
- Added `internal/cssvisualdiff/jsapi/snapshot.go` with:
  - `installSnapshotAPI(...)`
  - `unwrapSnapshotProbes(...)`
  - `lowerPageSnapshot(...)`
- Registered `cvd.snapshot(...)` from the native module.
- Added repository-scanned JS verb tests:
  - `TestCVDModuleSnapshotsPageWithProbeBuilders`
  - `TestCVDModuleSnapshotRejectsRawObjects`

- Marked Phase 8 tasks complete and set the active phase to Phase 9.

Why

`cvd.snapshot(...)` is the page-level counterpart to `cvd.extract(...)`. It lets users define reusable probe recipes and evaluate them together against a page, without writing inspect artifacts and without using YAML or raw JS object probes.

What worked

The service implementation could reuse `ExtractElement(...)`, so snapshot behavior remains aligned with strict extraction. The page serialization guard ensures all snapshot work runs exclusively on the page while the Promise API remains async.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/service -run 'TestSnapshotPage' -count=1

go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleSnapshot' -count=1

go test ./internal/cssvisualdiff/service ./internal/cssvisualdiff/jsapi ./internal/cssvisual

go test ./... -count=1
```

What didn't work

No failing test occurred after the first Phase 8 implementation pass. The main design compromise is that existing page objects are still ordinary Goja objects, not full Proxy objects, but they now carry the internal registry id needed for strict unwrapping as `cvd.page`.

What I learned

Strict API unwrapping can be introduced incrementally. Locator, probe, and extractor values are full Proxies. Page values are currently object wrappers, but tagging them with the same registry mechanism lets `cvd.snapshot(...)` enforce that they came from this API instead of accepting raw page-like objects.

Probe builders now need to track both human/debug `.build()` output and service-native extraction specs. Keeping both avoids parsing plain build output back into service specs.

What was tricky to build

The tricky part was the page-handle boundary. A full conversion of page wrappers to Proxies would be a larger behavior-changing refactor. For Phase 8, I chose the minimal strict-unwrapping step: bind the existing page object into

the registry under `cvd.page`. That preserves current method behavior while allowing strict APIs to reject raw objects.

Another subtle point is optional probe errors. `SnapshotPage(...)` records an error in the probe result for non-required probes but returns immediately for required probe errors. This gives future authoring workflows a structured non-strict mode without weakening strict required probes.

What warrants a second pair of eyes

- Whether page wrappers should become full Proxies in a cleanup phase rather than registry-tagged objects.
- Whether `cvd.snapshot(...)` should accept options now, or wait until artifact writing / strictness options are designed.
- Whether page snapshot result shape should include target metadata, created timestamps, or catalog integration fields.

What should be done in the future

Phase 9 should add diff/report/write primitives that can operate on the plain snapshots returned by `cvd.snapshot(...)`.

Code review instructions

Start with service snapshot code:

```
internal/cssvisualdiff/service/snapshot.go
internal/cssvisualdiff/service/snapshot_test.go
```

Then review JS strict API plumbing:

```
internal/cssvisualdiff/jsapi/snapshot.go
internal/cssvisualdiff/jsapi/module.go
internal/cssvisualdiff/jsapi/probe.go
internal/cssvisualdiff/jsapi/module.go
```

Then review integration tests:

```
internal/cssvisualdiff/verbcli/command_test.go
```

Validation commands:

```
go test ./internal/cssvisualdiff/service -run 'TestSnapshotPage' -count=1
go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleSnapshot' -count=1
go test ./... -count=1
```

Technical details

Example supported JS:

```
const snapshot = await cvd.snapshot(page, [
  cvd.probe("cta").selector("#cta").required().text().styles(["color"]),
  cvd.probe("copy").selector("#copy").text(),
])
```

Raw object probes are rejected:

```
await cvd.snapshot(page, [{ name: "cta", selector: "#cta" }])
```

with an error containing:

```
css-visual-diff.snapshot: expected array of cvd.probe() builders
```

Step 15: Complete Phase 9 diff/report/write primitives

I completed Phase 9 by adding the first snapshot diff, report, and write primitives. The new service layer can compare plain snapshot-like values structurally, render a small Markdown report, and the JavaScript API exposes synchronous diff/report construction plus Promise-returning file writes.

This is intentionally a deterministic first cut rather than a full CSS-aware visual regression diff engine. It gives scripts a usable in-memory comparison path for snapshots produced by `cvd.snapshot(...)` and creates extension points for future CSS normalization/tolerance rules.

Prompt Context

User prompt (verbatim): "phase 9"

Assistant interpretation: Implement Phase 9 from the ticket: add service diff/report primitives, expose `cvd.diff`, `cvd.report`, and `cvd.write.*`, validate with tests, update docs/diary/tasks, and commit.

Inferred user intent: Complete the lower-level snapshot comparison/reporting layer before final docs and delivery.

What I did

- Added `internal/cssvisualdiff/service/diff.go`.
- Defined:
  - `DiffOptions`
  - `DiffChange`
  - `SnapshotDiff`
- Implemented:
  - `DiffValues(...)`
  - `RenderDiffMarkdown(...)`
- Added `internal/cssvisualdiff/service/diff_test.go` covering changed/equal/ignored paths and Markdown rendering.
- Added `internal/cssvisualdiff/jsapi/diff.go`.
- Registered:

  - `cvd.diff(before, after, options)`
  - `cvd.report(diff).markdown()`
  - `cvd.report(diff).writeMarkdown(path)`
  - `cvd.write.json(path, value)`
  - `cvd.write.markdown(path, markdown)`
- Added `TestCVDModuleDiffReportAndWritePrimitives` to the repository-scanned JS verb tests.
- Marked Phase 9 tasks complete and set the active phase to Phase 10.

Why

The lower-level API now has locators, extraction, and snapshots. Scripts need a way to compare those snapshot values without returning to YAML workflows or file-artifact inspect modes. Diff/report/write primitives provide that final in-memory and file-output bridge.

What worked

The diff service normalizes values through JSON first, so it can compare Go structs, maps, and JS-exported values consistently. The path-based diff is deterministic because map keys are sorted before traversal.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/service -run 'TestDiff' -count=1

go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleDiffReportAndWritePrimitives' -c

go test ./internal/cssvisualdiff/service ./internal/cssvisualdiff/jsapi ./internal/cssvisual

go test ./... -count=1
```

What didn't work

The first repository-scanned JS smoke used:

```
const path = require("path")
```

That failed because the embedded runtime does not provide a Node `path` module:

```
promise rejected: GoError: Invalid module
```

I changed the test script to build paths with `outDir + "/diff.json"` and `outDir + "/diff.md"`.

The second failure was that `ignorePaths` did not apply at first because the JS adapter decoded options using direct `ExportTo(...)`, which did not populate the JSON-tagged `DiffOptions` field as expected. The symptom was

`ignoredEqual` being `false` instead of `true`. I fixed `cvd.diff(...)` option decoding to use the JSON-based `decodeInto[service.DiffOptions](call.Argument(2).Export())`, consistent with the other lowerCamel JS adapters.

What I learned

The go-go-goja runtime used by jsverbs should not be treated as a full Node runtime. Tests and examples should avoid `require("path")` unless that module is explicitly registered.

For lowerCamel JS options, the JSON-based adapter remains the safest decoding path. Direct Goja `ExportTo(...)` can miss fields when Go structs rely on JSON tags rather than exported field names.

What was tricky to build

The tricky part was choosing a useful first diff scope without overbuilding. I kept the service as a structural JSON diff with exact path ignores. That is enough to compare snapshots now and leaves CSS-specific normalization/tolerance for a later improvement.

Another subtle point is output shape: Go service structs use `change_count`, but the JS API lowers this to `changeCount`, keeping JS-facing fields lowerCamel.

What warrants a second pair of eyes

- Whether path ignore syntax (`results[0].snapshot.text`) is the right long-term format.
- Whether slice length changes should be reported as one whole-slice change or per-index add/remove changes.
- Whether Markdown output is too minimal for operator-facing reports.
- Whether `cvd.write.json` should create parent directories automatically.

What should be done in the future

Phase 10 should add public docs, examples, smoke scripts, final validation, and optionally regenerate/upload the implementation PDF.

Code review instructions

Start with service diff code:

```
internal/cssvisualdiff/service/diff.go
internal/cssvisualdiff/service/diff_test.go
```

Then review JS exports:

```
internal/cssvisualdiff/jsapi/diff.go
internal/cssvisualdiff/jsapi/module.go
```

Then review integration coverage:

`internal/cssvisualdiff/verbcli/command_test.go`

Validation commands:

```
go test ./internal/cssvisualdiff/service -run 'TestDiff' -count=1
go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleDiffReportAndWritePrimitives' -c
go test ./... -count=1
```

Technical details

Example supported JS:

```
const diff = cvd.diff(beforeSnapshot, afterSnapshot, {
  ignorePaths: ["results[0].snapshot.bounds.x"],
})
const markdown = cvd.report(diff).markdown()
await cvd.write.json("out/diff.json", diff)
await cvd.report(diff).writeMarkdown("out/diff.md")
```

Step 16: Complete Phase 10 embedded help, textbook guide, examples, and smokes

I completed the documentation and delivery phase by moving the standalone JavaScript docs into embedded Glazed help entries and adding a long-form textbook-style guide for using `css-visual-diff` as a pixel-accuracy feedback loop while building websites. The CLI now carries the main JavaScript API documentation inside the binary, so users can discover it with `css-visual-diff help ...` instead of needing separate markdown files in `docs/`.

The new guide teaches the workflow in layers: browser page, locator, probe, extractor, snapshot, diff, report, and catalog/inspect artifacts. It is intentionally written as teaching material rather than terse reference documentation, because the lower-level API is most useful when users understand why locators and probes are different and how to build repeatable scripts from small visual questions.

Prompt Context

User prompt (verbatim): "phase 10:

move all docs to be glazed help entries embeddedin the binary. Also, make a nice detailed user guide in a textbook style on how to use css visual diff to build up scripts to help build up website with pixel accuracy.

(read the textbook skill, be verbose, here's a great example for a similar workflow: /home/manuel/code/wesen/obsidian-vault/Projects/2026/04/21/PROJ - go-minitrace - JS Commands and Structured Query Catalog PR #6.md"

Assistant interpretation: Finish Phase 10 by converting public docs into embedded Glazed help entries, writing a detailed textbook-style user guide for pixel-accuracy scripting, adding examples/smokes, validating, and preparing final documentation/delivery.

Inferred user intent: The user wants the new API to be documented where operators actually use it: inside the CLI help system, with enough teaching prose that developers can build scripts for pixel-perfect UI work rather than merely copy method signatures.

What I did

- Read the textbook authoring skill and Glazed help page authoring skill.
- Read the provided go-minitrace project note as a model for narrative, architecture-oriented documentation.
- Moved standalone docs into embedded Glazed help entries:
  - `docs/js-api.md` -> `internal/cssvisualdiff/doc/topics/javascript-api.md`
  - `docs/js-verbs.md` -> `internal/cssvisualdiff/doc/topics/javascript-verbs.md`
- Added Glazed help frontmatter to both moved documents.
- Updated the JavaScript API help entry with Phase 5-9 lower-level APIs:
  - `page.locator(...)`
  - locator methods
  - `cvd.viewport(...)`
  - `cvd.target(...)`
  - `cvd.probe(...)`
  - `cvd.extractors.*`
  - `cvd.extract(...)`
  - `cvd.snapshot(...)`
  - `cvd.diff(...)`
  - `cvd.report(...)`
  - `cvd.write.*`
- Added a textbook-style tutorial:
  - `internal/cssvisualdiff/doc/tutorials/pixel-accuracy-scripting-guide.md`
- Updated `README.md` to point users to embedded help:
  - `css-visual-diff help javascript-api`
  - `css-visual-diff help javascript-verbs`
  - `css-visual-diff help pixel-accuracy-scripting-guide`
- Added an external lower-level verb example:
  - `examples/verbs/low-level-inspect.js`
- Updated `examples/verbs/README.md` with the new lower-level example.
- Added ticket smoke scripts:
  - `scripts/001-help-entries-smoke.sh`
  - `scripts/002-low-level-verb-binary-smoke.sh`
- Ran embedded help smoke checks and the lower-level binary smoke.
- Ran the full Go test suite.

Why

Standalone docs are easy to miss. This project already has a Glazed help system embedded in the binary, so the correct user-facing home for durable docs is `css-visual-diff help ...`. Moving the docs there makes the API discoverable offline, inside the exact tool users are running.

The textbook-style guide fills a different role than the API reference. The reference says what methods exist. The guide explains how to think: start with page state, use locators for live questions, turn stable questions into probes, extract facts, snapshot repeatable recipes, diff snapshots, and write evidence.

What worked

The existing `internal/cssvisualdiff/doc/doc.go` already embedded `topics`, `examples`, and `tutorials`, and `cmd/css-visual-diff/main.go` already wired `help_cmd.SetupCobraRootCommand(...)`. That meant the conversion mainly required moving markdown files, adding frontmatter, and validating that the help slugs rendered.

Validation commands that passed:

```
go test ./internal/cssvisualdiff/doc ./cmd/css-visual-diff -count=1

go run ./cmd/css-visual-diff help javascript-api >/tmp/cssvd-help-jsapi.txt

go run ./cmd/css-visual-diff help javascript-verbs >/tmp/cssvd-help-jsverbs.txt

go run ./cmd/css-visual-diff help pixel-accuracy-scripting-guide >/tmp/cssvd-help-pixel.txt

ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/scripts/001-

ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/scripts/002-

go test ./... -count=1
```

The help smoke confirmed the embedded slugs render. The lower-level binary smoke built the binary, served a local HTML page, ran the external `examples low-level inspect` verb, and verified `element.json` / `snapshot.json` outputs.

What didn't work

No final validation command failed. One small issue was caught during review before validation: the moved JavaScript API help entry initially had a YAML title with unescaped inner quotes:

```
Title: "JavaScript API: require("css-visual-diff")"
```

I fixed it by using single-quoted YAML:

`Title: 'JavaScript API: require("css-visual-diff")'`

What I learned

The binary already had the right help integration. The important change was not Go wiring; it was content placement and quality. Keeping docs in `internal/cssvisualdiff/doc/topics` and `tutorials` means the docs travel with the CLI and can be tested with normal binary smoke scripts.

The textbook guide benefits from the API design decisions made earlier. The locator/probe distinction is teachable because the API enforces it. Wrong-parent errors are not just developer ergonomics; they make the documentation's conceptual model executable.

What was tricky to build

The tricky part was interpreting "move all docs" without losing useful documentation history. I moved the two standalone JavaScript docs into embedded help entries and updated the README to point at `css-visual-diff help ...`. Existing help entries under `internal/cssvisualdiff/doc` were already embedded and remained in place.

Another subtle point is that `cvd.write.json(...)` does not create parent directories. The lower-level example and smoke assume the output directory exists. That is acceptable for now, but the guide and future examples should make this visible.

What warrants a second pair of eyes

- Whether the textbook guide is the right length and style for embedded CLI help, or whether it should be split into multiple tutorial entries later.
- Whether `javascript-api` should be split into high-level and lower-level reference pages now that it is long.
- Whether `cvd.write.json(...)` should create parent directories automatically before Phase 10 is considered polished.

What should be done in the future

- Consider adding a second tutorial focused only on CI gating and catalogs.
- Consider adding shorter example help entries for `cvd.extract(...)`, `cvd.snapshot(...)`, and `cvd.diff(...)` once users try the new API.
- Consider regenerating TypeScript-style declarations from the JS API reference after the API stabilizes.

Code review instructions

Review embedded docs first:

```
internal/cssvisualdiff/doc/topics/javascript-api.md
internal/cssvisualdiff/doc/topics/javascript-verbs.md
internal/cssvisualdiff/doc/tutorials/pixel-accuracy-scripting-guide.md
```

Then review examples and smoke scripts:

```
examples/verbs/low-level-inspect.js
examples/verbs/README.md
ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/scripts/001-
ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/scripts/002-
```

Validation commands:

```
go test ./... -count=1
ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/scripts/001-
ttmp/2026/04/24/CSSVD-FLEX-JS-API--design-a-flexible-lower-level-javascript-api/scripts/002-
```

Technical details

The new help slugs are:

```
css-visual-diff help javascript-api
css-visual-diff help javascript-verbs
css-visual-diff help pixel-accuracy-scripting-guide
```

Tasks

Tracking rules

- Work task-by-task and keep this file current before moving to the next phase.
- Keep implementation commits focused. Prefer one commit per phase, or one commit per risky subphase if a phase is large.
- Update `reference/01-investigation-diary.md` after each meaningful implementation step, including failed commands and validation output.
- Keep `changelog.md` aligned with completed milestones and commit hashes.
- Do not mark a phase complete until its validation commands pass.
- Current active implementation phase: **Complete — implementation, embedded docs, examples, and smokes delivered**.

Phase 0 — Ticket, design, and baseline bookkeeping

- ☒ Create ticket workspace for the flexible lower-level JavaScript API design.
- ☒ Read current user-facing docs: `README.md`, `docs/js-api.md`, and `docs/js-verbs.md`.
- ☒ Read the prior Goja/jsverbs implementation guide for historical context.
- ☒ Read current implementation files for config schema, Goja module adapters, repository-scanned verbs, and reusable services.
- ☒ Read the Discord bot Goja Proxy UI DSL references and implementation files.
- ☒ Write the detailed intern-facing analysis/design/implementation guide.
- ☒ Update the design guide with the `internal/cssvisualdiff/jsapi` subpackage plan.
- ☒ Update the design guide with the Go-backed Proxy builder/handle model.
- ☒ Remove workflow builder from the implementation plan; use ordinary JavaScript functions/loops for orchestration.
- ☒ Generate local PDF bundles.
- ☒ Upload the updated design guide bundle to reMarkable as `/ai/2026/04/24/cssvd-flex-api-updated`.
- ☒ Commit ticket/design docs: `17240de docs: add flexible js api implementation ticket`.

Phase 1 — Move native API into `internal/cssvisualdiff/jsapi` with no behavior changes

Goal: split the native `require("css-visual-diff")` module implementation out of `internal/cssvisualdiff/dsl` while preserving all existing JS API behavior.

Phase 1.1 — Orientation and safety checks

- ☒ Read the current diary before implementation.

- ☒ Confirm current branch and git status before editing.
- ☒ Commit existing ticket docs before changing code.
- ☒ Run a baseline targeted test set before completing the phase:
  - `go test ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff`
  - `go test ./internal/cssvisualdiff/service`

Phase 1.2 — Mechanical package move

- ☒ Create `internal/cssvisualdiff/jsapi/`.
- ☒ Move `internal/cssvisualdiff/dsl/cvd_module.go` to `internal/cssvisualdiff/jsapi/module.go`.
- ☒ Move `internal/cssvisualdiff/dsl/catalog_adapter.go` to `internal/cssvisualdiff/jsapi/catalo`
- ☒ Move `internal/cssvisualdiff/dsl/config_adapter.go` to `internal/cssvisualdiff/jsapi/config`
- ☒ Change moved files from `package dsl` to `package jsapi`.
- ☒ Rename `registerCVDModule(...)` to exported `Register(...)`.
- ☒ Add `internal/cssvisualdiff/jsapi/codec.go` or otherwise provide local `decodeInto(...)` helpers needed by moved adapters.
- ☒ Keep `dsl`-owned helpers such as `toPlainValue(...)` available for existing `diff` / `report` modules.

Phase 1.3 — Rewire runtime registration

- ☒ Update `internal/cssvisualdiff/dsl/registrar.go` to import `internal/cssvisualdiff/jsapi`.
- ☒ Replace `registerCVDModule(ctx, reg)` with `jsapi.Register(ctx, reg)`.
- ☒ Confirm there is no import cycle between `dsl` and `jsapi`.
- ☒ Confirm `jsapi` imports only service/config/engine/goja dependencies, not `dsl`.

Phase 1.4 — Compilation and formatting

- ☒ Run `gofmt -w` on touched Go files.
- ☒ Run targeted compile/tests:
  - `go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl`
  - `go test ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff`
- ☒ Fix any package-private symbol or import errors from the move.

Phase 1.5 — Behavior-preservation validation

- ☒ Run existing JS module integration tests:
  - `go test ./internal/cssvisualdiff/verbcli -run 'TestLazyCommand.*(CVD|Catalog|Error| -count=1`
- ☒ Run service tests that existing adapters depend on:
  - `go test ./internal/cssvisualdiff/service -count=1`
- ☒ Run broader validation:

  - `go test ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
- ☒ Run full suite if targeted validation is clean:
  - `go test ./... -count=1`

Phase 1.6 — Documentation and commit

- ☒ Update diary with what moved, validation output, and any failures.
- ☒ Update changelog with Phase 1 result.
- ☒ Mark Phase 1 tasks complete.
- ☒ Commit Phase 1 code and docs together if they describe the same completed implementation milestone.

Phase 2 — Add Proxy infrastructure and typed unwrapping helpers

Goal: establish the Go-backed object model before adding new public lower-level APIs.

Phase 2.1 — Proxy helper design

- ☒ Add `internal/cssvisualdiff/jsapi/proxy.go`.
- ☒ Define a `ProxyRegistry` or equivalent identity mechanism for mapping Goja Proxy values back to Go structs.
- ☒ Define shared method metadata: owner name, available methods, and optional wrong-parent hints.
- ☒ Add helper for constructing Proxy-backed objects with a Go backing value.

Phase 2.2 — Error helpers

- ☒ Add unknown-method error helper with available method list and optional did-you-mean suggestion.
- ☒ Add wrong-parent error helper for methods that exist on another object type.
- ☒ Add type-mismatch error helper for strict APIs expecting Go-backed handles/builders.
- ☒ Ensure errors are surfaced as JS-visible typed `CvdError` or a more specific subclass where appropriate.

Phase 2.3 — Typed unwrapping

- ☒ Add `internal/cssvisualdiff/jsapi/unwrap.go`.
- ☒ Add unwrap helpers for browser/page/catalog handles if converted in this phase.
- ☒ Add planned unwrap helpers or placeholders for locator/probe/extractor handles.

- ☒ Ensure unwrap errors include operation names and migration hints.

Phase 2.4 — Tests and validation

- ☒ Add Goja/runtime tests for unknown method errors.
- ☒ Add Goja/runtime tests for wrong-parent errors.
- ☒ Add Goja/runtime tests for raw-object type mismatch errors.
- ☒ Run `go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/verbcli -count=1`.
- ☒ Update diary/changelog and commit Phase 2.

Phase 3 — Add per-page operation serialization

Goal: make lower-level locator APIs safe when users call several page operations concurrently.

- ☒ Add a per-page mutex or queue to the page handle/state.
- ☒ Wrap `goto`, `prepare`, `preflight`, `inspect`, `inspectAll`, and `close` in the per-page operation guard.
- ☒ Ensure operations on different pages can still run concurrently.
- ☒ Add a test using `Promise.all` on one page that proves calls complete without races/panics/hangs.
- ☒ Add a test using two pages to ensure page-level isolation.
- ☒ Run targeted and full tests.
- ☒ Update diary/changelog and commit Phase 3.

Phase 4 — Implement service DOM locator primitives

Goal: add Go service functions for locator status/text/html/bounds/attributes/style without importing Goja.

- ☒ Add `internal/cssvisualdiff/service/dom.go`.
- ☒ Define `LocatorSpec`, `TextOptions`, `ElementHTML`, and related option/result structs if needed.
- ☒ Implement `LocatorStatus` reusing selector readiness logic from `PreflightProbes` where possible.
- ☒ Implement `LocatorText`.
- ☒ Implement `LocatorHTML`.
- ☒ Implement `LocatorBounds`.
- ☒ Implement `LocatorAttributes`.
- ☒ Implement `LocatorComputedStyle` reusing `EvaluateStyle` where possible.
- ☒ Add `internal/cssvisualdiff/service/dom_test.go` with existing/missing/hidden/invalid selector cases.
- ☒ Run `go test ./internal/cssvisualdiff/service -count=1`.
- ☒ Update diary/changelog and commit Phase 4.

Phase 5 — Expose `page.locator()` and locator methods as Go-backed Proxy handles

Goal: ship the first visible lower-level JS API.

- ☒ Add `internal/cssvisualdiff/jsapi/locator.go`.
- ☒ Add `page.locator(selector)` as a synchronous method returning a Go-backed Proxy handle.
- ☒ Add Promise-returning `locator.status()`.
- ☒ Add Promise-returning `locator.exists()`.
- ☒ Add Promise-returning `locator.visible()`.
- ☒ Add Promise-returning `locator.text(options)`.
- ☒ Add Promise-returning `locator.bounds()`.
- ☒ Add Promise-returning `locator.computedStyle(props)`.
- ☒ Add Promise-returning `locator.attributes(names)`.
- ☒ Add wrong-parent guidance for probe/extractor methods accidentally called on locators.
- ☒ Add repository-scanned JS verb tests for locator methods.
- ☒ Update docs/examples if public API is considered ready.
- ☒ Update diary/changelog and commit Phase 5.

Phase 6 — Add Go-backed target/probe/extractor builders

Goal: make YAML replacement ergonomic and strongly validated.

Phase 6.1 — Target and viewport builders

- ☒ Add `internal/cssvisualdiff/jsapi/target.go`.
- ☒ Implement `cvd.target(name)` as a Go-backed `TargetBuilder` Proxy.
- ☒ Implement `cvd.viewport(width, height)` and named viewport helpers if included in the first cut.
- ☒ Add builder validation for URL, viewport dimensions, wait time, root selector, and prepare settings.

Phase 6.2 — Probe builders

- ☒ Add `internal/cssvisualdiff/jsapi/probe.go`.
- ☒ Implement `cvd.probe(name)` as a Go-backed `ProbeBuilder` Proxy.
- ☒ Add `.selector(...)`, `.required(...)`, `.source(...)`, `.text()`, `.bounds()`, `.styles(...)`, `.attributes(...)`.
- ☒ Add helpful errors for `.style(...)` vs `.styles(...)` and other common mistakes.

Phase 6.3 — Extractor builders

- ☒ Add `internal/cssvisualdiff/jsapi/extractor.go`.
- ☒ Implement `cvd.extractors.exists()`.

- ☒ Implement `cvd.extractors.visible()`.
- ☒ Implement `cvd.extractors.text()`.
- ☒ Implement `cvd.extractors.bounds()`.
- ☒ Implement `cvd.extractors.computedStyle(props)`.
- ☒ Implement `cvd.extractors.attributes(names)`.

Phase 6.4 — Validation

- ☒ Add tests for builder chaining.
- ☒ Add tests that strict APIs can unwrap builders without requiring `.build()`.
- ☒ Add tests for invalid builder arguments.
- ☒ Update diary/changelog and commit Phase 6.

Phase 7 — Add strict `cvd.extract(locator, extractors)`

Goal: composable extraction with typed Go-backed inputs.

- ☒ Add `internal/cssvisualdiff/service/extract.go`.
- ☒ Define `ExtractorSpec`, `ElementSnapshot`, extraction options, and error/status result shapes.
- ☒ Implement extraction from a single locator using multiple extractors.
- ☒ Expose `cvd.extract(locator, extractors, options)` in `jsapi`.
- ☒ Require `LocatorHandle` and `ExtractorHandle` Proxy values; reject raw JS objects with helpful migration hints.
- ☒ Add tests for multiple facts from one locator.
- ☒ Add tests for missing selector behavior.
- ☒ Add tests for invalid selector typed errors.
- ☒ Update diary/changelog and commit Phase 7.

Phase 8 — Add strict `cvd.snapshot(page, probes, options)`

Goal: inspect many Go-backed probe builders into an in-memory structured result without necessarily writing standard inspect artifacts.

- ☒ Define `PageSnapshot` service result shape.
- ☒ Implement snapshot orchestration over a page and multiple probe builders.
- ☒ Expose `cvd.snapshot(page, probes, options)`.
- ☒ Require a `PageHandle` Proxy and Go-backed `ProbeBuilder` values.
- ☒ Return plain JSON-serializable snapshot data.
- ☒ Add optional artifact writing only when explicitly requested.
- ☒ Add tests for strict raw-object rejection.
- ☒ Add tests for snapshot data shape stability.
- ☒ Update diary/changelog and commit Phase 8.

Phase 9 — Add diff, report, and write primitives

Goal: compare snapshots without YAML.

- ☒ Add `internal/cssvisualdiff/service/diff.go`.
- ☒ Implement deterministic structural JSON diff.
- ☒ Add initial CSS-aware normalization hooks if low risk; otherwise document as follow-up.
- ☒ Expose `cvd.diff(before, after, options)`.
- ☒ Expose `cvd.report(diff)` with Markdown rendering.
- ☒ Expose `cvd.write.json(path, value)`.
- ☒ Expose `cvd.write.markdown(path, markdown)` or equivalent.
- ☒ Add tests for equal/changed/ignored/tolerance cases.
- ☒ Update diary/changelog and commit Phase 9.

Phase 10 — Public docs, examples, smoke scripts, and delivery

Goal: make the new lower-level API usable and reviewable by operators and future implementers.

- ☒ Add embedded Glazed help coverage for the lower-level API instead of a standalone `docs/js-low-level-api.md`.
- ☒ Move `docs/js-api.md` into embedded help as `internal/cssvisualdiff/doc/topics/javascript-api.` and update it with locator/extract/snapshot/diff APIs.
- ☒ Add `examples/verbs/low-level-inspect.js`.
- ☒ Add ticket smoke scripts under `scripts/` with numeric prefixes.
- ☒ Add and run a compiled-binary smoke for the external lower-level verb.
- ☒ Run `go test ./... -count=1`.
- ☒ Run current Phase 10 binary smokes for embedded help and lower-level external verb behavior.
- ☒ Run `docmgr doctor --root ./ttmp --ticket CSSVD-FLEX-JS-API --stale-after 30`.
- ☐ Regenerate and optionally upload the updated implementation PDF to reMarkable.
- ☐ Update diary/changelog and commit final docs.

Changelog

2026-04-24

- Initial workspace created.
- Created the primary design document: `design-doc/01-flexible-javascript-api-analysis-design-a`
- Documented the current architecture from `README.md`, `docs/js-api.md`, `docs/js-verbs.md`, `internal/cssvisualdiff/config`, `internal/cssvisualdiff/dsl`, `internal/cssvisualdiff/verbcli`, and `internal/cssvisualdiff/service`.
- Proposed a layered lower-level JavaScript API: page locators, extractors, snapshots, fluent builders, diffs, reporting, workflows, and catalog integration.
- Added a phased implementation plan suitable for an intern to follow.
- Updated `tasks.md` and `reference/01-investigation-diary.md`.
- Ran `docmgr doctor --root ./ttmp --ticket CSSVD-FLEX-JS-API --stale-after 30`; after normalizing ticket topics to known vocabulary, all checks passed.
- Generated local PDF bundle at `pdf/CSSVD-FLEX-JS-API-flexible-javascript-api-design-guide.pd`
- Attempted reMarkable upload, but cloud create/upload returned `request failed with status 400`; upload was skipped for now at user request.
- Retried reMarkable upload with a short local PDF filename (`/tmp/cssvd-flex-api.pdf`); upload succeeded to `/ai/2026/04/24/cssvd-flex-api` and the remote listing was verified.

2026-04-24 — Proxy-backed JS API architecture update

- Updated the design guide to move the native `require("css-visual-diff")` implementation out of `internal/cssvisualdiff/dsl/cvd_module.go` and into a dedicated `internal/cssvisualdiff/jsapi` subpackage.
- Added a preparatory no-behavior refactor phase for moving promise/error/browser/page/catalog/config adapters into `jsapi`.
- Reworked the lower-level API design to use Go-backed Goja Proxy wrappers for live handles and DSL builders instead of raw JS object builders.
- Added method-owner, wrong-parent, unknown-method, and type-mismatch error guidance based on the Discord bot UI DSL Proxy pattern.
- Clarified that final result data should remain plain JSON-serializable data, while handles/builders/specs should be Go-backed Proxy objects.
- Clarified compatibility boundaries: existing high-level methods may continue accepting raw probe objects, while new strict lower-level APIs should require Go-backed locators/probes/extractors and reject raw objects with helpful errors.
- Removed workflow builder from the plan. Future orchestration should use ordinary JavaScript functions, loops, and branches.
- Regenerated updated local PDF bundle at `pdf/CSSVD-FLEX-JS-API-flexible-javascript-api-design`
- Uploaded updated PDF to reMarkable as `/ai/2026/04/24/cssvd-flex-api-updated`

and verified the remote listing.

2026-04-24 — Detailed implementation task breakdown

- Expanded `tasks.md` into granular phased implementation checklists so CSSVD-FLEX-JS-API can proceed task by task.
- Added explicit validation and commit checkpoints for each phase.
- Recorded that Phase 1 is the current active implementation phase and should remain a no-behavior `jsapi` package refactor.
- Committed the initial ticket/design docs before continuing implementation: `17240de docs: add flexible js api implementation ticket`.

2026-04-24 — Phase 1 and Phase 2 implementation

- Completed Phase 1 no-behavior refactor by moving the native `require("css-visual-diff")` implementation into `internal/cssvisualdiff/jsapi`.
- Updated `internal/cssvisualdiff/dsl/registrar.go` to call `jsapi.Register(ctx, reg)` while keeping `dsl` responsible for jsverbs runtime/module wiring.
- Added `internal/cssvisualdiff/jsapi/codec.go` for local JS adapter decode helpers.
- Completed Phase 2 initial Proxy infrastructure with `proxy.go`, `unwrap.go`, and `proxy_test.go`.
- Added tests for unknown-method, wrong-parent, successful unwrap, raw-object rejection, and wrong-owner rejection behavior.
- Validation passed:
  - `go test ./internal/cssvisualdiff/jsapi -count=1`
  - `go test ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff ./internal/cssvisualdiff/service -count=1`
  - `go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
  - `go test ./... -count=1`

2026-04-24 — Phase 3 page operation serialization

- Completed Phase 3 by adding a per-page mutex to `pageState` in `internal/cssvisualdiff/jsapi/module.go`.
- Routed `goto`, `prepare`, `preflight`, `inspect`, `inspectAll`, and `close` through `pageState.runExclusive(...)`.
- Added repository-scanned JS verb tests for same-page `Promise.all(...)` safety and separate-page concurrent operations.
- Validation passed:
  - `go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModule(SerializesSamePageProm -count=1`

  - `go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
  - `go test ./... -count=1`

2026-04-24 — Phase 4 service DOM locator primitives

- Completed Phase 4 by adding `internal/cssvisualdiff/service/dom.go`.
- Added service-layer locator primitives for status, text, HTML, bounds, attributes, and computed style.
- Added `internal/cssvisualdiff/service/dom_test.go` with existing, missing, hidden, invalid selector, text, HTML, bounds, attributes, and style coverage.
- Validation passed:
  - `go test ./internal/cssvisualdiff/service -run 'TestLocatorDOMPrimitives' -count=1`
  - `go test ./internal/cssvisualdiff/service -count=1`
  - `go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
  - `go test ./... -count=1`

2026-04-24 — Phase 5 `page.locator()` JavaScript API

- Completed Phase 5 by adding `internal/cssvisualdiff/jsapi/locator.go`.
- Added synchronous `page.locator(selector)` returning a Go-backed Proxy locator handle.
- Added Promise-returning locator methods for status, exists, visible, text, bounds, computedStyle, and attributes.
- Routed locator browser work through per-page operation serialization.
- Added wrong-parent Proxy guidance for probe methods accidentally called on locators.
- Added repository-scanned JS verb tests for locator method behavior and wrong-parent errors.
- Validation passed:
  - `go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModule(ExposesLocatorMethods| -count=1`
  - `go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
  - `go test ./... -count=1`

2026-04-24 — Phase 6 Go-backed target/probe/extractor builders

- Completed Phase 6 by adding Go-backed Proxy builders for targets, probes, viewports, and extractors.
- Added `internal/cssvisualdiff/jsapi/builder_helpers.go`, `target.go`, `probe.go`, and `extractor.go`.
- Registered `cvd.target`, `cvd.viewport`, `cvd.probe`, and `cvd.extractors` from the native module.
- Added builder chaining, validation, and wrong-parent tests in `internal/cssvisualdiff/jsapi/builders_test.go`.
- Added a repository-scanned JS verb smoke for the builder APIs.
- Validation passed:
  - `go test ./internal/cssvisualdiff/jsapi -run 'Test(TargetProbeAndExtractorBuilders|P -count=1`
  - `go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleExposesTargetProbeAndEx -count=1`
  - `go test ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
  - `go test ./... -count=1`

2026-04-24 — Phase 7 strict `cvd.extract(locator, extractors)`

- Completed Phase 7 by adding `internal/cssvisualdiff/service/extract.go` and `extract_test.go`.
- Added service-layer `ExtractorSpec`, `ExtractorKind`, `ElementSnapshot`, and `ExtractElement(...)`.
- Updated Proxy registration to use a shared default registry when no registry is explicitly provided, enabling strict cross-call unwrapping.
- Added `internal/cssvisualdiff/jsapi/extract.go` and registered strict `cvd.extract(locator, extractors)`.
- Updated extractor handles to convert into service-native extractor specs.
- Added repository-scanned JS verb tests for successful extraction and raw-object rejection.
- Validation passed:
  - `go test ./internal/cssvisualdiff/service -run 'TestExtractElement' -count=1`
  - `go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleExtract' -count=1`
  - `go test ./internal/cssvisualdiff/service ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
  - `go test ./... -count=1`

2026-04-24 — Phase 8 strict `cvd.snapshot(page, probes)`

- Completed Phase 8 by adding `internal/cssvisualdiff/service/snapshot.go` and `snapshot_test.go`.
- Added service-layer `SnapshotProbeSpec`, `ProbeSnapshot`, `PageSnapshot`, and `SnapshotPage(...)`.
- Registered page wrappers in the shared Proxy registry so strict APIs can unwrap `cvd.page` values.
- Updated probe builders to retain service-native extractor specs for snapshot execution.
- Added `internal/cssvisualdiff/jsapi/snapshot.go` and registered strict `cvd.snapshot(page, probes)`.
- Added repository-scanned JS verb tests for successful snapshots and raw-object probe rejection.
- Validation passed:
  - `go test ./internal/cssvisualdiff/service -run 'TestSnapshotPage' -count=1`
  - `go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleSnapshot' -count=1`
  - `go test ./internal/cssvisualdiff/service ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
  - `go test ./... -count=1`

2026-04-24 — Phase 9 diff/report/write primitives

- Completed Phase 9 by adding `internal/cssvisualdiff/service/diff.go` and `diff_test.go`.
- Added structural `DiffValues(...)`, `SnapshotDiff`, `DiffChange`, `DiffOptions`, and Markdown rendering.
- Added `internal/cssvisualdiff/jsapi/diff.go` and registered `cvd.diff`, `cvd.report`, and `cvd.write.*` helpers.
- Added repository-scanned JS verb coverage for diffing, ignored paths, Markdown report generation, and JSON/Markdown writes.
- Validation passed:
  - `go test ./internal/cssvisualdiff/service -run 'TestDiff' -count=1`
  - `go test ./internal/cssvisualdiff/verbcli -run 'TestCVDModuleDiffReportAndWritePrimi -count=1`
  - `go test ./internal/cssvisualdiff/service ./internal/cssvisualdiff/jsapi ./internal/cssvisualdiff/dsl ./internal/cssvisualdiff/verbcli ./cmd/css-visual-diff -count=1`
  - `go test ./... -count=1`

2026-04-24 — Phase 10 embedded help, textbook guide, examples, and smokes

- Moved standalone JavaScript docs into embedded Glazed help entries:
  - `internal/cssvisualdiff/doc/topics/javascript-api.md`
  - `internal/cssvisualdiff/doc/topics/javascript-verbs.md`
- Added a long-form textbook-style tutorial: `internal/cssvisualdiff/doc/tutorials/pixel-accuracy-`
- Updated the JavaScript API help entry with locator, extraction, snapshot, diff, report, and write APIs.
- Updated `README.md` to point users to embedded help slugs instead of standalone docs files.
- Added external lower-level verb example `examples/verbs/low-level-inspect.js` and documented it in `examples/verbs/README.md`.
- Added Phase 10 smoke scripts for embedded help and lower-level external verb behavior.
- Validation passed:
  - `go test ./internal/cssvisualdiff/doc ./cmd/css-visual-diff -count=1`
  - `go run ./cmd/css-visual-diff help javascript-api`
  - `go run ./cmd/css-visual-diff help javascript-verbs`
  - `go run ./cmd/css-visual-diff help pixel-accuracy-scripting-guide`
  - `scripts/001-help-entries-smoke.sh`
  - `scripts/002-low-level-verb-binary-smoke.sh`
  - `go test ./... -count=1`